



UNIVERSITY OF TRENTO

Department of Information Engineering and Computer Science

Master's Degree in
Computer Science

FINAL DISSERTATION

REFLECTOR: AGENTIC SYSTEM FOR POST-OPERATIVE SURGICAL DEBRIEFING

Supervisor

Paolo Giorgini

Student

Lorenzo Fumi

249367

Academic year 2024/2025

Acknowledgements

I would like to express my gratitude to my supervisor, Paolo Giorgini, for his guidance throughout every stage of development.

I am grateful to the surgeons at Santa Chiara Hospital in Trento, particularly Dr. Sara Allievi and Dr. Stefano Bonvini, and professor Zenati, for their willingness to listen to my requests, and offer their valuable feedback.

I'd like to thank my parents, family, and friends, for their unconditional support and encouragement throughout my University years.

This thesis is also dedicated to my grandfather, Pino. I'm sure he would have been very happy for me, and I wish he could have been here to see me finish this journey.

Contents

Abstract	1
1 Introduction	2
1.1 Thesis Outline	3
2 Background and State of the Art	4
2.1 From LLMs to Agentic Systems	4
2.2 Core Reasoning and Planning Techniques	8
2.3 Frameworks for Building LLM Agents	11
2.4 Multi-Agent Architectures	15
2.5 Applications and Benchmarking of Agentic Systems	15
3 ReflectOR, a Reflective Agent for the Operating Room	17
3.1 Post-Operative Debriefing	17
3.2 System Architecture	19
3.3 Complete Data Flow	25
4 Implementation Details	27
4.1 Technology Stack	27
4.2 Configuration	28
4.3 Input Pipeline Implementation	29
4.4 Transcription	31
4.5 Agent System Implementation	32
4.6 Web Interface	41
4.7 Memory Implementation	45
4.8 Prompt Engineering	47
4.9 Testing and Evaluation	49
4.10 Repository and Codebase	50
4.11 Development Challenges and Solutions	51
4.12 Deployment	53
5 Evaluation and Results	54
5.1 Evaluation Methodology	54
5.2 Input Pipeline Analysis	54
5.3 Qualitative Evaluation for the Coordinator Agent	56
5.4 Case Study	58
5.4.1 System Input	58
5.4.2 System Output Analysis	60
5.4.3 PDF Report	62
5.5 Performance and Trade-offs	62
5.6 Surgeons' Evaluation	62
5.7 In-depth Analysis of Testing and Troubleshooting	64
5.8 Chapter Summary	68

6	Conclusions & Future Work	69
6.1	Concluding Remarks	71
	Bibliography	71

Abstract

Effective communication and teamwork are critical for patient safety during surgical operations. During post-operation debriefings, surgeons should discuss fundamental issues that occurred in the operating room, to prevent them from happening again. This is often not the case, however, due to the rushed and informal nature of those meetings.

While Large Language Models (LLMs) excel at processing and generating text, they lack the necessary agency to handle structured, dynamic, and data-rich conversations like a post-operative debriefing. A simple RAG pipeline is not able to guide a structured discussion, learn from past interactions, and analyze operational data to identify important moments.

This thesis addresses this issue by introducing ReflectOR, an agentic system designed to act as an autonomous facilitator for post-operative debriefings. The primary contribution is the design of a multi-agent architecture where a Coordinator agent orchestrates specialized modules to overcome the challenges of traditional debriefings: its planning module guides the debriefing in a structured and data-rich way, to counter the informal nature of these meetings; its memory module can remember past interactions, previous issues, and team patterns, reducing the reliance on human memory; its tool-use module analyzes a surgical operation, evaluates teamwork, and identifies errors.

The system's capabilities are demonstrated and evaluated with simulated EVAR operations executed in the University's Laboratory of Augmented Health Environments. The evaluation validates the system's ability to identify critical events, facilitating well-structured and data-rich debriefings, using quantitative metrics and qualitative feedback from performing surgeons.

1 Introduction

In this chapter we'll go over the technological evolution that opened the path towards LLM agents, the main problem that ReflectOR wants to solve, and the structure for the entire thesis.

In the last few years, the field of AI, and in particular that of LLM Agents, has been shifting rapidly, moving beyond simple models that can process and retrieve information, towards models that can reason, plan, and act.

The **evolution** began with simple rule-based programs of the 1980s, it progressed to statistical pattern recognition with machine learning of the 2000s, and was recently revolutionized with the invention of Large Language Models (LLM). The advent of models like Google Gemini and OpenAI GPT marks a new era where computers are able to understand us and respond in a way that wasn't possible before. These models have demonstrated the ability to understand and generate human text, creating a new generation of conversational AIs, code-completion, and AI assistants. But simply reading and generating text is not enough, that's why we are entering the "Agentic Era", characterized by the introduction of LLM Agents. These agents are able to reason about a problem, perceive and even execute actions in the environment to achieve the goals, while keeping a memory of previous interactions and observations. Agents are often composed of a few key components, like memory for long-term learning, planning capabilities to reason about a complex problem, and the ability to use external tools to interact with the world. This leap from passive conversational partner to active individual opens the door to solve complex, real-world problems that require continuous support, like healthcare, which have thus far been beyond the scope of traditional AI.

Regardless of these innovations, high-risk **problem domains** like healthcare require skills that standard LLM Agents fail to approach. An example of this is the domain of post-operative clinical debriefings.

In the critical field of surgery, **ineffective teamwork** and communications are huge issues that can lead to medical errors. **Post-operation debriefings** are a key tool for improvement, yet they are often rushed or do not cover important problems, due to the physical and psychological stress that doctors go through after a surgery. Studies [2] have shown that communication breakdowns in surgery contribute significantly to adverse outcomes, making debriefings not just helpful, but critical for patient safety. Based on an article by the Hipaa Journal [1], research showed that the most common communication failures between clinicians is due to poor documentation of patient information.

While LLMs and Retrieval-Augmented Generation (RAG) systems are often enough for tasks like Q&A and summarization, they are usually not enough for complex or **dynamic problems**, since they cannot adapt. One such case is post-operation debriefing. Effective debriefing is not a simple Q&A session; it's an iterative process that requires knowledge, data, and ability to act with the team to understand issues and analyze the situation. A simple RAG system can retrieve a best-practice protocol, but it **cannot** guide the conversation based on what exactly happened in the operation room, it cannot identify errors so that they are not committed again, and it lacks the ability to use the **memory** of past interactions to understand patterns and try to find solutions.

There is a need for an intelligent system that doesn't act as a passive tool, but a reflective partner, an agent that can **remember** a team's history, **identify** critical moments from a procedure, and **facilitate** a structured, data-rich, and psychologically safe debriefing. This requires an agentic system that can reason, plan, act and learn in a continuous loop, thanks to conversational data, experiences, and domain-specific expertise.

To fill this gap, this thesis explores the design, implementation, and evaluation of a LLM-based agentic system that can structure post-operative debriefings.

In particular, it determines the **components** and **techniques** required to build a LLM agent able to guide the surgical team in a well-structured, data-rich, post-operative debriefing. Furthermore, it explores different **architectures** for agentic systems, to understand what architecture can solve such a complex problem. Additionally, it explains how the system can be **evaluated** on the problem domain. Finally, the **ethical implications** of deploying such a system in a delicate settings are discussed, exploring the potential influence to human's behavior.

The main contributions for this thesis are, firstly, the summary of the current **state of the art** for agentic architectures. Then, the design and implementation of **ReflectOR**, a multi-agent system for post-operative

surgical debriefing. The architecture is based on a hierarchical model, where a ReAct Coordinator controls **specialized agents** to analyze surgical operations, and communicates with surgeons during debriefings, designed for the variable and dynamic nature of post-operative debriefings. Finally, the evaluation of the system through a **case study** in simulated EVAR surgeries in the University’s Laboratory of Augmented Health Environments, combining quantitative input metrics (e.g. WER, DER) with qualitative feedback from surgeons.

1.1 Thesis Outline

This thesis is organized as follows:

- **Chapter 2: Background and State of the Art:** Explores the evolution of AI systems that culminated in LLM-based agents, and explains the basic concepts that are required to understand LLM Agents, such as planning, memory, tool use. Then, reasoning techniques such as Chain-of-Thought are introduced. Finally, popular frameworks used to build LLM Agents, such as LangChain and LangGraph, are discussed.
- **Chapter 3: ReflectOR, a Reflective Agent for the Operating Room:** Presents a thorough description of the ReflectOR system and all its components, including the problem domain, the goal and the requirements for the system.
- **Chapter 4: Implementation Details:** Explains the actual implementation details of the system, with a more in-depth explanation on the frameworks and techniques used.
- **Chapter 5: Evaluation:** The system output is evaluated through a case study, measuring quantitative and qualitative metrics.
- **Chapter 6: Conclusions & Future Work:** Summarizes the contributions of this thesis and the results obtained. Then, possible future directions for further research are discussed.
- **Glossary:** Contains definitions for various terms used inside this thesis.

2 Background and State of the Art

This chapter will provide the theoretical knowledge required to understand the ReflectOR design, starting from the foundational concepts of Large Language Models (LLM) and their evolution into autonomous agents. Then, the main reasoning techniques will be discussed, followed the frameworks used to implement such agents, and the main multi-agent architectures. Finally, the most important practical application of such systems will be presented, together with their evaluation methodology.

2.1 From LLMs to Agentic Systems

A foundational model, also referred as Large Language Model, is at the core of any agentic system. Examples of these include OpenAI's GPT-4o, Meta's LLAMA 3, and Google's Gemini. These models are deep neural network, typically based on the Transformer architecture [14], which allows models to understand grammar, context, and other natural language patterns. This result is possible thanks to their training using an incredible amount of text and code. This is what enables ReflectOR to understand and analyze surgical operations, allowing the system to guide a structured debriefing by answering the team's questions.

LLMs are a type of deep neural network built using the previously mentioned transformer architecture (Figure 2.1), made up of layers in a design that is derived from how the human's brain functions. Each layer contains processing units (parameters) which pass data to each other starting from the input layer, until they reach the output layer and form the actual output. In recent years the number of parameters has skyrocketed, since it's what allows a LLM to be more capable, as seen in table 2.1.

Model	Release Date	Parameter count	Notes
BERT	10-2018	340M	Google
GPT-2	11-2019	1.5B	OpenAI
GPT-3	06-2020	175B	OpenAI
GPT-4	03-2023	1T+	OpenAI
Llama 3.1	07-2024	405B	Open Weights, Meta
DeepSeek R1	01-2025	671B	Open Weights, DeepSeek
Kimi K2	07-2025	1T	Open Weights, MoonshotAI

Table 2.1: A look into the exponential increase in the number of parameters

Contrary to what one may think, LLMs do not actually understand human words, since they work on what is known as a token, a chunk of text that the LLM can process. A token is then further processed in the embedding layer, where it's given a mathematical representation that captures the token's semantic and syntactic meaning.

The Self-Attention mechanism, introduced by a revolutionary paper by Vaswani et al. [14] in 2017, is considered the cause for the AI revolution that happened in the 2020s. This is what allows a model to process the relationship between a word and all other word in the text, enabling the model to consider the meaning of a word based on the context within the input.

After designing the architecture, a LLM needs to be trained appropriately before it can understand and generate text. First, a model is trained on a vast amount of text data, including text from books, web pages, and more. The quality of the text is what affects the resulting model, which is why it's important to use high quality material. This type of training is "unsupervised", meaning that all the training data is given without any instruction and without any feedback. After a model has been trained, it can be "fine-tuned" if needed. During the fine-tuning, the model is re-trained on a smaller dataset containing specific data to improve the model's performance on a particular task.

After being fine-tuned, prompt engineering can be used to instruct the model on how to act, and what to do. This is done by placing instructions in the "System Prompt". With few-shots prompting, the instructions include a few examples on how the input that it will receive, and what output it should give. Zero-shot prompting is

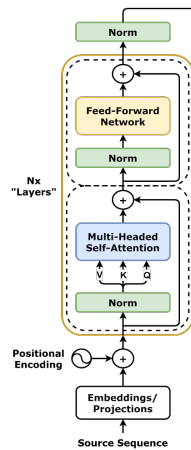


Figure 2.1: Architecture of the Transformer encoder block (Source: Deep Learning Visuals [5]). Each input token is transformed into a vector representation, called embedding. The embeddings are passed to the self-attention mechanism, to capture the relationships between tokens. The normalization is applied to stabilize and accelerate the training. The feed-forward network processes each token, and allows the model to capture complex relationships and process information gathered by the self-attention mechanism.

used to tell the model directly what to do and how, without giving any examples.

Examples

Few-shot-prompting:

You are an assistant used to classify security vulnerabilities on a scale from 1 to 10. The higher is the likelihood of the issue being exploited, and the impact of said exploit, the higher the score should be.

Example 1: Vulnerability: An unauthenticated user is able to directly access the admin panel.
Score: 10

Example 2: Vulnerability: An input field accessible only to authenticated users may be affected by Reflected XSS, potentially exposing some non-sensitive information. Score: 6

Example 3: Vulnerability: The webapp leaks detailed error messages including the stack trace.
Score: 4

Zero-shot prompting:

You are an assistant used to classify security vulnerabilities on a scale from 1 to 10. The higher is the likelihood of the issue being exploited, and the impact of said exploit, the higher the score should be. Give lower scores if the attack is very difficult to execute, or if it doesn't impact sensitive data. Give higher scores if the attack is easy to execute, and it could leak sensitive information.

LLMs are limited by their context window, the amount of tokens a model can process at one time, which includes both user input and the output generated previous by the model itself. Once the context window is full, the LLM starts losing some information (either the data in the middle of the conversation, or the oldest data).

A huge issue related to LLMs is hallucination, which happens when the LLM generates an output that is not based on the training data, doesn't conform to the instructions, is nonsensical or inaccurate.

These two problems are important to consider in real world applications. For example, the context-window defines how much information you can pass to the LLM. If the agent needs to assist in writing, it may not be able to hold an entire book in the context window. If the agent needs to be in healthcare, it needs to fit all the patient's information. Hallucinations are even more critical in sensitive scenarios such as healthcare, as generating inaccurate results could lead to severe consequences. To avoid running out of space in the context window, the previous interactions with the model can be summarized periodically before being fed back into it, reducing the amount of space required. Avoiding hallucinations is trickier however, as LLMs are still inherently

probabilistic models. Still, it's possible to reduce the probability of that happening by grounding the LLM with external knowledge (see the section on RAG), or by using techniques such as ReAct or Reflexion.

The LLM acts as the brain of the agent. Everything an agent can do is made possible by the LLM's ability to comprehend, reason, and generate text. Just the LLM is not enough to develop an autonomous agent however, as there are more components needed to carry out intricate tasks autonomously.

The **evolution** from a basic LLM to a fully fledged autonomous system is marked by several stages, each building upon the last to overcome previous limitations.

In their most basic form, **LLMs** act as text predictors. Given a prompt, they generate a statistically likely continuation. This allows the creation of simple applications like chatbots, and text completion. However, these models are stateless, possess information only up to their last training data, and they are prone to "hallucinations" (generating nonsensical or inaccurate outputs). They are passive tools that respond only when prompted, and cannot interact with the world beyond a basic text-based interface.

Retrieval-Augmented Generation [7] was the first evolution from basic LLMs. RAG grounds the LLM with external data sources. It involves two main steps (Figure 2.2): first, a "retriever" searches an external source (such as vector database or document) for information related to the user's query. Then, in order to produce a more accurate response, the user's query and the information that was retrieved are sent to the LLM. Although this method works well for basic Q&A, it is still essentially reactive. It improves what the LLM knows, not what it *can* do.

For surgical debriefing domain, a RAG could provide knowledge about best-practice checklist, but it wouldn't be able to analyze the transcript of the operation to determine whether the checklist was actually followed. It provides only static knowledge, lacking the ability to take actions.

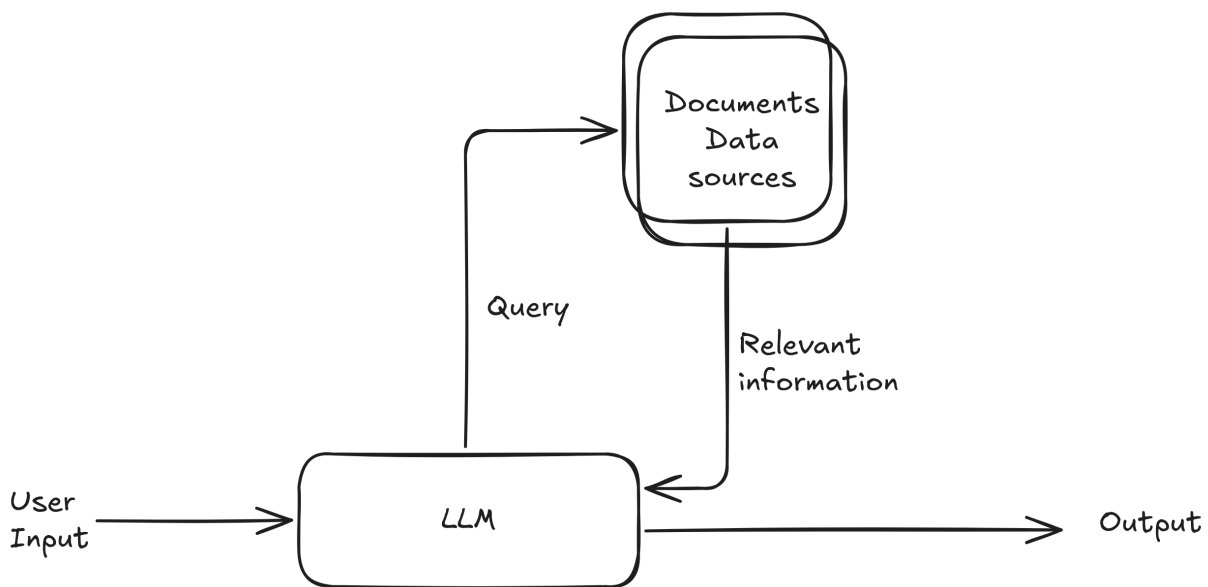


Figure 2.2: Basic RAG pipeline: The LLM receives the **user's input**. Then, it retrieves relevant information from the **external data sources**. Finally, using the inherent knowledge from the training, plus the information retrieved, it compiles the **final output** that is sent to the user.

One of the most important turning points for the agentic leap was the discovery that with specific prompting, it was possible to improve the LLM's reasoning capabilities. For instance, **Chain-of-Thought** [16] illustrates that by prompting the model to think "step-by-step", it could break down difficult problems into a series of intermediate steps that were far easier to solve, producing more accurate outputs. This ability to produce sequences of thoughts is the source of contemporary LLM agents' planning capabilities, allowing ReflectOR to analyze a complex surgical event from start to finish. This technique is discussed in more details in section 2.2.

The shift from passive conversational partner to an active, goal-oriented actor is known as the "**agentic leap**". Agents can use tools to monitor and interact with the environment, remember past exchanges using memory access, and devise a plan to accomplish a goal. Since agents work in a loop of reasoning, acting, and observation until the goal is achieved, they can tackle complex tasks that are above the capacity of basic LLMs.

An agent is more than a simple LLM. It's an **entire system** built around a LLM, It's composed of different modules that allow the autonomous nature of an agent. The most important are the planning, memory, and tool use modules (Figure 2.3).

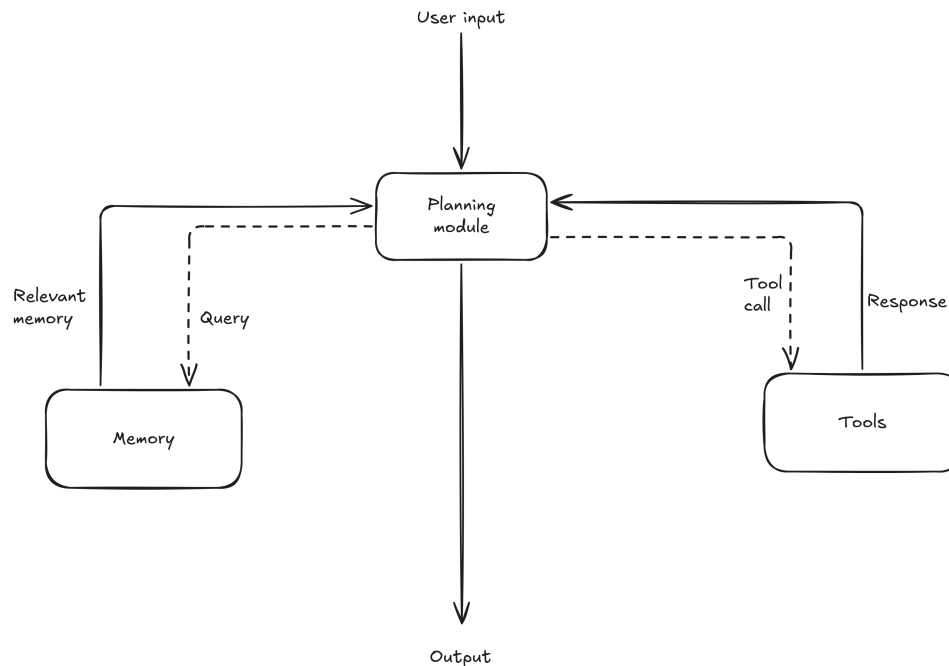


Figure 2.3: Anatomy of an Agent: The user input is sent to the **Planning** module, which uses the **Memory** module to retrieve relevant memories, and the **Tools use** module to interact with the environment and execute actions. Based on the gathered information, the final **output** is sent to the User

The **planning module** is responsible for the agent's ability to reason about a task and decompose it in smaller, more manageable sub-problems. Instead of trying to solve the entire problem in one go, the agent first creates a plan that dictates what it does next. This plan can be generated with various techniques, such as Chain-of-Thought (CoT) [16] prompting (see section 2.2), mentioned before, or more structured frameworks like ReAct (Reason, Act) [20] (see section 2.2), where the models think about what to do next, then use a tool to complete the action, and observes the result to decide on the next step. The Tree-of-Thought (ToT) [19] (see section 2.2) method simultaneously examines several reasoning paths, evaluating each one before committing to one.

For example, ReflectOR uses ReAct planning:

1. Reason: The user asked to analyze the transcript; first, I should extract useful information;
2. Act: Call the SummaryAgent tool;
3. Observe: The summary mentions a potential protocol deviation;
4. Reason: A potential error was found, I must use the ErrorAgent to get more details;

The **memory module** allows the agent to maintain context over time, enabling it to learn from experiences. This is separated by the LLM's knowledge about the world acquired during the training stage. Agent memory is typically divided into two categories:

- **Short-term:** the conversational context held within the LLM's context window, it's lost at the end of the session;
- **Long-term:** an external database to memorize information across different sessions. It's often implemented using vector databases, which enable an agent to perform semantic search to find relevant memories.

Long-term memory can be implemented in various ways, such as external databases. The most popular choice for this is a vector database, which differs from traditional databases because they allow for semantic searching through vector representation. All information (text, images) are converted into embeddings using a machine learning model. For example, if the database holds information about a patient and the query is: ‘Information about patient John Smith’, the database will return all information related to John Smith, without having to use a classic SQL query. Popular products include ChromaDB, used in this thesis, and Pinecone.

For ReflectOR, the short-term memory allows the system to understand the current session, keeping information about the analysis of the operation, and answering questions to the team about it during the debriefing. Long-term memory, instead, enables it to retrieve information about previous operations and debriefing, allowing a performance analysis over multiple sessions.

An agent uses **tools** to observe and interact with the environment. Tools can be anything from external programs, APIs, or data sources, and are used to perform operations or gather information. For example, an agent could perform web searches, execute code, or use specific APIs. Agents use "function-calling" to execute a tool while passing parameters.

ReflectOR, for example, uses the MaterialExtractor tool to receive a JSON object containing information about all the materials used during the operation, and their estimated cost.

This module is what allows an agent to stop being a passive responder, and start operating as an autonomous agent.

To be able to use a tool, the agent needs some information. First, it needs a name for identification. Then, a description that specifies when to use said tool and what it does. A tool can have a list of parameters that the agent can pass, such as the text query for a web search tool. Finally, there is the return type, which is what the tool will actually return (a number, JSON text, structured data, . . .).

There are different ways to call upon a tool. Initially, developers would just check the text returned by the LLM, to see if it was calling any tool. This approach was rather easy, but was prone to errors and was not easily scalable. Later, OpenAI introduced ‘function calling’, a structured way with JSON to call tools based on a list given to the agent. More recently the Model Context Protocol (MCP) was introduced, a standardized and extensible protocol with high portability and security.

As the agent is free to pass any parameter and receive any output, there needs to be some sort of check on the typing of said arguments. Tools such as Pydantic offer said support, allowing to define input and output types for any tool, and checking automatically when the tool is called. Other than typing, there are other security issues that can arise by allowing an agent access to tool. If an agent has unlimited access to the database, for example, it could lead to situations where the entire database is erased by accident by the agent. For such cases it’s advisable to have a ‘human-in-the-loop’ approach, asking for permission before doing sensitive changes.

Since a tool can fail due to wrong parameters or other problems, there needs to be strategies for when such things happen. An agent may be given the reason for the error so that it can try again, or have fallback strategies to other tools, or have graceful degradation if nothing works. If the agent can’t complete an action, it should have ways to communicate that to the users so that they are not left in the dark.

The agent has a list of tools it can use, so how does it choose what tool to call, and when? There are different approaches, such as rule-based tool selection policies, prompting engineering strategies, or dynamic tool selection. For instance, it may be based on keywords (‘search’ calls the search tool) or based on reasoning steps. Additionally, tools can be chained together (summarize operation → extract information → generate report), giving the output of one tool as input to another.

2.2 Core Reasoning and Planning Techniques

The evolution from simple LLM to an agent wasn’t possible due to some huge change in the architecture of the model itself, but due to discoveries in the prompting techniques and the use of orchestration strategies. They enable models to decompose problems, interact with the environment, and reflect on their actions. These techniques are not just used by themselves, but often combined to form powerful reasoning agents.

One of the most foundational breakthrough for enabling complex reasoning was **Chain-of-Thought Prompting** [16], which introduces a series of reasoning steps in the LLM’s answer. The technique works by using few-shots prompting using triples, which means giving a series of < input, chain-of-thought, output > examples in the prompt. A chain of thought is a sequence of intermediate steps that lead to the final output (Figure 2.4).

An example of System Prompt given to a LLM to generate the Chain-of-Thought using few-shot prompting

is (source: Chain-of-Thought Prompting Elicits Reasoning in Large Language Models [16]):

Q: There are 15 trees in the grove. Grove workers will plant trees in the grove today. After they are done, there will be 21 trees. How many trees did the grove workers plant today?

A: There are 15 trees originally. Then there were 21 trees after some more were planted. So there must have been $21 - 15 = 6$. The answer is 6.

Q: If there are 3 cars in the parking lot and 2 more cars arrive, how many cars are in the parking lot?

A: There are originally 3 cars. 2 more cars arrive. $3 + 2 = 5$. The answer is 5.

Q: Leah had 32 chocolates and her sister had 42. If they ate 35, how many pieces do they have left in total?

A: Originally, Leah had 32 chocolates. Her sister had 42. So in total they had $32 + 42 = 74$. After eating 35, they had $74 - 35 = 39$. The answer is 39.

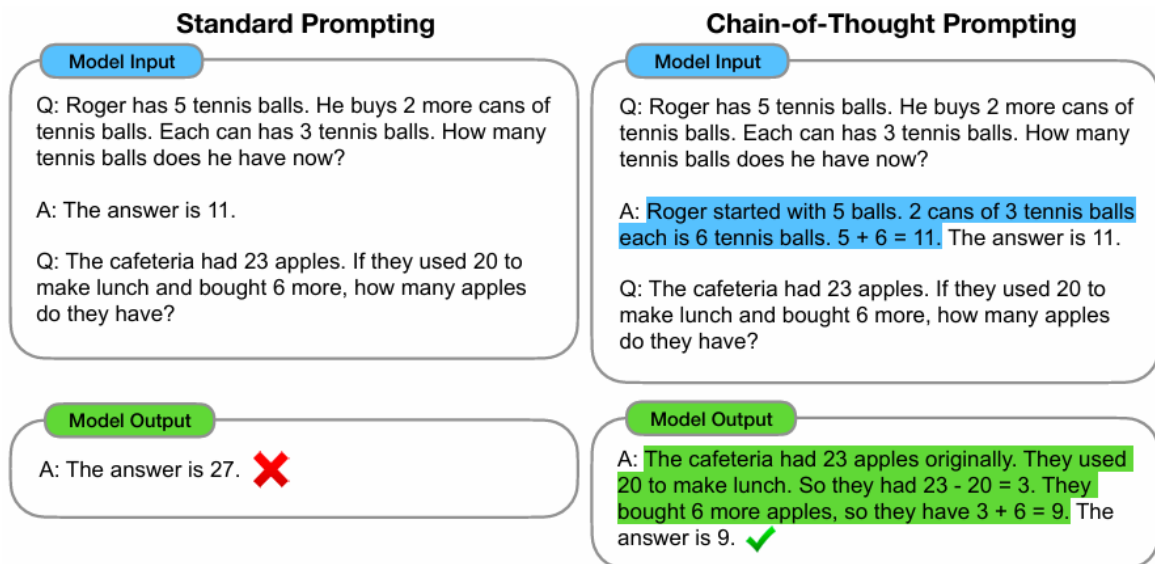


Figure 2.4: Chain of Thought Example (from Wei et al. [16]). Using **standard** prompting, the model gives the wrong result. Using **Chain-of-Thought** prompting, the model first reasons about the problem, producing the correct output

There are two main advantages to CoT:

1. Decomposing a complex problem into a series of simpler problems that can be solved individually.
2. Creating an interpretable window into the behavior and the thought process of the model.

It was later demonstrated by Kojima et al. (2022) [6] that this is a natural property in larger models, requiring no examples at all. By just using the phrase "Let's think step by step" in the prompt (a technique known as Zero-shot prompting), the model would spontaneously generate a coherent chain of thought.

However, CoT is still fundamentally a static planning technique, it has no external feedback or self-correction. It cannot interact with the real world to adjust the plan based on new information.

This technique is now fundamentally integrated in LLMs from the training stage, allowing models used in ReflectOR to reason about the problem as they answer the question, enabling the reasoning necessary to decompose the analysis of surgical transcript into the final report.

While CoT improves reasoning in LLM models, it only follows a single reasoning path which could lead to an incorrect answer. **Tree of Thought**, developed by Yao et al. (2023) [19], corrects this issue by creating a

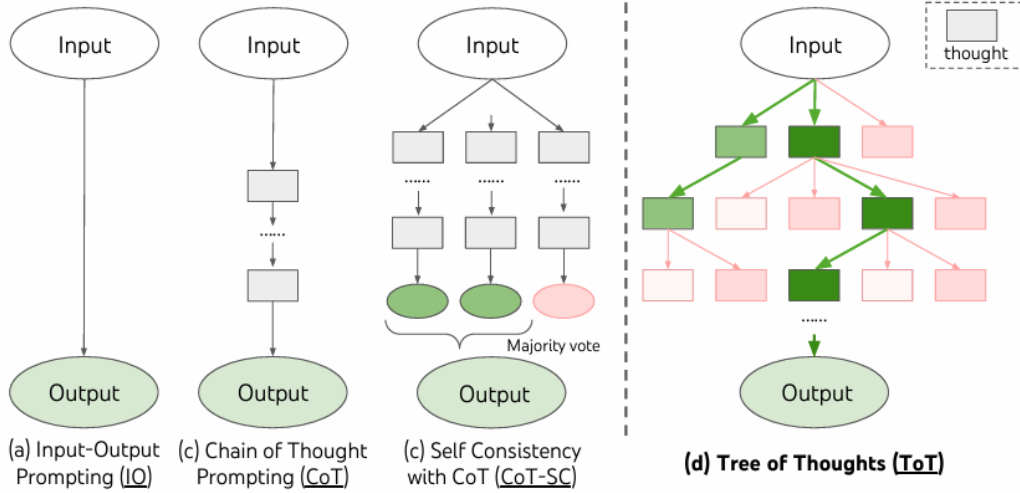


Figure 2.5: Tree Of Thought example (from Yao et al. [19]). Classical architectures like CoT producing a **single** reasoning path. Tree-of-Thoughts, instead, produces **different branches**, each taking different steps towards the final answer. Only one path is selected by the **Evaluator**.

tree. In the tree, each branch corresponds to a different reasoning path, allowing the LLM to evaluate a decision and possibly backtrack to a different one if needed, exploring multiple paths simultaneously (Figure 2.5).

With this approach, first the LLM generates different possible plans, with each of them being a different branch in the tree. Then, an "evaluator", another LLM, scores each path based on how likely it will lead to a successful outcome. Finally, a search algorithm like breadth-first-search (BFS) is used to navigate the tree. It explores each branch and backtracks when needed, looking for the best solution.

While ToT offers a robust method for exploring different reasoning paths, this would require a higher computational complexity and a slower execution, and it was deemed unsuitable for the more linear workflow of debriefing for the ReflectOR system.

While CoT and ToT only improve the reasoning capabilities of agents, **ReAct (Reason, Act)** [20] combines planning with acting, moving towards real agents.

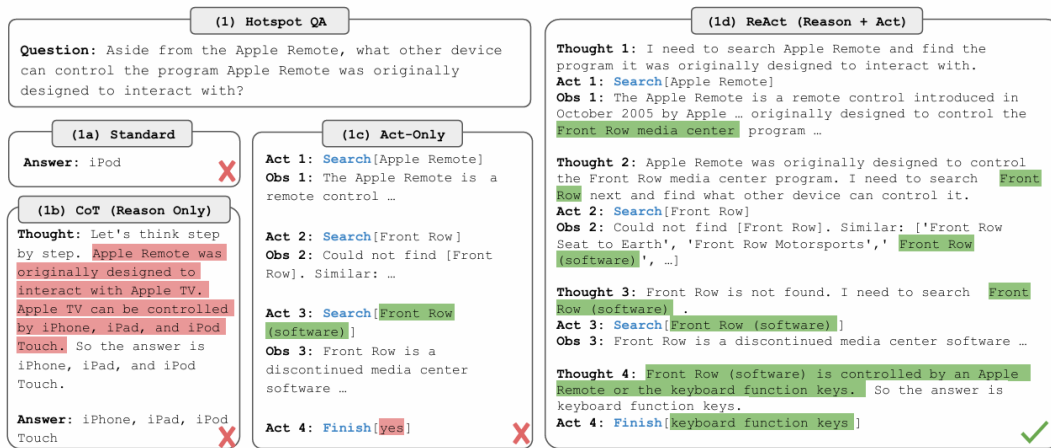


Figure 2.6: ReAct example (from Yao et al. [20]). With **CoT** prompting and with **Act-only**, the model is unable to find the right answer. With **ReAct**, instead, it is able to reason on the output received from the tools, finding the correct result.

This techniques first reasons about a problem, decomposing it into simpler sub-tasks. Then, the agent uses tools (act) to interact with the environment, gather information, etc. Finally, the agent observes the result it obtained, iterating on this process until the final result is reached.

Thanks to this approach, the agent moves away from being a passive responder, towards an active participant in the environment. This process can also help solve hallucinations, since the agent can ground its knowledge on real information about the environment, and adjust its plan as new information becomes available.

The ReAct framework is the reasoning engine for the ReflectOR Coordinator, allowing it to continuously loop between reasoning, acting and observing to serve the most precise information to the users. It enables the system to reason about the operation, call tools to analyze the transcript, and observe the result from the tool's output to reason about the next step.

While ReAct can correct its plan during the task, the **Reflexion framework** [13] introduces a mechanism for agents to learn from their failure, something that required slow and expensive fine-tuning before.

It employs a two-phases process:

- **Execution:** an agent tries to execute a task, often using a framework like ReAct, with the entire thought process recorded.
- **Reflection:** the agent is prompted to reflect on the failed execution, identifying issues and points of failures, verbally identifying what went wrong and what better strategies it could have used. This reflection is stored in its context for future use (Figure 2.7).

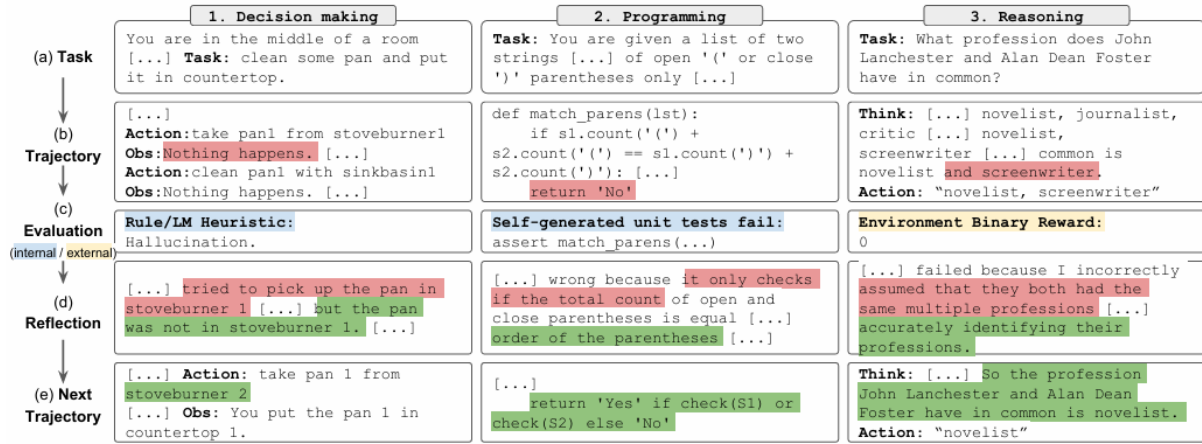


Figure 2.7: Reflexion (from Shinn et al. [13]). First the model executes some actions, using for example ReAct. Then, it **evaluates** the result, and **reflects** upon the evaluation to identify issues, preparing for the next execution.

The verbal feedback helps the agent to avoid doing the same mistakes multiple times. The Reflexion framework demonstrates that an agent's performance can be improved by giving it the ability to learn from its own experience.

The current ReflectOR implementation does not make use of the Reflexion technique, but a future version could 'reflect' on the debriefing output to refine internal strategies for analysis and error detection based on user's preferences and feedback.

2.3 Frameworks for Building LLM Agents

The techniques described above provide "how" the agentic reasoning works, but developers need the actual tools to implement agents. In the last few years, many open-source frameworks have emerged to abstract the code involved in building LLMs, allowing developers to focus on building the logic of the agents. These frameworks provide interfaces for managing prompts, handling tools, connecting to LLMs, and managing the complex logic that connects the components.

LangChain [4] is possibly the most popular framework for developing LLM-based systems. In this framework, various components can be "chained" together to achieve the desired outcome. For example, text can be extracted from a document, and chained to a LLM to execute some actions on the resulting text (Figure 2.8).

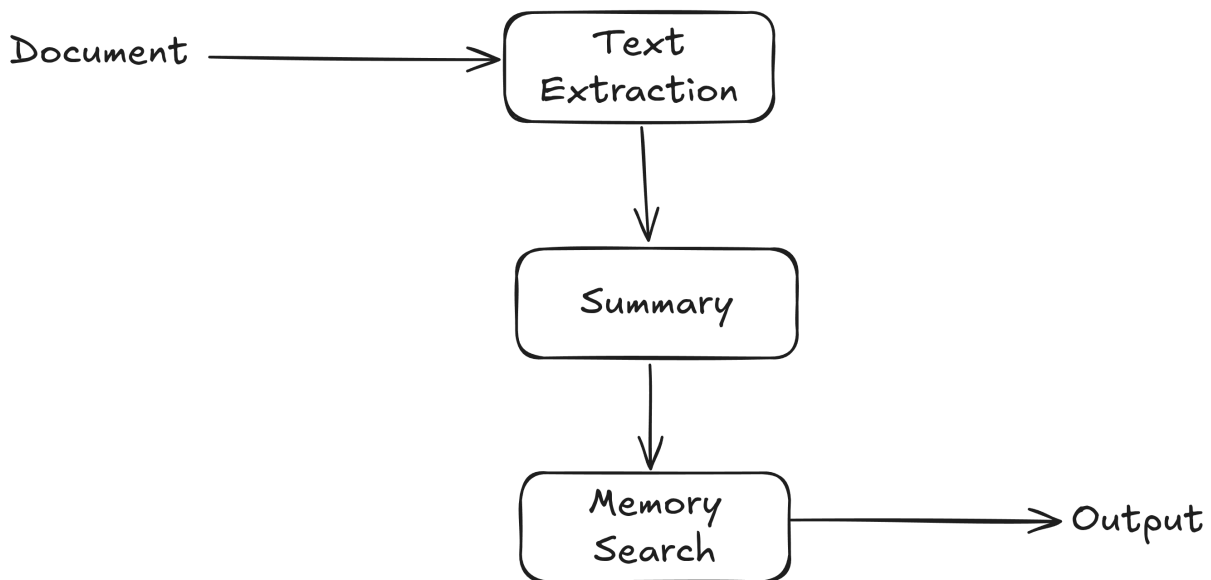


Figure 2.8: Example of LangChain’s architecture. The agent **extracts** text from the document, **summarizes** it, and **searches** the memory for relevant information, generating the final result.

The framework provides some easy to use tools to use modules from different vendors (OpenAI, Google, Anthropic, etc.); handle System Prompts with input variables; access external memory, tools and APIs; and chaining blocks together.

This snippet of code shows an example of chaining three components, a prompt template that accepts the ‘user’ variable, a call to a LLM (gpt-4o-mini), and a parse of the result to a string.

```

1 # Create template for a prompt, requiring the 'user' variable
2 prompt_template = PromptTemplate(
3     input_variables=["user"],
4     template="Greet {user}"
5 )
6 # Initialize an OpenAI model
7 llm = OpenAI(
8     temperature=0.7,
9     model="gpt-4o-mini"
10 )
11 # Parse the result of the LLM into a string
12 output_parser = StrOutputParser()
13 # Create the chain
14 chain = prompt_template | llm | output_parser
15 # Execute the chain
16 result = chain.invoke({
17     "user": "John",
18 })
19 print(result)
  
```

To create agentic systems, however, the linear "chaining" logic provided by LangChain may not be enough. That’s why the developers introduced LangGraph, a graph-based version of LangChain. In this new framework, each node represents a tool or a LLM call, and edges represents conditional logic to determine which next step to execute, as seen in figure 2.9.

This code snippet shows the creation of the state shared by the various components. The graph has three nodes: an input node, a node that executes an analysis, and a node that executes the summary. Finally, there is a conditional edge that can take from the input node to either the analysis or the summary node, based on the result of the ‘should_continue’ function.

```

1 # Create the state passed in the graph
  
```

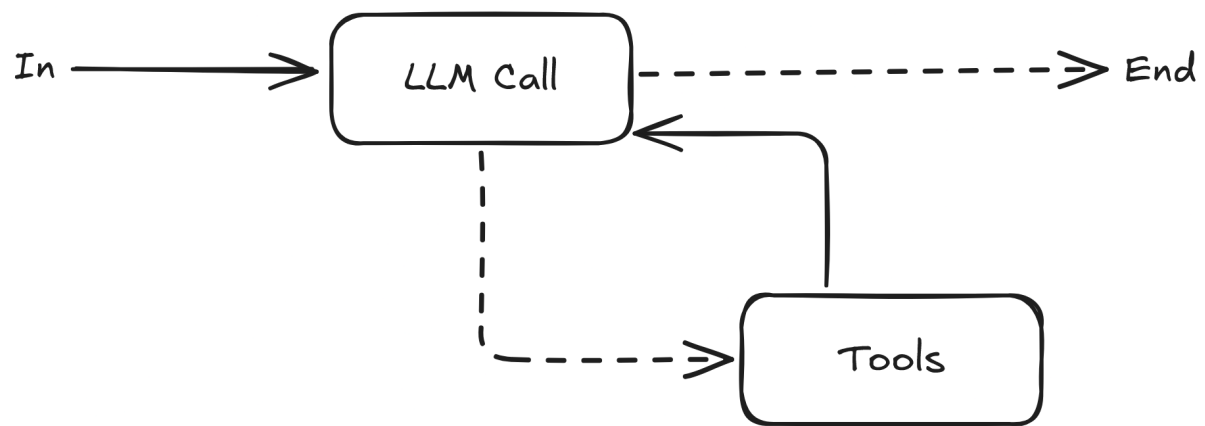


Figure 2.9: Example of LangGraph’s architecture (redrawn from LangGraph’s documentation). A graph is built to design the agent, where the **nodes** are LLM calls and Tools, while **edges** are connections that the agent follows based on conditions.

```

2 class GraphState(TypedDict):
3     messages: Annotated[list, add_messages]
4     user_input: str
5 # Initialize the graph
6 workflow = StateGraph(GraphState)
7 # Add the nodes to the graph
8 workflow.add_node("input", analysis_node)
9 workflow.add_node("analysis", analysis_node)
10 workflow.add_node("summary", summary_node)
11 # Add a conditional edge between the nodes
12 workflow.add_conditional_edges(
13     "input",          # Starting node
14     should_continue,  # boolean function
15     {
16         "analysis": "analysis",
17         "summary": "summary"
18     }
19 )

```

Another great advantage to LangChain and LangGraph is LangSmith. This is an additional tool natively integrated in LangChain, which can be enabled simply by adding a couple of environment variables inside the .env file. This tool allows developers to trace their entire system, including knowing in real time the cost of the calls to external LLMs that have been made, the inputs and outputs that were given, the tool calls, and much more. It allows a complete analysis of what the system did and what is doing, enabling an easy debugging and evaluation.

ReflectOR uses a combination of LangGraph and LangChain to create the agentic system, and LangSmith for testing and debugging purposes.

While LangChain and LangGraph allow the creation of single, powerful agents, Microsoft’s **AutoGen** framework is designed to simplify the creation of multi-agent systems. Its design focuses on "conversational-based programming", where instead of a fixed graph of execution, developers define a set of rules to govern the interaction between multiple agents.

In AutoGen, agents exchange messages to solve tasks. Usually, a typical system would include a User Proxy Node, representing the user, and a number of Assistant Agents, LLM agents with different roles that collaborate to solve a task.

The agents engage in a conversation until the User Proxy decides that the task is complete. This framework is particularly powerful for tasks that benefit from an approach where different agents have different expertise, criticizing each other’s work while working towards a solution.

The following is a snippet of code where two agents create a story. One is a creative writer who generates the story, and the other is the editor. They talk to each other and refine the story iteratively.

```

1 # Creative writer agent
2 writer = autogen.AssistantAgent(
3     name="CreativeWriter",
4     system_message="\"You are a creative writer.
5     You write engaging, imaginative content.\"\"",
6     llm_config=llm_config,
7 )
8 # Editor agent
9 editor = autogen.AssistantAgent(
10     name="Editor",
11     system_message="\"You are an editor.
12     You provide constructive feedback to improve the writing.\"\"",
13     llm_config=llm_config,
14 )
15 # Create user proxy, don't use user input
16 user_proxy = autogen.UserProxyAgent(
17     name="UserProxy",
18     human_input_mode="NEVER",
19     max_consecutive_auto_reply=3,
20     is_termination_msg=lambda x: x.get("content", "").rstrip().endswith("TERMINATE"),
21 )
22 # ... Create group chat, manager, omitted
23 # Start chat
24 user_proxy.initiate_chat(
25     manager,
26     message="\"Writer: Please write a short story (2-3 paragraphs)
27     about a robot learning to paint.
28     Then, Editor: review the story and suggest improvements.\"\"",
29 )

```

Several other frameworks exist, each with their advantages and disadvantages:

- **LlamaIndex** [8]: a data framework for LLM applications. It provides many tools to handle data, create indexes, query the data, etc. It's the first choice when connecting an agent to complex data repositories.
- **CrewAI**: focuses on building "crews" of agents to solve complex tasks. Compared to AutoGen, it focuses more on the orchestration of roles, compared to the conversational mechanics.

To summarize the different frameworks:

Name	Notes
LangChain	For simple systems
LangGraph	Advanced workflows, graph based
AutoGen	Conversation between agents
LlamaIndex	For data-based agents
CrewAI	Orchestration of agents with different roles

Table 2.2: Comparison between frameworks

After reviewing the most popular framework for building LLM agents, LangChain and its extension LangGraph were selected to implement ReflectOR. While frameworks such as AutoGen and CrewAI are excellent to simulate conversations between agents, they are not suited for a structured pipeline required to process a surgical transcript. LangGraph's graph-based architecture, instead, enables the creation of precise workflows, allowing accurate control over what component do, how, and when. LlamaIndex, while powerful for data analysis, doesn't possess the orchestration capabilities needed for ReflectOR's design.

2.4 Multi-Agent Architectures

While a single agent can solve many problems, some types of challenges benefit from a multi-agent approach, where multiple specialized agents cooperate.

There are various reasons as to why use a **multi-agent approach**. First, it promotes modularity and specialization. It's more appropriate and easier to maintain when there are multiple smaller agents, each with a specific expertise (e.g., a "research agent", a "coding agent", etc.), compared to one monolithic agent that must master all domains.

Furthermore, it helps with single agent limitations, such as context window or the use of multiple system prompts. Multi-agent systems also scale much better than monolithic ones because it's much easier and cheaper to train agents to do specific tasks well, compared to creating agents that are able to do everything, and risking a suboptimal result.

For a complex task such as surgical debriefing, which involves various steps such as error identification, timeline reconstruction, and performance evaluation, a single agent would require an impossibly complex prompt, reducing the accuracy, and raising the likelihood of imprecise answers. A multi-agent approach allows specialized agents with smaller and more focused tasks, ensuring more robust and precise answers.

Multi-agent systems can be organized in different architecture **patterns**, the most common ones are:

- **Supervised:** A supervisor agent decomposes the task and delegates sub-tasks to subordinate agents. The ReflectOR system employs this paradigm, with one coordinator and multiple specialized agents.
- **Hierarchical:** Similar to the supervised architecture, but with multiple levels. A manager has several "employees", who in turn have their own sub-agents, and so on.
- **Custom:** The relationships are defined based on the task.

There are various ways for agents to communicate with each other. They could do so by using message queues, so sending messages to each other to exchange information. Another way is to use shared memory, like an external database they all have access to.

ReflectOR uses function-calling, where the agents are treated as tools, and the coordinator can call upon a tool by passing some parameters (for example the transcript of the operation, information about the patient, etc.), and the agent that was called will return the appropriate output based on its expertise.

2.5 Applications and Benchmarking of Agentic Systems

The feasibility of creating an agent able to work in a complex environment like healthcare is supported by recent innovations in agentic systems in different domains. While not directly linked to the clinical field, ReflectOR's problem domain, these products demonstrate that the core technology for planning, tool use, and environment interaction can work for real-world application.

The most widespread usage of agents to date has been **software development**. Modern IDEs such as VSCode, IntelliJ IDEA, and emerging platforms like Cursor have integrated their option for agents capable of much more than just code completion. Current solutions can navigate complex repositories, create new files, execute tests, and refine their own outputs. Even "hallucinations", one of the biggest problems for LLMs, is partly fixed thanks to this iterative process. This evolution has introduced a new way of development, called "vibe programming", where users can create their own applications by using natural language with the agent, without even having to modify the code.

Examples include GitHub Copilot and Cursor's Composer.

Despite the advances, current generation in-IDE agents are still limited by their context window limitation, and have difficulties keeping up in complex codebases.

After the introduction of multi-modal LLMs, models that are able to understand not only text, but even images and speech, the development for agents able to **control the entire computer** started. These are agents able to "see" the graphical user interface, and complete actions on behalf of the human. They can develop code, execute it, browse the web, compile forms, and much more.

For example, OpenAI's Operator uses GPT-4o, a multi-modal LLM, to interact with the browser. It can "see" the screen using screenshots, and execute actions through a simulated mouse and keyboard. This agent uses as a base CoT and ReAct to perform tasks such as document editing and online shopping. To prevent

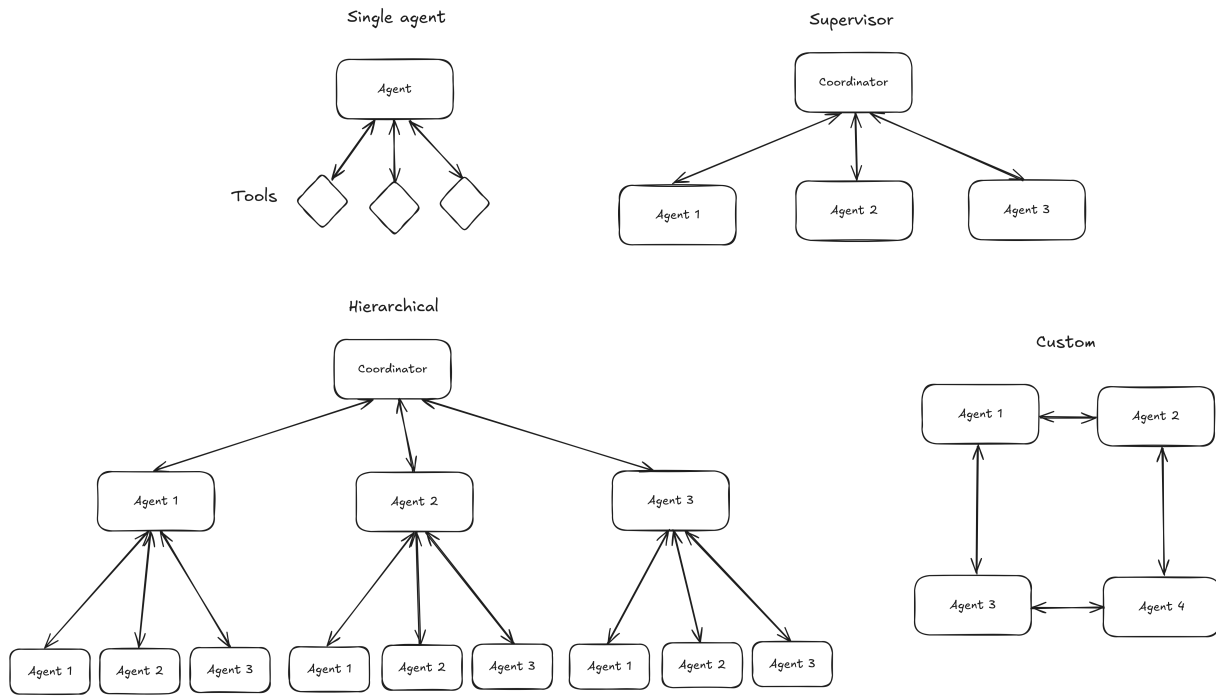


Figure 2.10: Multi-agent Architectures. The **single agent** uses only its own tools. A **supervisor** can call upon other sub-agents, coordinating them. A **hierarchical** structure is a multi-layer supervisor, where sub-agents are supervisors for other agents. The **custom** architecture is flexible, depends on the design of the system.

security risks, the model is "programmed" to wait for the user's input before executing possibly harmful actions, such as deleting information or making a payment.

ScreenAgent [12] introduced an environment for Visual Language Model agents to interact with computer screens. In that environment, agents can perceive the GUIs through screenshots, and execute mouse and keyboard operations. It also presents an interactive process of planning, acting and reflecting to observe the environment, execute actions, and reflect until the task is completed.

Embodied agents extend LLMs to interact and explore the environment. An example of this is Voyager [15], an agent designed to play Minecraft. Voyager can explore the environment, learn new skills through iterative prompting, and get real-time feedback from the game. It maintains an extensive "skill library" and employs an automatic "curriculum" to structure exploration.

The advancements in coding, OS control, and other environments prove the ability and power of modern agentic systems. However, a significant gap remains in applying these concepts to high-stakes, dynamic, and collaborative domains of clinical communication. ReflectOR is designed to address this gap, applying agentic techniques for post-operative surgical debriefings.

Evaluating agentic system can be very challenging because of a few different reasons. Firstly, agents have a probabilistic nature due to the use of LLMs, so it's hard to guarantee that similar inputs will lead to similar outputs. Additionally, such systems can be designed to do many tasks, so it's difficult to define precise criteria for the evaluation. Finally, it's complex to design metrics on abstract concepts like conversation quality, creativity, or planning.

Despite the challenges, numerous benchmarks have been created over the years to assess the capabilities of agents.

For example, **WebArena** [21] evaluates the interaction with the web browser, measuring how well an agent can navigate the web, retrieve information, or fill forms. **OSWorld** [17], focuses on the agent's performance on a simulated operating system environment, with tasks such as file management and document editing. For testing reasoning and decision-making, instead, **AgentBench** [9] can be used.

While the mentioned benchmarks are enough to assess general agent capabilities, they are not designed to evaluate a qualitative task like surgical debriefing. For this reason, ReflectOR combines quantitative transcript metrics with qualitative feedback from surgeons.

3 ReflectOR, a Reflective Agent for the Operating Room

This chapter outlines the problem domain related to the ReflectOR system, and outlines the design of the architecture for the entire system, to understand what components constitute the ReflectOR Agent.

3.1 Post-Operative Debriefing

Post-operative debriefing is a structured conversation between team members after a surgical operation. The debriefing session is critical to identify errors committed in the surgical operation, discuss what went well, and talk about eventual incidents.

However, due to the informal and rushed nature of these meetings these meetings often fail to achieve the desired effect. Additionally, their reliance on human memory, as noted by Loukissas et al. [10], can create errors in the documentation, possibly leading to medical error.

Arriaga et al. [2], instead, addresses the importance of timely and structured debriefing sessions to address psychological problems, and issues with teamwork, introducing the WATER checklist (Welfare check, Acute Corrections, Team Reflection, Education, Resource Awareness, Figure 3.1).

Traditional post-operative debriefing, based on team member's memory and small amounts of data, has several issues. Surgical teams often operate with very strict timelines, having to jump from one operation to the other without sufficient time for reflection and analysis. Additionally, debriefing sessions are often unstructured conversation without a standardized protocol, failing to capture the most pressing problems or learning opportunities. Finally, team members' recollection of events may be skewed due to the high pressure during surgeries, leading to inaccurate and unreliable debriefing outcomes.

This context provides a few direct challenges that need to be addressed by the ReflectOR system:

1. The need to process unstructured and often noisy audio;
2. Identification and structuring of events from dialogue;
3. The need for long-term memory to track performance over time;
4. The challenge of guiding a debriefing based on dynamic data;

The requirements and goals were gathered through meetings with surgeons from Hospital Santa Chiara in Trento. The surgeons described how their meetings were structured, and what could be helpful for them.

ReflectOR was designed to face these challenges. In particular, the goal of ReflectOR is to simplify and improve the post-operative debriefing process by introducing an autonomous LLM-based agent. The main goals are:

- **Process audio from the operation:** Transcribe and diarize spoken interaction in the operating room, making them available for structured analysis;
- **Analyze the transcription:** Generate a summary, identify key events, extract details (e.g. materials used, cost) from the processed transcription;
- **Identify errors:** Detect teamwork issues, protocol deviations, highlighting them for the debriefing session;
- **Facilitate structured discussion:** Guide the debriefing based on the WATER checklist, on the analysis, and on previous operations committed;

Potential elements for debriefing just after a perioperative event include (but are not limited to):

- 1. Welfare check:**
 - Assess if team members feel able to continue providing care.
- 2. Acute/short-term corrections:**
 - Matters to be addressed before next case?
 - Clinical/patient care needs?
- 3. Team reactions and reflection:**
 - Summarize case and listen to team member reactions.
 - Plus/Delta: Matters that went well and matters that could be improved.
- 4. Education:**
 - Lessons learned from the event and the debriefing.
- 5. Resource awareness and longer-term needs:**
 - Improve awareness of local peer-support and employee assistance resources.
 - Assess if any follow-up needed (e.g. safety/QI report).

Figure 3.1: WATER checklist (source: Arriaga et al. (2021) [2]). The WATER checklist is divided into five steps, used to analyze the current psychological state of the team, discuss feedback, and prepare for the next case.

- **Learn over time:** Store debriefing sessions, recommendation, and team progress to be used in future debriefings;
- **Generate reports:** Generate PDF reports to support documentation at the end of the debriefing;

To achieve these objectives, the system must satisfy the following functional requirements:

1. **Audio Processing:** Handling the audio from the operating room, including transcription, noise reduction, and diarization;
2. **Information Extraction:** Find information about the operation, the patient, and the team from the transcript;
3. **Event Detection:** Identification of critical incidents (errors, protocols deviation);
4. **Agent Architecture:** Integration of LLM-based planning, memory, and tool modules;
5. **Performance analysis:** Use long-term memory to analyze team performance based on past operations, measuring their improvement and highlighting recurring errors;
6. **Discussion planning:** Create a discussion plan for the surgical team to focus on, based on the analysis;
7. **Modularity:** Use a modular architecture for maintainability and extensibility;
8. **Reliability:** The system should be reliable in case of failures or errors;
9. **Reporting and Visualization:** Generate the PDF report after the debriefing, to be archived for documentation;
10. **Image support:** The Coordinator should support image inputs, to enable users to use images during the debriefing;

3.2 System Architecture

The system architecture is represented through the diagrams in Figure 3.2 and Figure 3.2.

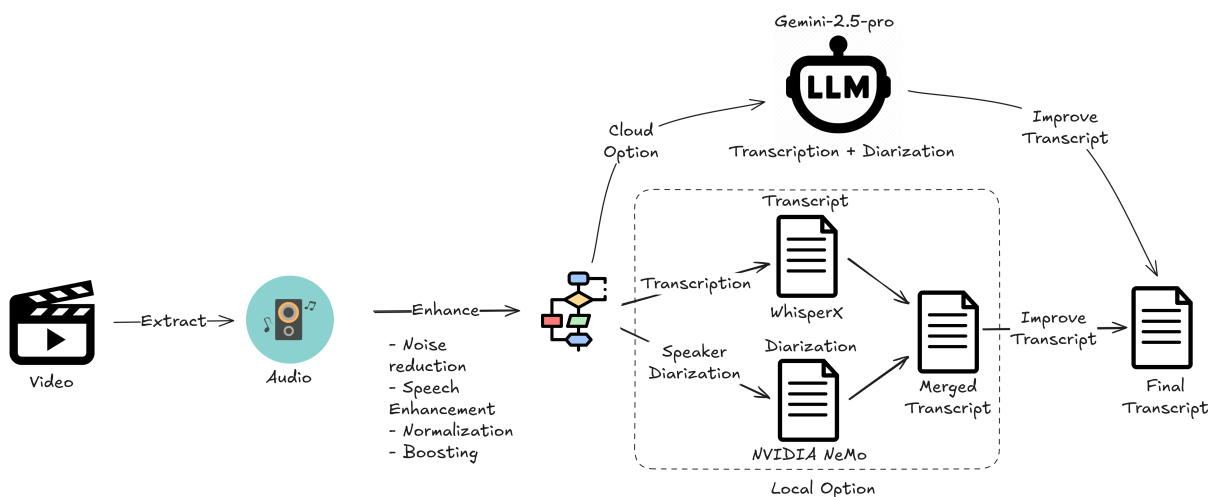


Figure 3.2: Input processing architecture: The audio gets **extracted** from the video of the operation, and it is **processed** through the previously described pipeline. The resulting audio is then **transcribed** and **diarized** through either the Local option (Nvidia NeMo and the WhisperX model), or with the cloud option (Gemini-2.5-pro). The resulting transcript is **improved** using an LLM, and it will be used as the **input** for the analysis by the Coordinator.

The choice for the components and their interactions was guided by the following key concepts:

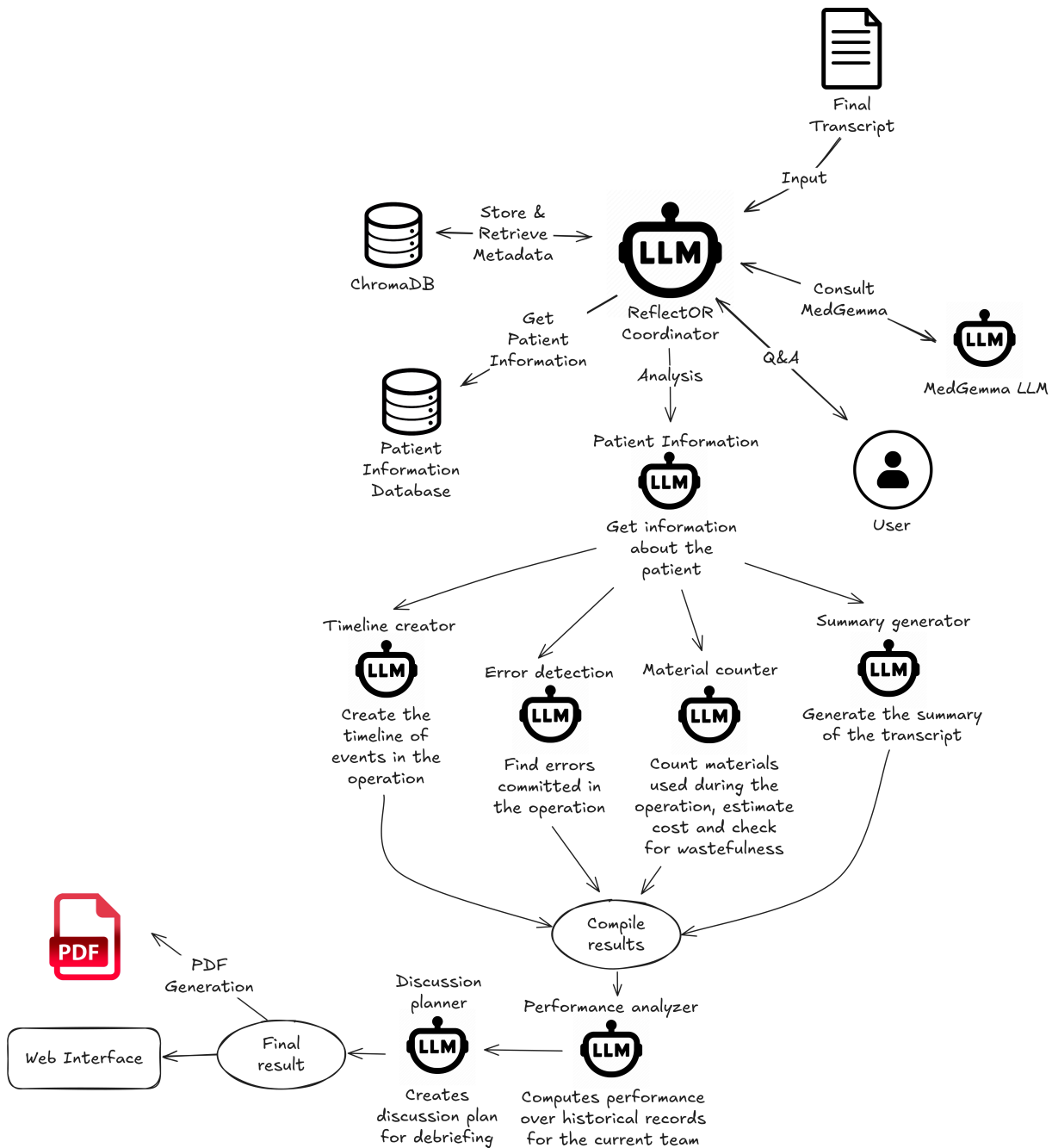


Figure 3.3: ReflectOR’s Coordinator architecture: The Coordinator is at the **center** of the entire system. Based on questions from the User, it can use its other **tools** to answer. It can read the transcript from an operation, and execute the **Transcript Processing Analysis**, which calls all other specialized sub-agents to analyze the transcript. It can read and write to the **long-term memory**, saving user’s preferences and retrieving relevant memories. It can then use other tools such as the Patient Information Database to get patient data, and communicate with MedGemma about technical medical information.

- **Design Rationale and Alternatives:** To satisfy functional requirement 1 (Audio Processing), a multi-step input pipeline was designed (see Figure 3.2). The pipeline is used to process the noisy audio coming from the operating room, to eliminate background noise, improve speech, execute diarization, and transcription.

It was decided for the system to be a multi-agent architecture. This was chosen because of the different tasks that needed to be completed. For example, summarizing the transcript would need a different expertise compared to finding errors in the surgery. It's also easier to implement parallelism of tasks with a multi-agent approach. Additionally, a monolithic agent would have been more difficult to extend and maintain. A RAG design would not have been a good fit, because as explained in previous sections, it wouldn't allow the agent to remember the team's past interactions, analyze the surgery, and have actual interaction with the environment. In particular, a multi-agent architecture is used to satisfy Functional Requirements 2 and 3 (Information Extraction, Event Detection). By using specialized agents for each task, each module can focus on a simpler problem, leading to a more reliable, maintainable and accurate system.

For the memory, the vector database was chosen due to its semantic search ability. Thanks to it, the agent can search for memories related to current events and people, without having the rigid structure required by regular SQL databases. This satisfies Functional Requirements 4 and 5 (Agents Architecture, Performance Analysis), enabling the system to remember past operations, and execute performance analysis with those operations in context.

To address Functional Requirement 6 (Discussion Management) and 9 (Reporting and Visualization), the generation of outputs is handled by separate modules, which creates a context-sensitive discussion plan, and generates the final PDF report with all previous information.

The choice of a supervisor architecture with specialized agents addresses Functional Requirement 7 (Modularity), and 8 (Reliability). The central Coordinator handles the state and memory, and it invokes stateless tools while passing only the necessary information for their task. This makes the system highly modular, where new tools can be added with minimal changes to the existing architecture. It's also reliable, since a failure of a tool is caught by the Coordinator, and can be handled gracefully by notifying the user of the error.

To satisfy Functional Requirement 10 (Image Support), the Coordinator should use a multi-modal model. ReflectOR supports various providers, and examples of supported models are GPT-4o and Gemini-2.5-flash.

- **Modularity and Extensibility:** Thanks to the system's modularity, it's possible to extend and maintain it in various ways.

Each tool is a different, separated component, so they can be swapped or updated without causing any issue. For example, adding a new tool such as a sentiment analysis tool would involve creating a new independent agent, register it inside the Coordinator, and updating the system prompt file for the Coordinator. This satisfies Functional Requirement 7 (Modularity), making the system easily maintainable.

The System Prompts are organized in a way where they can be easily changed, all inside folders divided by language. Because of that, it could be extended to other surgical operations aside from EVAR, updated to add new tools, and modified to add new instructions.

The internationalization is basic but effective. In the '.env' file, among other configuration options, it's possible to specify the folder containing the system prompts. By then creating the translated files, it's possible to apply the new language by setting the new path in the environment file. This strategy is very effective also because changing the prompt for any component is very easy, and doesn't affect other parts of the system, causing minimal disruption.

- **Error Handling:** The current ReflectOR system uses basic error handling, such as exception catching and logging. Comprehensive logging saves into the 'logs/' folder any call to external APIs (such as OpenAI), with the entire request and response, and each step the system takes, which components are called, and so on. Thanks to this, it's possible to analyze what the system does and why, providing easier debugging and performance analysis.

If a component fails, there is a retry mechanism which tries to execute the component again, hoping that it succeeds. If the component fails multiple times, then it's considered a real error. A custom error message is used as the output, to notify the Coordinator that the component failed, and the system proceeds with the rest of the execution, noting in the final report that a component has failed.

For increased robustness in future versions, a fallback option should be implemented. With such a mechanism, if a component were to fail, it should use the fallback option, such as using another provider for the LLM.

This system satisfies Functional Requirement 8, Reliability, ensuring that the system is reliable and robust even in case of failures.

- **Security and Privacy:** Due to the sensitive nature of the information involved in surgical operations, such as patient and operation data, security and privacy are essential for real deployment. In the current implementation, security measures are minimal since it involves only simulated surgeries.

For real deployment, the following measures should be considered:

- Encryption: Encrypt all sensitive data about patients and operations at all time, including encrypting databases at rest;
 - Anonymization: Anonymize all sensitive and identifiable data before processing, using for example unique IDs before they are sent to external services;
 - Access control: Implement strict authentication to access debriefing data and reports, and to access any sensitive information from the memory or Patient Info database;
 - Strict Logging: All accesses and operations should be logged for traceability, including all requests to access sensitive data;
- **Input Processing Pipeline** The input for the ReflectOR agent consists of audio recordings from simulated operating room sessions. To ensure an analysis over accurate data, the pipeline processes the initial audio with different tools, until the final transcript is acquired:
 1. **Audio Normalization:** The first step is to normalize the audio levels, making sure that even murmurs can be heard;
 2. **Noise Reduction:** The normalized audio is then processed to reduce ambient noise and improve clarity of speech, which is required for accurate transcription;
 3. **Speech Isolation and Enhancement:** The speech is isolated from background noise for more precise noise reduction, and it's enhanced further to make sure it's as clear as possible;
 4. **Speaker Diarization:** To attribute segments of speech to different individuals, the denoised audio is passed to a speaker diarization module.
 5. **Transcription:** The processed audio is transcribed, to obtain the text of everything that was said during the surgery;
 6. **Merger:** The diarization output is then combined with the transcription, assigning each phrase to the appropriate speaker;

The result of this pipeline is a structured JSON file containing a list of objects, with the timestamp of the phrase, the content, and which individual said that phrase.

- **ReflectOR Coordinator:** The core of the agentic system is the *ReflectOR Coordinator*, a LLM-based agent responsible for orchestrating the entire post-operative debriefing workflow. The coordinator receives as input the processed transcript of the operation, and plans and executes the actual system. It embodies the Supervisor design pattern described in Chapter 2, acting as the decision maker that decomposes the task of debriefing into simpler, more manageable sub-tasks, as can be seen in Figure 3.4.

Using both short and long-term memory, the Coordinator decomposes the debriefing task into a sequence of sub-tasks. For each sub-task, it invokes the appropriate node (e.g., summary generation, event marking,

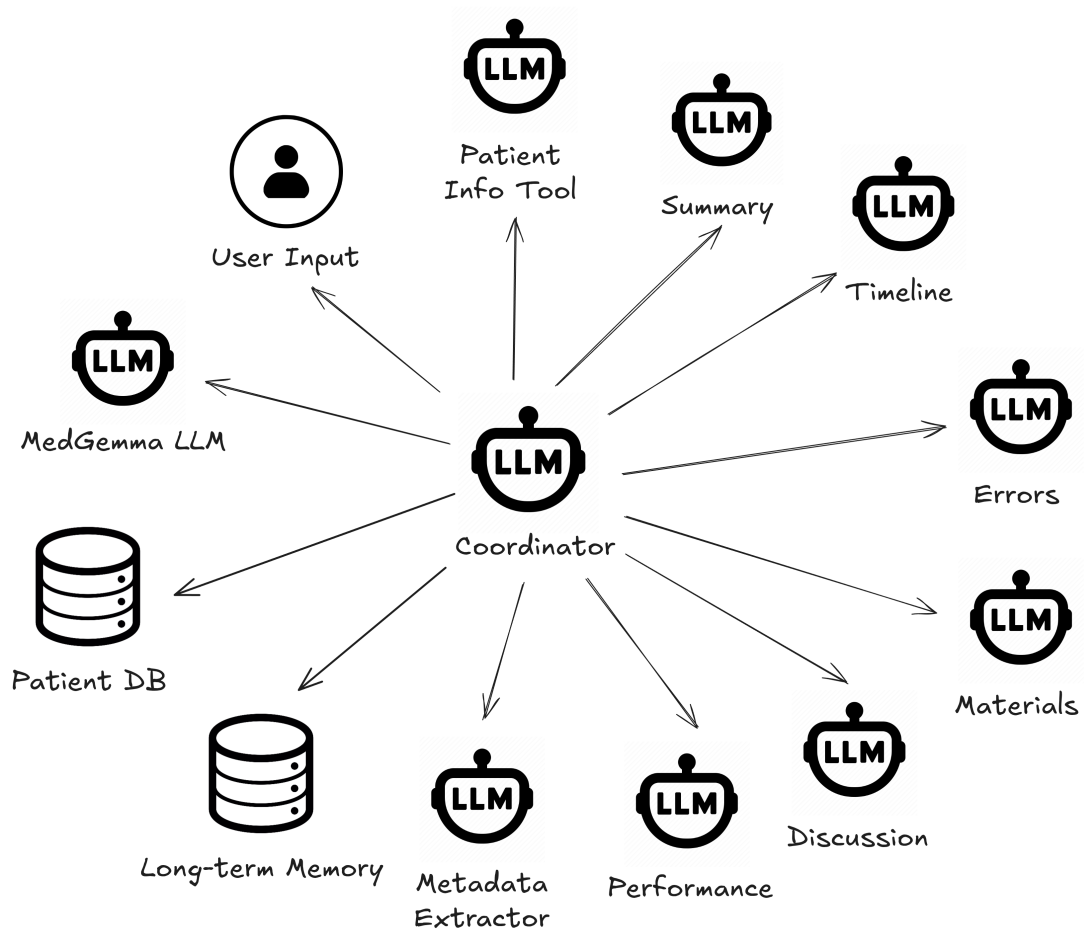


Figure 3.4: Coordinator's connections: The coordinator is connected to its specialized sub-agents, to the user input tool, to the MedGemma tool, and to the long-term memory.

material extraction) as a tool, merges the outputs, and returns the appropriate result for the debriefing. The Coordinator can personalize the result by referencing past team performance, flagged incidents, making the debriefing session unique to the surgical context.

- **Multi-Agent Subsystem:** While the Coordinator can plan and use the memory module, ReflectOR employs a set of LLM-based specialist modules that operate as tools in the system. Each module performs a specific task in the workflow:
 - **Metadata Extraction Tool:** extract metadata from the surgery, such as information about the patient, about the team performing the surgery, the outcome of the surgery, etc. The information obtained is passed to other tools.
 - **Material Extractor Tool:** Identifies the materials used during the operation, and gives an estimate of the total operational costs.
 - **Timeline Creator Tool:** Identifies the key events in the operation, constructing a detailed timeline of the surgery.
 - **Error Detection Tool:** Detects protocol deviations, teamwork issues, and other errors.
 - **Summary Generator Tool:** Produces a summary of the operation, with key actions, errors, and the outcome.
 - **Performance Analyzer Tool:** Computes the performance of the team over historical records, checking what needs to be improved, and what was better.
 - **Discussion Planner Tool:** Plans the discussion based on the information extracted from other tools.

Although these agents are stateless, they are developed to provide domain-specific perspectives on the debriefing process, while maintaining a simpler architecture than an actual multi-agent system, ensuring that the state management is centralized within the Coordinator for easier and more predictable debugging.

Additionally, two more tools are available for the Coordinator:

1. MedGemma: a way to call the MedGemma LLM, a model created by Google and trained specifically on medical data. It can be used for more technical information;
 2. User Input: lets the Coordinator asks something to the user, in case it needs more information while doing the analysis, or while answering a query;
- **Memory System:** ReflectOR leverages a vector database for long-term semantic storage and retrieval of data. This allows the Coordinator to:
 - Retrieve relevant past operations, team performance records, and best practices;
 - Identify recurring patterns;
 - Reference previous recommendation;
 - Remember preferences for the debriefing;

Short-term memory is maintained within the context window of the agent itself, while long-term knowledge is stored in the vector database, allowing for semantic search (search using natural language instead of SQL syntax).

Other than these two modules, for testing purposes an additional relational database was used: the Patient Information database. It contains information about patients, their past operations, and past exams.

- **Output Module:** The final stage of the ReflectOR workflow is the PDF Generator module, responsible for compiling the final output into a PDF report.

This module integrates:

- Team and patient information;

- Surgery summary, timeline, detected errors;
 - Materials and cost breakdown;
 - Discussion points and recommended follow-up actions;
 - Performance analysis for the team, discussing recurring errors and areas of improvement;
 - Expenses graphs based on past data, divided by operation type and date;
 - Raw data of the analysis and execution for testing and debugging;
- **User Interface:** There are two ways for users to access the system.

The first one is through Python's modules. By installing and instantiating the project, a developer could execute the workflow or extend it, adding new modules or modifying the prompt.

For users without development knowledge, it's possible to use the GUI accessible through the web interface.

The website is divided into four sections. In the first one, it's possible to chat with the Coordinator. The site supports both text input, sending images, and the usage of a microphone, and includes support for image inputs. The output of the Coordinator is in text form, with a button to generate the Text-to-Speech automatically. Then, there is a button to start the analysis of the generated transcript, and another one to generate the PDF.

In the second tab the user can generate the transcript by uploading an audio/video file, after processing it with the Input Pipeline described before.

The third tab is for testing and debugging purposes, and it allows to insert in the transcript the name of the team and the name of the patient, in case those information are not present in the original audio.

The last tab is for the settings, containing options to use Gemini diarization or local models, which TTS provider to use, the model for the Coordinator, and so on.

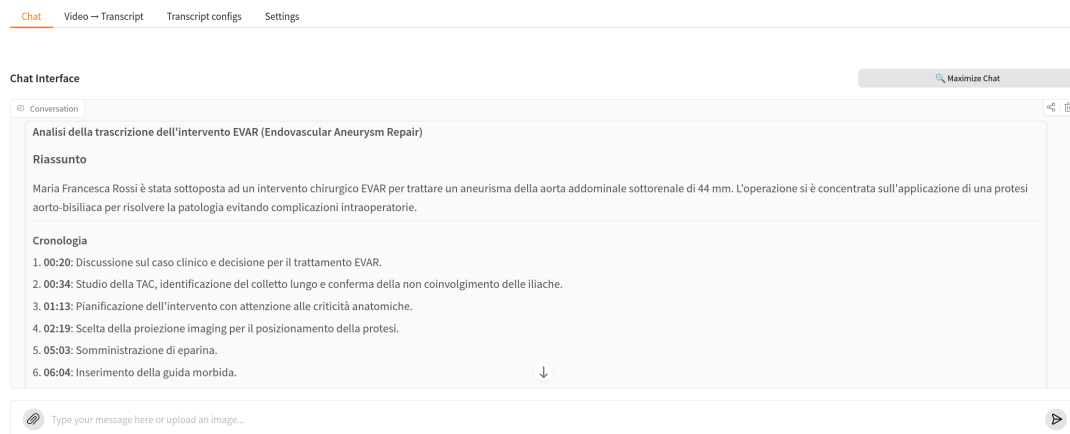


Figure 3.5: WebUI screenshot: Part of the first tab of the web interface, showing the tab layout of the application, with the chat interface to chat with the Coordinator.

3.3 Complete Data Flow

The ReflectOR system is designed to operate as an orchestrated pipeline, where the flow is managed by the Coordinator agent.

The overall flow is as follows:

- **Input:** The merged, diarized transcription is provided as input to the Coordinator;
- **Task Planning:** The Coordinator looks into the long-term memory for relevant experiences, and plans what tools to execute;

- **Tools invocation:** The Coordinator invokes each specialist module (summarization, timeline creation, etc.) and collects the outputs;
- **Integration:** The outputs are integrated into one result, assembling a coherent plan for the team;
- **Report Generation:** Based on the information obtained and the analysis, the PDF report for the debriefing is generated.

4 Implementation Details

This chapter is about the actual implementation details of the *ReflectOR* system. We'll go over the choices for the technology stack, including the programming language used, the framework, libraries, and other components. Then, we'll focus in details on how the entire system works, from the input pipeline all the way to each tool. We'll talk about which LLM model was chosen and why, explore how to check this thesis' repository, and go over the challenges encountered during the development.

Example: EVAR-001 Through this chapter a short example of a simulated **EVAR** operation will be used to illustrate each component. Only snippets will be provided in the chapter, but the full transcript and outputs are available in the GitHub repository.

4.1 Technology Stack

This section provides a brief overview over the technology stack used in this project.

- **Language:** The project was developed with **Python** (≥ 3.10) under the Apache 2.0 license. Python was chosen due to its leading role in the AI ecosystem, being the de facto standard for AI applications, amassing a vast amount of libraries and frameworks for AI and machine learning.
- **Package Manager:** **Poetry** is used for dependencies and building instructions, ensuring reproducibility.
- **Configuration:** The configuration file is handled by **python-dotenv**, which allows an easy configuration through the use of a '.env' file, holding the environment variables. The configuration is parsed, validated, and loaded into a structured object by **Pydantic**. Pydantic is also used to assure type and structural safety in the returns types for the agents.
- **Agent Framework:** As mentioned in a previous section (section 2.3), a number of frameworks have been released for building LLM agents in recent years. After careful consideration, the frameworks selected were **LangChain** and **LangGraph**. These two frameworks are designed to be used together. LangChain contains various utilities that make creating agents and tools easier, and LangGraph allows the design of complex logic and workflows, instead of linear workflows. Other options were considered, such as **OpenAI Agents** framework, or **Google Agent SDK**, but compared to the previously mentioned frameworks they were newer, with less support and documentation, and being less vendor-agnostic, providing less flexibility compared to the LangChain ecosystem. **CrewAI** and **AutoGen**, instead, were not suitable for the requirements requested by this project.
- **Model selection:** Due to local hardware constraints local LLMs were not an option, so external APIs had to be used. There were two main possibilities for this: Google's Gemini, and OpenAI's GPT. Comparisons and evaluation will be discussed in a later chapter (Chapter 5). The integration of these external APIs was easy thanks to LangChain's native integration with the two services.
- **Input processing:** A critical part of the system was the input processing pipeline, requiring robust manipulation of multi-speakers audio from surgical simulations. If the input is a video, the audio is extracted with the **audio-extract** library. The audio is first passed through **FFmpeg** for normalization, and then **DeepFilterNet** is used for noise reduction. Other neural network libraries such as SpeechBrain and Demucs have been used to further improve the resulting audio. Since the external LLM's APIs have limits on the size of data sent, and higher size would imply a higher latency, the audio is then compressed and split into chunks using the **pydub** library.
- **Diarization:** Nvidia's **NeMo** and **pyannote.audio** were used for speaker diarization using local models, while 'Gemini-2.5' can be used as cloud option, yielding better results.

- **PDF generation:** For the PDF report generation the **reportlab** library was chosen due to its ability to create PDF with ease. **Pandas**, a very popular plotting library, was used to create expenses graphs.
- **Long-term memory:** The vector database chosen to store memories was **ChromaDB**, due to LangChain’s integration with it and its popularity. This database uses a SQLite database as the actual storage, and enables the use of different embedding functions to handle the semantic search and storage. The ‘sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2’ LLM was used for sentence embeddings, supporting multi-language texts. The Patient Database, used to simulate the Sistema Informativo Ospedaliero (SIO), was implemented through **SQLite**, using the `sqlite3` Python native library.
- **Web Interface:** The web interface was created using **Gradio**, a popular library for AI applications that enables the development of web applications with ease through the use of pre-made components.
- **Testing:** Finally, **pytest** was chosen as the testing framework, due to its reliability and robustness.

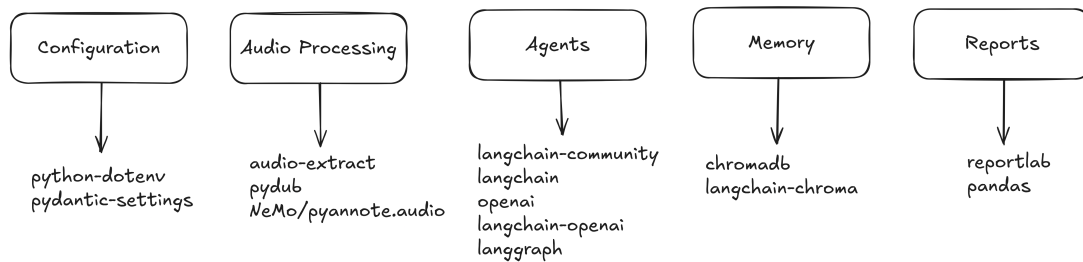


Figure 4.1: Dependency Graph: Main dependencies used for each component of the ReflectOR system

4.2 Configuration

As mentioned before, **python-dotenv** is used to load the environment variables from a `.env` file at startup. These environment variables include API keys for the external services (e.g. OpenAI or Google’s Gemini), model selection, log level, resource paths (such as the prompts’ folder), and more. This allows for easy modification of parameters without changing the codebase.

Pydantic is then employed to parse, validate, and structure the configurations into a Python object. This provides type safety and ensures all configurations fields are properly formatted.

This setup was used since it’s easy to implement and adapt during the development phase. For deployment in a real environment, some changes would have to be made. For example, the API keys and other sensitive information should be encrypted using systems such as **SOPS** or **git-crypt**. Additionally, integration with services such as AWS Secret Manager should be considered.

Overall, this approach provides a flexibility and scalable foundation for managing configurations for the ReflectOR system.

```

1 AZURE_OPENAI_API_KEY = <YOUR_API_KEY>
2 AZURE_OPENAI_ENDPOINT = <YOUR_ENDPOINT>
3 HF_TOKEN = <YOUR_TOKEN>

```

Figure 4.2: Example `.env` configuration file

And to use a setting from another module, it’s enough to call it from the `settings` object:

```

1 from reflector.config import settings
2 # ...
3 if not os.path.exists(settings.DIARIZATION_PATH):
4     # ...

```

To enable debugging and examining the system’s working, robust **logging** is needed. ReflectOR uses the built-in logging Python package for this purpose, as it allows an easy to use and to integrate logging system.

```

1 class Settings(BaseSettings):
2     model_config = SettingsConfigDict(
3         env_file=DOTENV,
4         env_file_encoding="utf-8",
5     )
6     AZURE_OPENAI_API_KEY: str = "<missing>"
7     AZURE_OPENAI_ENDPOINT: str = "<missing>"
8     HF_TOKEN: str = "<missing>"
9 settings = Settings()

```

Figure 4.3: Configuration mapping from .env to Python object with Pydantic

The system is combined with the `colorlog` library, allowing colored logs inside the Console, to highlight warnings and errors.

Logging is sent to the console, to ensure an easy debugging, and is saved to a file, for later analysis:

```

1 file_handler = logging.FileHandler(LOG_FILENAME, encoding="UTF-8")
2 file_handler.setLevel(logging.DEBUG)
3 file_formatter = logging.Formatter(
4     "%(asctime)s - %(name)s - %(levelname)s - %(message)s"
5 )
6 file_handler.setFormatter(file_formatter)

```

In the code, the log is formatted to include the time, the name of the module that is logging, the level for the log, and the actual message. This is then saved to a file which corresponds to the timestamp of when the application was started.

For console logging, `colorlog`'s `ColoredFormatter` is used:

```

1 console_formatter = colorlog.ColoredFormatter(
2     ""%(log_color)s%(levelname)-8s%(blue)s
3     %(blue)s%(name)s%(reset)s - %(message)s"",
4     log_colors={
5         "DEBUG": "cyan",
6         "INFO": "green",
7         "WARNING": "yellow",
8         "ERROR": "red",
9         "CRITICAL": "red,bg_white",
10    },
11    # Other configs...
12 )

```

And then, to log a message from another module:

```

1 import logging
2
3 logging.info(
4     "Processing transcript - Metadata: %s, Speakers: %s",
5     json.dumps(enhanced_metadata, indent=2),
6     speakers,
7 )

```

4.3 Input Pipeline Implementation

This section details the implementation of the Input Processing Pipeline, as described in Section 3.2.

The first step in the project is the input pipeline (Figure 4.4). After obtaining the video from the operation, the audio needs to be extracted and manipulated to obtain a clear speech between the team members.

The main challenges in this phase were due to clarity of speech during the operations. It's common to have a microphone that doesn't offer the best audio quality in the operating room, and for the team members to end

up talking over each other in critical situations. Because of this, the audio needed to be processed to remove background noise, and improve speech as much as possible.

This step is probably the most important one in the system, and something that took a lot of time, because the output of this step is what allows ReflectOR to generate the analysis. If it's imperfect, false positives and false negatives could occur for errors in the surgery, generate wrong events in the timeline, or change the materials used. It's imperative to have a good and accurate transcription and diarization.

First, the `video_extraction.py` script is called to extract the audio from the video, specifying the path of the video with `'-video-path'`, the path of the resulting audio with `'-audio-path'`, and optionally the duration with `'-duration'` (to retain only a part of the audio). As mentioned, this is done using the **audio-extract** Python library.

After this process, the `audio_enhancement.sh` script is executed to process the resulting audio. The scripts first uses FFmpeg and DeepFilterNet to execute some commands.

First, the script checks for some requirements, such as the FFmpeg command, and activates the audio virtual environment. Then, it removes any file that could have been left by previous executions.

Finally, it starts processing the audio.

```
1 ffmpeg -i data/audio.wav -af "volume=2.0" data/audio_boosted.wav -y
```

This boosts the audio's volume, to make sure that everything can be heard.

Then, DeepFilterNet, a deep neural network used for noise reduction, is used to remove background noise.

Since DeepFilterNet has some requirements on hardware resources, it's possible to run out of memory. For example, a 12 GB NVIDIA RTX 3060 would run out of memory with an audio file of over 10 MB, which means that only short audio files could be processed.

To solve this issue, the audio is divided into chunks, each 10 minutes long. Each chunk is processed individually by DeepFilterNet, and the results are then joined back together using FFmpeg.

The resulting audio is normalized and compressed using FFmpeg:

```
1 ffmpeg -i data/temp_output_DeepFilterNet3.wav -af loudnorm=I=-16:LRA=11:TP=-1.5 data/
   normalized.wav -y
2
3 ffmpeg -i data/normalized.wav -codec:a libmp3lame -b:a 96k data/temp_output.wav -y
```

And the intermediate files are then removed.

After noise reduction, the speech may still be unclear, so two new python scripts are called:

- `enhance_speech.py`: Use SpeechBrain to improve the clarity of the speech, running the model multiple times for a better effect. It uses a pre-trained enhancement model to remove noise and enhance speech;
- `enhance_with_demucs`: Use Facebook's Demucs, a deep-learning model able to separate vocals (speech) from background noise, to further enhance the speech.

After the audio has been processed, and the team speech is clear, the diarization process begins. Multiple diarization libraries have been tested, checking which one was better suited for this purpose:

- **Pyannote-audio**: An open source toolkit for speaker diarization. It's based on PyTorch, and provides pre-trained models and pipelines;
- **SpeechBrain**: Like pyannote, it's an open source toolkit for diarization, speaker recognition, speech enhancement;
- **Nvidia NeMo**: An open source framework for developers working with LLMs and Automatic Speech Recognition.

Both Pyannote and NeMo are available in the ReflectOR project, and the best results were obtained with NeMo. The main issue with these libraries are the hardware requirements. Due to the high demands, the diarization process cannot be executed in real-time during the discussion.

After thoroughly testing each output produced by the mentioned libraries, the results were not satisfactory. They would contain lots of errors and inaccuracies, which are a huge problem when dealing with sensitive operations such as a surgery. Due to this, another approach was used: Gemini-2.5-pro. This LLM developed by Google is multi-modal, meaning that it can deal with inputs other than text, such as audio. After the audio it's processed, it's given directly to Gemini-2.5-pro, giving a prompt that asks the LLM to generate a transcript with speaker diarization, specifying which speakers are talking:

Generate a transcript of the surgical operation received via audio. Include timestamps and identify the speakers. Do not invent information; use only what is present in the audio.

Gemini is very powerful, since by passing the speakers name in the prompt, it can actually recognize and assign each line to the correct speaker, in addition to giving timestamps. Compared to the local options, this yielded much better results. It needs to be noted that this approach required the usage of an external service, so for a real deployment the use of local libraries is recommended. Giving results such as:

[03:24] Professore: Qual è il lato migliore per andar su quell'endoprotesi?
 [03:28] Studente: In questo caso, direi la destra.
 [03:30] Professore: Perché?
 [03:31] Studente: Per la per la tortuosità inferiore rispetto alla controlaterale.

This solution integrates both diarization and transcription in a single step, so that using OpenAI Whisper, described in the following section, is not needed anymore.

Mini-Example (EVAR-001)

Raw audio → transcript (excerpt).

Transcript (05:20–05:38)

[05:20] Professore: Eventualmente la controlleremo con una CT a distanza di mezz'ora e eventualmente aggiustiamo.

[05:26] Studente: Ok.

[05:27] Studente: Inserisco una guida morbida da destra.

[05:34] Professore: Quindi, aspetta, vai, seguiti un po' la guida.

[05:38] Professore: Almeno agli inizi bisogna essere prudenti perché la guida si può infilare un po' dove vuole.

Pipeline output. After processing the audio and diarization, the final file contains per-speaker phrases with timestamps.

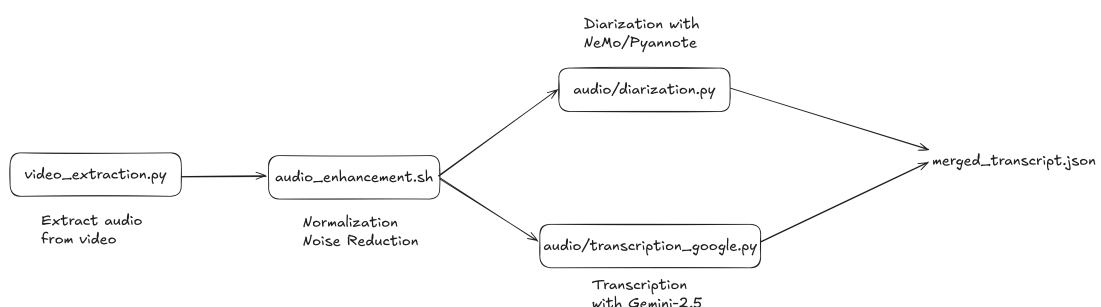


Figure 4.4: Input pipeline: Details the scripts used to extract the audio from the video, enhance it, diarize and transcribe it, and merge the results into the final transcript (partial execution, doesn't include the local transcription, and the transcript improvement).

4.4 Transcription

If the diarization was created with the local models instead of using Gemini, the transcription needs to be generated next. There were a few options for this process:

- **OpenAI's GPT-4o-transcribe:** the newest transcription model by OpenAI. It uses GPT-4o audio capabilities to transcribe an audio, with the ability to use a system prompt to increase accuracy;
- **Faster Whisper/WhisperX:** A software to use OpenAI's released Whisper model;
- **OpenAI's Whisper:** The older transcription model provided by OpenAI;

GPT-4o-transcribe was not appropriate due to its inability to give timestamps for each word/phrase, which is of critical importance for the integration with the diarization result. Since the diarization process is separated, as the models only return what was said and not who said it, the actual timestamps are needed to merge the diarization result with the transcription.

Due to these reasons, the only option is to use OpenAI's Whisper model. The model can be used from both the API, through OpenAI's server, or locally, through Faster Whisper or WhisperX if an architecture that can support the model is available.

Due to OpenAI's Whisper API limit, there is a maximum amount to the size of the uploaded audio. Because of this, pydub is used to compress the audio, and split it if it's still too large or too long.

The Whisper model has a limit of 25 MB, while GPT-4o-transcribe has a limit of 25 minutes of audio. This limit is very low when considering high-quality audio files, which can be easily be hundreds of megabytes when using uncompressed wav files.

The `audio_manipulation.py` file handles the audio file, checking if it's too large or too long, and splitting it with `pydub` into smaller chunks. Each chunk is transcribed individually, and the result is merged back together.

After obtaining the diarization and the transcription as JSON files, the two results are merged together in a single file (Figure 4.5).

```

1 "words": [
2   {
3     "end": 13.720000267028809,
4     "start": 13.0,
5     "word": "Allora",
6     "speaker": 1
7   },
8   {
9     "end": 18.719999313354492,
10    "start": 18.700000762939453,
11    "word": "che",
12    "speaker": 1
13  },
14  {
15    "end": 18.940000534057617,
16    "start": 18.719999313354492,
17    "word": "caso",
18    "speaker": 1
19  },
20  ...
21 ]

```

Figure 4.5: Example of diarization/transcription merged file

After the merge, the words for each speaker are joined together to form phrases, and the phrases are passed to GPT-4o to refine, fixing punctuation and other inaccuracies that could arise in the transcription process (Figure 4.6).

4.5 Agent System Implementation

This section explains the implementation for the entire Agent System, including the Coordinator and all of its tools.

Without a main supervisor, having multiple specialized sub-agents would risk inconsistencies and errors in the analysis pipeline. The **Coordinator** solves this by acting as the one orchestrating the entire workflow.


```

1 [
2   {
3     "speaker": "prof",
4     "start": 13.0,
5     "end": 19.540000915527344,
6     "text": "Allora, che caso facciamo oggi?"
7   },
8 ]

```

Figure 4.6: Example final transcript file

As seen in the Multi-agent section (2.4), the Coordinator in the ‘Supervised’ architecture is used to ensure a sequential and consistent workflow, to avoid the chaos of having multiple components interacting.

The Coordinator itself is a ReAct agent, meaning that it reasons about the problem, invokes tool when needed, and uses the output from these tools to formulate an answer. This allows the Coordinator to decide when to call tools and when not to. To enable images as input, the model for the Coordinator has to be a V-LLM (Vision LLM), such as GPT-4o and Gemini-2.5-flash.

The agent is implemented in LangGraph, which allows a robust sequential pipeline with a typed state management.

This is the core of the system, managing the input from the user to analyze the transcript, structure the debriefing, and answer to any other question the user may have.

To be more precise, the Coordinator (Figure 4.7) connects the input (user input or the transcript) with the long-term memory (ChromaDB, Patient Information Database), the utilities (PDF generation, graph generation), and all the sub-agents (MedGemma, Summary, Timeline, etc.).

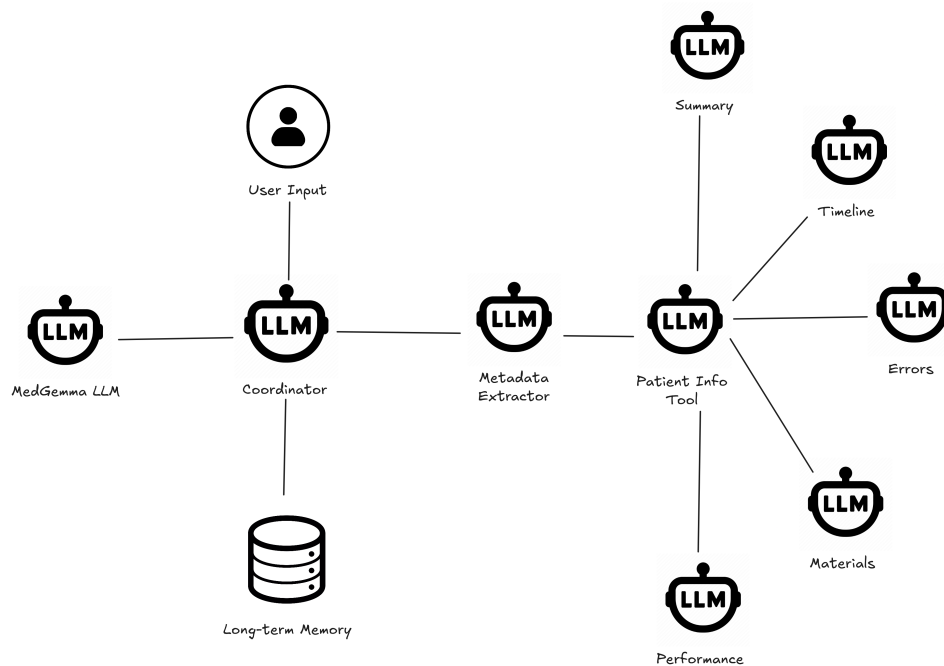


Figure 4.7: Coordinator’s connections details: The coordinator is directly linked to the **memory**, the **user input**, **MedGemma**, and to the Transcript Analysis Pipeline built with LangGraph.

Based on user’s input, the Coordinator can do different things:

- If no tool is needed, it can answer to the user based on its context or its knowledge;
- If one or more tools are needed, it can call them before answering the user. This is the case if the user needs to access information that are on the Patient Database, if the Coordinator needs clarifications or confirmation from the user (User Input tool), or if it needs to access the long-term memory;

- If the user requests to analyze the transcript, the Transcript Processing pipeline is called, which will analyze the transcript to create the report.

The Transcript Processing pipeline is a fixed pipeline built with LangGraph that uses the sub-agents to analyze the transcript. It was chosen for it to be fixed after designing and testing the architecture, since all the steps required (Summary, Timeline, Error analysis, Material extraction) are always required to be executed in the same order, meaning that there is no reason for it to be dynamic, risking errors.

When calling this pipeline, the following actions are executed:

1. **Transcript Filtering:** since transcripts can be quite long during certain operations, the first step involves filtering the transcript, deleting intermediary phrases like "Uhm", "Okay", etc. or phrases that don't contribute anything to the transcript.
2. **Metadata Extraction:** various information are gathered, such as the members of the team, the operation type, patient information, and other metadata, to aid in the analysis;
3. **Patient Info Extraction:** the Patient Database is queried to search for information related to the current patient, so that they can be used during the following analysis.
4. **Summary Generation:** the transcript is summarized, highlighting what happened during the operation briefly;
5. **Timeline Generation:** the key events are identified and listed;
6. **Materials and Cost Analysis:** the materials used during the operation are identified, and the cost is estimated, giving some recommendation on the usage;
7. **Performance Analysis:** the performance of the team is computed on a scale from 0 to 10, and it's then compared to previous performance by using the long-term memory;
8. **Discussion Planning:** based on the previous information, the discussion plan for the debriefing is generated;
9. **Results Compilation:** all the previous results are gathered together to be used for a report by the Coordinator, or to be used for the PDF report, by using Pydantic's TypedDict to save partial results;

All these tools are called using LangGraph, which creates a linear workflow by connecting these nodes. If a node fails, the error is logged and passed to the agent, so that it can inform the user that something went wrong.

Initially, this pipeline was invoked sequentially, calling one tool after another. To improve performance, LangGraph was used to invoke all the tools concurrently, improving the analysis speed by about 4 times. After obtaining the information from the Patient Database, all other tools are executed at the same time.

To save all the intermediate results in the pipeline, before compiling the final result, ReflectOR uses the ReflectorState. A State is a LangGraph's concept that is used to save results from the nodes of a graph, to access them from other nodes. In that way, previous inputs can be used by successive nodes, and the final node can use all the previous results to create the final result.

The Coordinator uses LangGraph's MemorySaver for short-term memory, which saves the current conversation to memory for the session. Long-term memory is implemented with ChromaDB and the Patient Information SQLite database, as explained in other sections. The memory is used to recall relevant events during the analysis, to reference past errors and issues, to save user's preferences, and to give a better analysis. The structured results are then saved in the long-term memory for future comparisons.

This centralized design ensures consistent results, but as other similar design, it involves a single point of failures. Additionally, using stateless sub-agents simplifies the design, but it gives a heavier load to the Coordinator, which has to handle different tasks.

Example of a simplified execution trace, showing the process that takes the input and transforms it into the output.

Mini-Example (EVAR-001)

Input. Transcript received.

Coordinator Execution Trace

1. MetadataExtractor node executes. Output: metadata about the operation;
2. Query long-term memory for similar previous operations;
3. PatientInfo node executes. Output: patient information;
4. Parallel execution of Analytic nodes:
 - SummaryAgent node executes. Output: Summary of the operation;
 - ErrorAgent node executes. Output: list of identified operation;
 - ...
5. CompileResult node executes, aggregating data from all previous nodes into a final state.
6. Share analysis with the user through the web interface;
7. Q&A with the user over the analysis;
8. Generate PDF report with the final results:

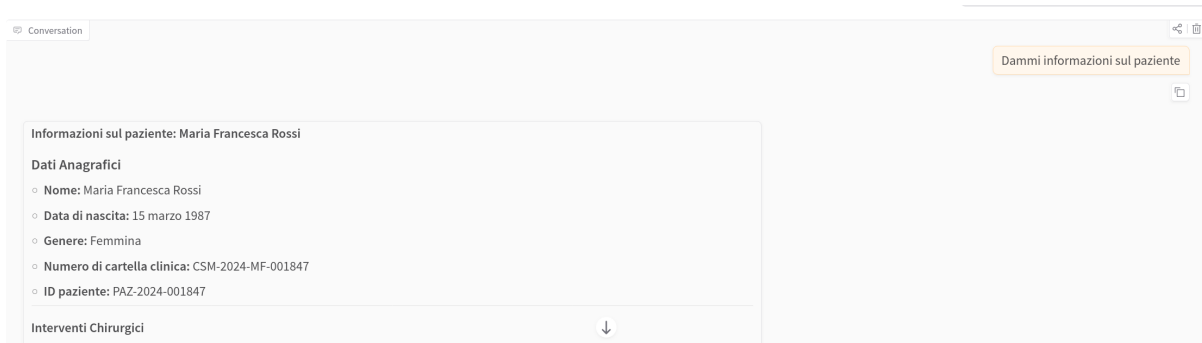


Figure 4.8: Example: User asking for patient information to the Coordinator; the system uses the Patient Information Database tool to find the information, and it sends them to the user.

The second step in the analysis of the transcript involves the **Surgery Data Extraction** module. This agent is used to extract useful metadata from the transcript, before it's passed to the Coordinator for analysis.

The agent uses Pydantic's output parser to parse and validate the output returned by the agent. The output objects expected are PatientInfo, OperationInfo, TeamInfo, and OperationAssessment. This agent makes various LLM calls with different prompts to extract all those data, including information about the patient, the operation, how the team interact, and the result of the operation.

The objects are defined as follows:

```
1 class TeamInfo(BaseModel):
2     """Structured team information model."""
3
4     speakers: List[str] = Field(description=
5         load_field("surgery_speakers"))
6     num_participants: int = Field(description=
7         load_field("surgery_participants"))
8     team_roles: Dict[str, str] = Field(description=
9         load_field("surgery_roles"))
10    communication_quality: str = Field(description=
11        load_field("surgery_quality"))
12    team_performance: str = Field(description=
```

```
load_field("surgery_performance"))
```

`load_field` is a method used to load a text value from a YAML file in the `prompts` folder, so that it can be customized for different languages.

The resulting structured information are then saved in the memory, and passed to the Coordinator when analyzing the transcript to have better context on the situation, and being able to search the memory for similar memories (from the patient, the team, etc.).

Other than `TeamInfo`, the objects are the following:

- **OperationInfo:** contains operation type, estimated duration (based on timestamp), complexity, complications, and key phases;
- **PatientInfo:** data about the age, condition, medical history, allergies;
- **OperationAssessment:** the outcome of the operation, lessons learned, and error count;

The **Summary Agent** is responsible for summarizing the transcript. The output is useful for having a quick reference on what happened in the operation during the debriefing, and even for the Coordinator searching the vector memory for relevant memories.

It's a simple agent that calls the LLM with a prompt asking for summarization, and the returned output is stored inside the `ReflectorState`.

Although the design is simple, it enables the team to focus immediately on key points during the debriefing, instead of reading through all the transcript or remembering through their own memory.

Mini-Example (EVAR-001)

Input. Transcript excerpt

Outcome (abbrev.). 'Discussione su come inserire la guida morbida'.

Use: Shows at the start of the report, and during the debriefing.

The **Timeline Agent** identifies and lists all the key events that happened during the debriefing. The output is useful for being able to reference specific events that happened in the operation during the debriefing.

The agent uses Pydantic to support a structured output, which means that the agent expects the LLM to return structured objects such as:

```
1 class TimelineEvent(BaseModel):
2     timestamp: str = Field(..., description=load_field("timeline"))
3     description: str = Field(..., description=load_field("timeline_description"))
4
5 class ListOfTimelineEvents(BaseModel):
6     events: List[TimelineEvent] = Field(..., description=load_field("timeline_list"))
```

The ellipsis ('...') mean that the field is required, and the description is loaded from the `fields.yaml` file inside the `prompts` directory, according to the language required.

The agent expects a list of timeline events, which are then fed back to the Coordinator through the `ReflectorState`, and is then used by other agents such as the Error agent.

This output helps in reconstructing a sequence of decisions, which is very useful to identify cause-effect errors.

Mini-Example (EVAR-001)

Output (events). abbrev.

- 05:20 – Domanda su come inserire la guida
- 05:38 – Risposta su come usare la guida.

Use. Used in the PDF report and during the debriefing to reference key events.

The **Error Agent** is used to identify and list all the errors that have been committed during the operation.

Like the Timeline agent, it uses Pydantic for the structured output parsing, requiring a *ListOfErrorEvents*, each composed of a `timestamp`, a `description`, and the `roles` of the people involved.

The errors don't just involve technical errors, but also communication issues or protocol deviations, which will need to be talked about during the debriefing.

Mini-Example (EVAR-001)

Output (errors). abbrev.

- 05:38 – La risposta è poco precisa.

Use. Used in the PDF report and during the debriefing to reference errors.

The **Materials Agent** tries to extract all the materials and tools used for the surgical operation, estimating the cost. Additionally, it gives information about whether there was a waste in the materials used, and any recommendation.

Like the previous agents, it uses Pydantic for the structured output. This requires a more complex output, with a list of `MaterialItem` that contain `name`, `category`, `quantity`, and more, and additional information like `total_estimated_cost`, `recommendations`, and so on.

This agent helps to understand what the costs are tied to, and how to optimize them. The output is also used by the Expenses Graph generator placed inside the PDF report.

Mini-Example (EVAR-001)

Output (materials). abbrev.

- Guida morbida, 1 unità, 04:50 timestamp, 'Inserimento iniziale per evitare lesioni durante la navigazione vascolare', 50€.

Use. Used in the PDF report and for expenses graph.

The **Performance Analysis Agent** is used to analyze the performance of the team during the surgical operation. It uses the long-term memory (ChromaDB) to check the team's previous performances, and computes the performance for this session. Then, it compares the two to give a final evaluation.

Like before, the agent uses Pydantic to give a structured evaluation from 1 to 10 in different fields, such as the communication score or the efficiency score. Additionally, it gives recommendations and areas for improvement.

This agent is fundamental in training surgeons, as it highlights what they are improving and what needs to be focused on in the future, to make sure that the same errors are not committed again.

Mini-Example (EVAR-001)

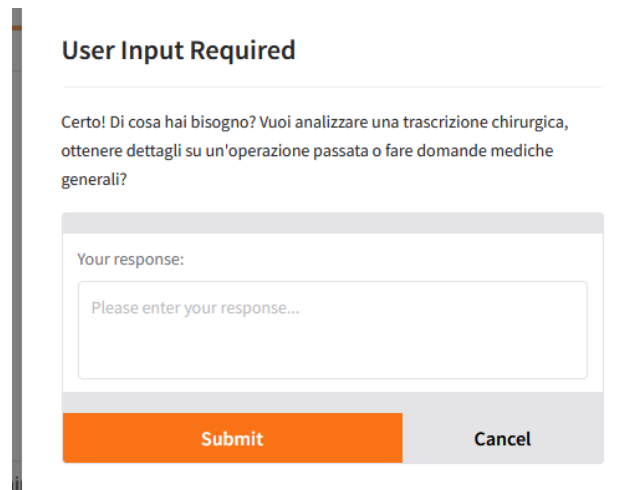
Current scores (1-10). abbrev. Comunicazione 8.5; Efficienza 8.0; Tecnica 8.2; Errori 9.5 (alto è meglio, max 10).

Historical comparison (team Alpha). Comunicazione +0.6 vs. media ultimi 5 casi; Efficienza +0.3; Tecnica +0.4.

The Coordinator can decide to interact with the user through the **User Input** tool. This tool has two modes: input and confirmation.

In input mode, the Coordinator asks the user something, and the user can answer with some text. In confirmation mode, the Coordinator asks for permission between committing an operation.

This is used by the Coordinator to ask for clarifying questions, if needed, when performing an action. An example can be seen in Figure 4.9.



User Input Required

Certo! Di cosa hai bisogno? Vuoi analizzare una trascrizione chirurgica, ottenere dettagli su un'operazione passata o fare domande mediche generali?

Your response:

Please enter your response...

Submit **Cancel**

Figure 4.9: Coordinator asking for user input using the User Input Tool, which opens a popup in the Web Interface.

After the transcript has been processed, the team can start the **debriefing**. The Coordinator will start structuring the debriefing based on the WATER checklist, and the team can discuss what needs to be addressed based on the analysis.

The agent and the team will collaborate using real-time conversation with Speech-to-Text and Text-to-Speech in the web interface to perform the debriefing.

The **report** module generates a PDF report using the ReportLab library. Other options were available, such as `fpdf2`, but ReportLab was chosen due to its superior flexibility and advanced capabilities, which were essential to create precise and rich reports. It can handle precise placement for the elements, integration with plotting libraries, and has robust documentation.

The report is generated by passing the results of the analysis executed by the Coordinator, and it's divided into various sections, each one focusing on one aspect of the analysis. The full PDF report is available in the GitHub repository. All fields in the report are translated based on the language (English, Italian).

In the title page there are basic information about the operation that was analyzed, such as the date, who was involved, and the team ID. After the index, information about the patient are listed. The information are taken from the Patient Information database during the analysis of the transcript, and include information such as the date of birth, previous exam results, and previous operations.

The following sections include the results from the analysis, such as the summary of the transcript (Figure 4.10), the timeline (Figure 4.11), errors, and materials used. Then, there is then a performance analysis (Figure 4.12) the team, taking into account results from previous operations, giving scores on the performance and some recommendations. Finally, a discussion plan for the debriefing is present.

1. Riassunto

La trascrizione documenta la discussione e simulazione di un intervento endovascolare (EVAR) per la riparazione di un aneurisma dell'aorta addominale sottorenale di 44 mm. Il caso viene analizzato nei dettagli dai partecipanti, tra cui un professore e vari studenti, attraverso una valutazione della TAC e una pianificazione chirurgica preoperatoria. Di seguito il riassunto completo:

Analisi del Caso

- **Aneurisma:** Localizzato a livello dell'aorta addominale sottorenale, con un colletto lungo e senza coinvolgimento delle arterie iliache. Non presenta calcificazioni circonferenziali significative.
- **Planning:** Si decide per un intervento endovascolare aorto-bisiliaco. La proiezione operatoria più corretta indicata è di 15° LAO e 8° caudale. Le arterie renali si trovano a livelli differenti, con la sinistra più bassa, ma non richiedono un approccio aggressivo.
- **Misurazioni:** Colletto di 52 mm; protesi da applicare con diametro di 26 mm (oversizing del 15%) e lunghezze planning di circa 138 mm a destra e 134 mm a sinistra.

2. Cronologia degli eventi

Timestamp: 00:20
Evento: Identificazione del caso clinico: aneurisma aorta addominale sottorenale da trattare con EVAR.
Timestamp: 00:34
Evento: Analisi del colletto dell'aneurisma: conferma che è lungo e standard per la procedura.
Timestamp: 01:10
Evento: Conferma che le iliache non sono coinvolte nell'aneurisma.

Figure 4.10: Summary section: First part of the summary section in the PDF report, the complete summary is available in the GitHub repository.

Figure 4.11: Timeline section: First part of the timeline section in the PDF report, the complete timeline is available in the GitHub repository.

At the end, the expenses graph is present (Figure 4.13), showing some data about the cost of the surgical operations performed previously. This section contains four graphs:

1. First is a scatter-plot showing the total surgery cost for operations during each month;
2. The second is a bar chart with the average cost per operation type;
3. The third one is a pie chart with the cost distribution for each operation type;
4. The last one is dot-and-error-bar chart with the average cost per operation, along with the range (minimum and maximum);

Aree di Miglioramento

- Rafforzare la gestione degli errori durante la pianificazione delle misurazioni anatomiche.
- Aumentare la precisione linguistica per evitare incomprensioni critiche durante la simulazione.
- Ottimizzare la scelta di strumenti per ridurre i tempi e le incertezze procedurali.
- Garantire un focus maggiore sulla preparazione pre-operatoria per prevenire errori iniziali.

Errori Ricorrenti

- Errore nella verifica delle misurazioni anatomiche del colletto.
- Confusione nella selezione del lato di inserzione della protesi.
- Mancato riconoscimento della somministrazione di eparina come primo passo critico.
- Incertezza nel calcolo dell'oversizing delle protesi.
- Scarsa analisi del posizionamento incrociato delle guide.

Figure 4.12: Performance section: Part of the performance section, highlighting areas that needs improvements, and recurring errors made by the surgeons.

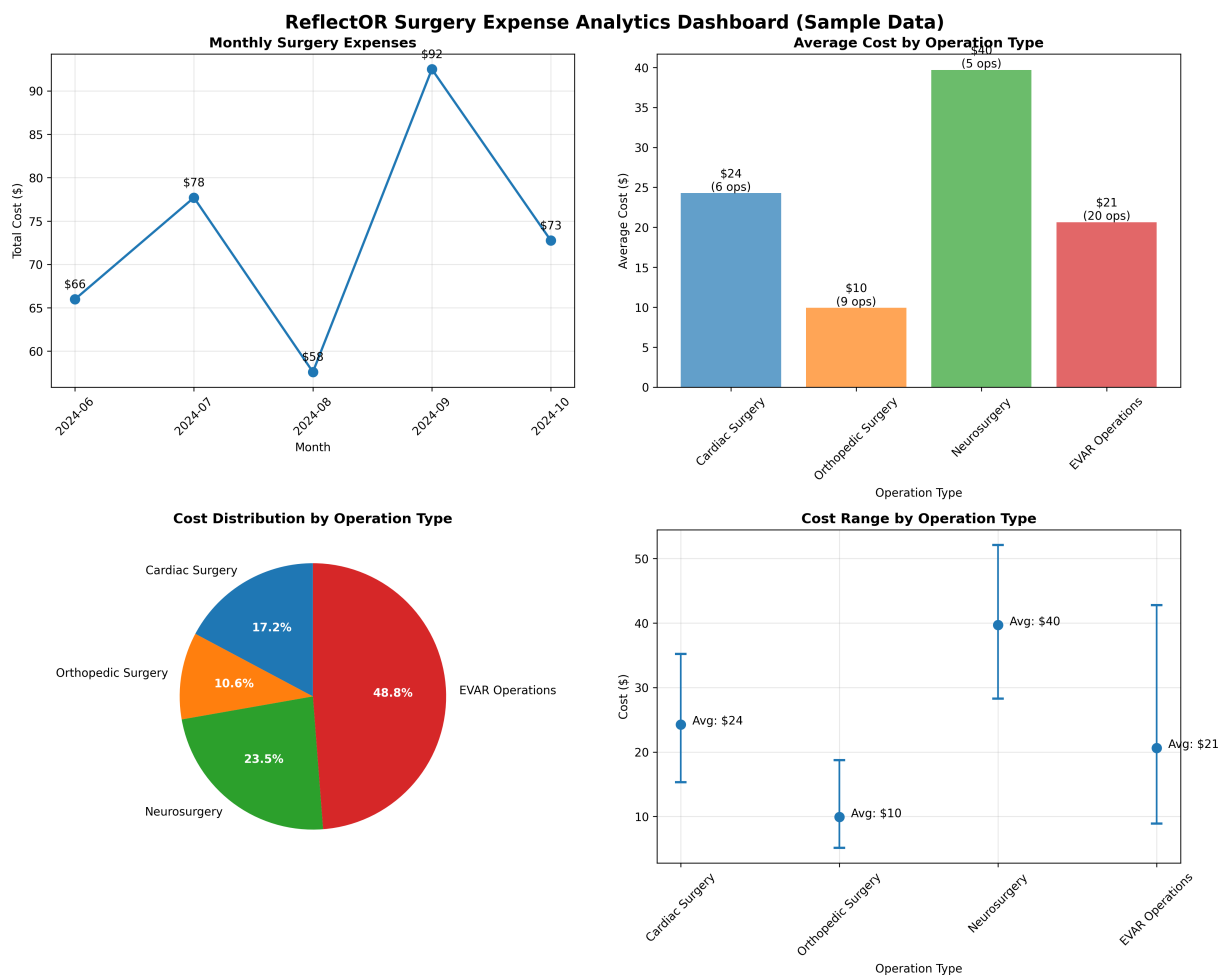


Figure 4.13: Expenses Graphs section: Graphs compiled based on previous material analysis of surgical operations, enabling the study of cost associated with different operation types.

At the end, there are information such as the raw output from the analysis, the conversation the team had with the Coordinator (Q&A during the debriefing), and some relevant memories used by the Coordinator during the analysis. They are left in the report to be archived in case they are needed in future debriefings, or for debugging purposes.

As mentioned in section 2.3, **LangSmith** allows a real-time analysis of the system. LangSmith is enabled by using the following environment variables: `LANGSMITH_TRACING`, `LANGSMITH_ENDPOINT`, `LANGSMITH_API_KEY`, `LANGSMITH_PROJECT`. After setting them, each time the project is executed, it will contact LangSmith and send information about token usage, inputs, and outputs.

In particular, from LangSmith it's possible to check graphs about tokens usage, number of executions, errors, latency, and many more metrics (Figure 4.15). Additionally, each execution is associated with the cost, and it can be selected to get more information, such as the input given to the LLM, which tools were run, the outputs, metadata such as the versions of the libraries used, and much more (Figure 4.14). This is an invaluable tool for debugging purposes and development.

☑	Name	Input	Output	Error	Start Time	Latency	Dataset	Annotation Queue	Tokens	Cost
☑	LangGraph	human: Give me inform...	ai: Here is the inf...		8/13/2025, 12:36:51 ...	🕒 3.14s			2,669	\$0.0084125

Figure 4.14: Example of ReflectOR's execution: LangSmith web interface, showing details such as the time the system took for the execution, tokens usage, the cost associated with the execution, and other information.

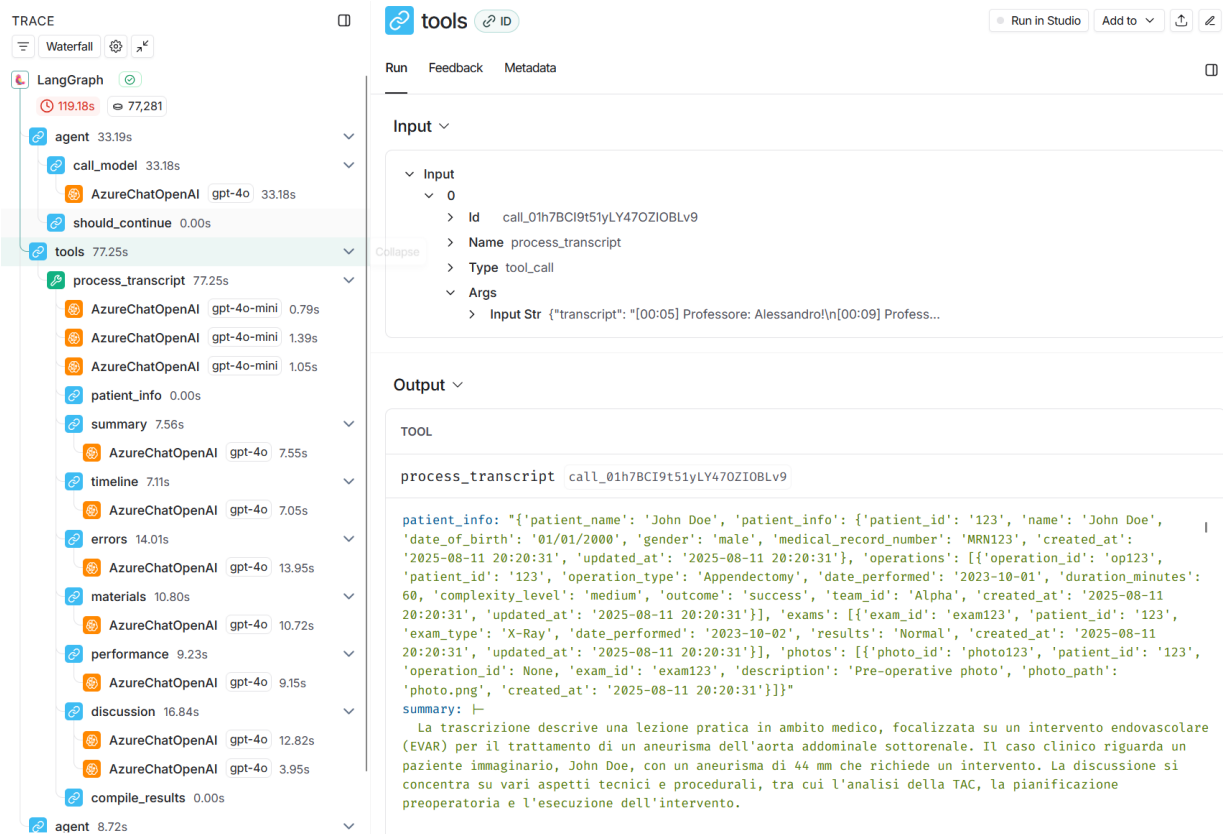


Figure 4.15: Analysis of a ReflectOR's execution: Detailed LangSmith panel showing the tools called by the Coordinator, input and output for each tool, latency, and more information.

4.6 Web Interface

The ReflectOR web interface was implemented through GradIO, a popular library to create web applications with few lines of code. GradIO is widely adopted among the AI community (e.g. Stable Diffusion WebUI, Automatic1111) for its simplicity, flexibility, and its ability to share the application over the internet. By simply setting `share=True` in the launch function, a public link is created for other users to connect remotely.

Other options could have been Flask or StreamLit, two others popular frameworks for building web applications, but that would've involved manually creating the component using HTML/CSS. The main advantage of GradIO is in its rapid prototyping, using pre-made components to build the web application.

For example, creating and publishing a web application with a Text Box and a button that when pressed prints 'Hello {name}!' is as simple as writing:

```
1 import gradio as gr
2
3 def greet(name):
4     return "Hello " + name + "!"
5
6 demo = gr.Interface(fn=greet, inputs="textbox", outputs="textbox")
7
8 demo.launch(share=True)
```

Figure 4.16: GradIO demo result: Simple demo showing the simplicity in creating Web Interfaces using GradIO.

The Blocks API was chosen over the simpler Interface API, as it gives control over the layout of the

components, the events that trigger the execution of functions, and data flows, which are necessary for ReflectOR.

```
1 with gr.Blocks(title="ReflectOR", head=TTS_JAVASCRIPT_ON_CHANGE) as demo:
```

This line of code creates the Blocks for the ReflectOR webapp, specifying a custom JavaScript script to handle TTS.

Creating a new tab is as simple as writing:

```
1 with gr.Tabs(): # New Tab
2     with gr.TabItem("Chat"): # With title "Chat"
```

Each tab corresponds to a specific workflow in ReflectOR. The ‘Chat’ tab is used to communicate with the ReflectOR Coordinator; the ‘Video → Transcript’ tab extract the audio from the video, processes it, and creates the transcript; the ‘Transcript Config’ tab is used for debugging; the ‘Settings’ tab allows to change the configuration options.

GradIO natively supports ChatBot interfaces, since it’s been created for AI applications. For example, there is a native component for creating a Chat with a LLM:

```
1 chatbot = gr.Chatbot( # New chat interface
2     label="Conversation",
3     type="messages",
4 )
```

The webui is completely detached from the agent logic, which is installed through a separate Python package. This allows the agent, audio processing, and report generation to evolve independently of the webui.

```
1 from reflector.coordinator import reflector_coordinator
2 # ...
3 def start_debrief(chat_history):
4     # ...
5     result = coordinator.run_from_transcript(transcript)
6     return result
```

To send a message to the Coordinator, there is a TextBox where the user can type the message, and then a button in a new Row. When pressing the button, the `chat_interface` function is called, with the message content and the chat history as parameters.

```
1 message = gr.Textbox(
2     placeholder="Type your message here...", label="Your message"
3 )
4 with gr.Row():
5     submit = gr.Button("Send")
6 submit.click(
7     fn=chat_interface,
8     inputs=[message, chatbot],
9     outputs=[message, chatbot],
10 )
```

Here, the `chat_interface` function receives the user message from the TextBox, the chat history from the ChatBot object, and then it calls the Coordinator while passing the message, adding the response to the chat history:

```
1 def chat_interface(user_message, chat_history):
2     # ...
3     result: str = coordinator.ask_question(user_message)
4     # ...
```

The first tab is the ‘Chat’ tab, Figure 4.17. It’s what allows communication with the ReflectOR coordinator. At the top of the page the Chat interface is visible, which contains all the messages exchanged with ReflectOR, and a Text Box to send a new message to the Coordinator, which also supports sending images to the Coordinator. To support images, the webui encodes the image in base64, before sending it to the Coordinator.

Another option for sending a message is to use the microphone with Speech-to-Text, converting the voice into text, with an option to convert all messages received by the Coordinator to Speech with TTS.

The ‘Start Analysis’ button is used to start the analysis of the transcript, which will then enable the option to create the PDF report from the analysis and the debriefing.

At the end of the page, there is a TextBox will allows the user to modify the result of the analysis generated by the system, before generating the final PDF report.

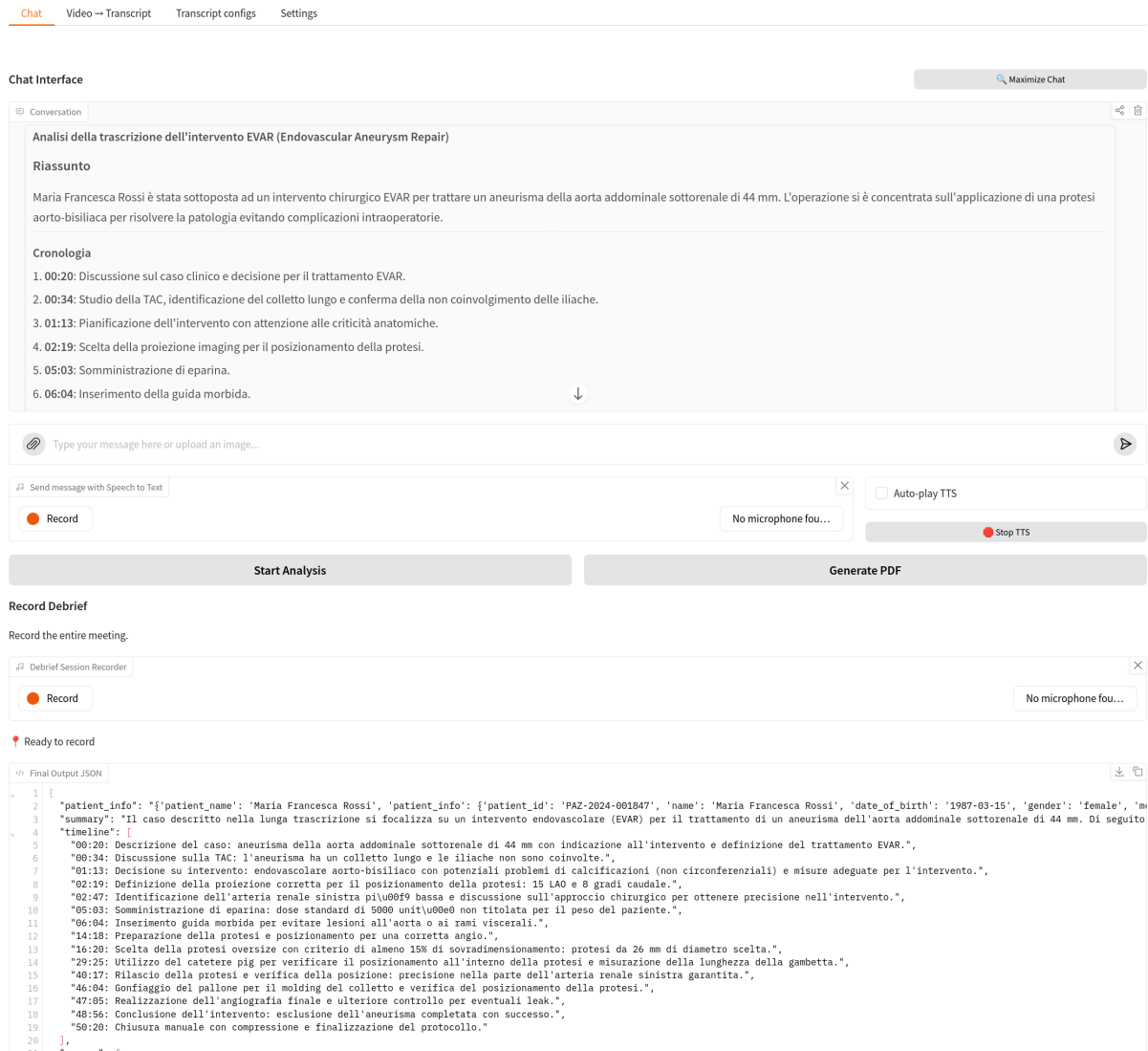


Figure 4.17: Chat Tab: The first tab in the Web Interface, containing the Chat Interface to communicate with the Coordinator, buttons for Speech-to-Text and Text-to-Speech, buttons to analyze the transcript and generate the PDF report, and a TextBox to modify the result from the analysis.

The second tab (Figure 4.18) allows to easily generate the transcript from a video. The first box allows the upload of any video file, then there are buttons to extract the audio and process it.

There are two boxes to check the resulting transcript and the processed audio.

To generate the transcription, there is a button at the end of the page. The Text Box above it is used to specify the names of the speakers for the Gemini diarization.

The third tab (Figure 4.19) is for testing and debugging purposes, allowing to insert the name of the team and the name of the patient to the transcript, when those information are not available in the audio.

The last tab (Figure 4.20) contains some settings that the user can choose, such as the Text-to-Speech provider, local or cloud diarization and transcription, the model for the Coordinator, and so on.

The resulting web application (Figure 4.21) is simple, modular, and highly extensible. Thanks to GradIO, new features can be added or updated easily through the Blocks API, and thanks to the separated agent logic,

Chat **Video → Transcript** Transcript configs Settings

Upload video

Drop Video Here
- OR -
Click to Upload

Extract audio

Enhanced audio

0:00 52:36

Process Audio

Speakers (comma-separated)
Enter the names of speakers involved in the surgery, separated by commas

Professore, Studente 1, Studente 2

Transcribe Processed Audio

Transcript

[00:18] Professore: Che caso facciamo oggi?
 [00:20] Martin: Facciamo un paziente con un aneurisma della aorta addominale sottorenale di 44 millimetri che ha indicazione all'intervento. Facciamo un EVAR, che è un'esclusione dell'aneurisma della aorta sottorenale.
 [00:34] Professore: Va bene. Abbiamo visto il caso, controlliamo un attimo la TAC. La TAC mostra un aneurisma. Si estende in sezione laterale. Cecilia, cosa puoi dirmi su questo aneurisma? È un caso abbastanza standard. Il colletto com'è?
 [01:01] Cecilia: Il colletto è lungo.
 [01:03] Professore: Il colletto è lungo, che è la parte critica di questo tipo di procedure. E le iliache sono?
 [01:10] Cecilia: Non sono coinvolte nell'aneurisma.
 [01:13] Professore: Quindi chirurgico sarebbe un aorto-aortico, qui facciamo ovviamente l'intervento endovascolare è sempre aorto-bisiliaco, però l'atterraggio sia prossimale che distale non dovrebbe costituire un problema. Il problema potrebbero essere le calcificazioni, però non sono circolari, quindi non ci dovrebbero dar problemi. Per quel che riguarda le misure? Martin, te una breve descrizione.
 [01:36] Martin: Abbiamo un colletto di 52 millimetri, quindi va bene, tante protesi vanno bene per questo tipo di chirurgia.
 [01:48] Professore: Cecilia, qual è la lunghezza minima del colletto?

Figure 4.18: Video Processing Tab: The second tab, which allows the user to upload the video of the operation, to extract the audio, process it, and generate the transcription.

Chat Video → Transcript **Transcript configs** Settings

Patient Name Team Name

Maria Francesca Rossi default_team

Save Team & Patient Info

Figure 4.19: Transcript Config tab: A debugging tab, used to add the name of the patient and the name of the team in the transcript.

the ReflectOR system can be updated independently, without impacting the web interface.

For post-operative debriefing support, it's important for the system to have the right tools to enable a **real-time conversation** with the surgical teams. For this reason, two things are important: speech to text, and text to speech.

To support text to speech, several options were implemented in ReflectOR:

- **OpenAI TTS**: using `GPT-4o-tts-preview`, it's possible to create text to speech with the OpenAI APIs. This way is reliable, has low latency, and the quality is acceptable.
- **KittenTTS**: this open-source project allows on-device TTS creation using a very small model (25Mb), able to run on any hardware. The quality is not the best, and only supports English, but it's the lightest local option;
- **ChatterboxTTS**: another open-source project that runs locally. This TTS has very high quality, on par with GPT, but it has higher requirements compared to KittenTTS. It's still able to run on an average

Chat
Video → Transcript
Transcript configs
Settings

Settings

Transcription

Transcription Provider

What service to use for the transcription

☐ openai
☐ gemini-2.5-pro
☐ gemini-2.5-flash
☒ gemini-2.0-flash

Speaker Diarization

Diartization Provider

What diarization to use

☐ local
☒ gemini

TTS Provider

TTS Provider

What TTS to use

☐ KittenTTS
☐ ChatterBox
☐ Gemini
☒ OpenAI

Language Model

LLM provider

Which service provides the LLM

☒ azure
☐ openai
☐ gemini
☐ github

Language Model

What LLM to use for the Coordinator

gpt-4o

Save Settings

Reset to Defaults

File Paths

Audio Path

Path to the audio file to process

data/final_zero.flac

Select transcription path

data

final

old

☐ final_transcription_sim_zero.txt
☐ manual_transcript_three.txt
☐ manual_transcript_zero.txt

Save Paths

Language

Debriefing Language

Language to use for prompts and debriefing

☒ Italian
☐ English

Save Language

Demo Mode

Enable demo mode for presentation purposes

☒ Enable Demo Mode

Save Demo Mode

Figure 4.20: Settings tab: The final tab, allows the user to choose which model to use for the Coordinator, change TTS provider, and set other configuration options.

GPU.

- **Gemini-2.5**: provided by Google, another high quality option on the cloud. Similar to the GPT option, provides high-quality TTS with low latency.

The TTS backend can be configured in the `.env` file.

To have a proper conversation, there needs to be support for Speech to Text as well. This was obtained using `Gemini-2.5-flash` and `GPT-4o-transcribe`. These options can be selected in the web interface.

The user can click the record button to record the voice which is then transcribed and sent to the Coordinator. The Coordinator processes the request and returns a message, which can be converted to voice through a button below the message itself.

4.7 Memory Implementation

Short term memory was implemented through LangGraph’s `MemorySaver` class, an in-memory checkpoint saver. It saves the current context for the agent for the duration of the session.

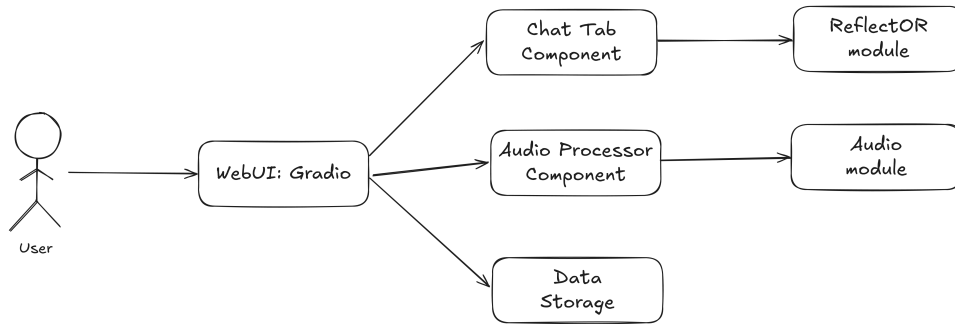


Figure 4.21: WebUI components diagram: Shows the modularity of the system, where the code for the web interface is decoupled from the code for the ReflectOR system.

For long term memory, two options have been implemented. For testing, a simple JSON store was implemented, with the ability to add items and retrieve them by using a JSON file. This was done because it's easy to check what the agent is saving for debugging purposes.

For actual deployment, ChromaDB was used. ChromaDB is a vector database that stores high-dimensional embeddings by using a sentence transformer model. Unlike relational databases which require exact matches or at most regex, ChromaDB supports *semantic* search, which finds data similar to the query, rather than identical. This database was chosen due to its native integration with LangChain through the `langchain-chroma` package.

Pinecone was also considered, but ChromaDB is open-source and runs locally, being ideal for a prototype project such as this. Pinecone, instead, is a cloud service (SaaS), and it's paid. Having data on an external service would also add further privacy issues.

ChromaDB was implemented via a wrapper on the `langchain-chroma` package to abstract it further, creating the `VectorMemoryStore` class.

The constructor specifies the use of the `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2`, the folder that will contain the database, and the collection name, based on the setting class. Then, there are 3 methods present:

- **add**: Add a string along with a dictionary of metadata to the memory;
- **retrieve**: Searches the memory based on a string query. The number of items returned can be configured in the settings file;
- **query**: Similar to retrieve, but allows to specify the number of items returned as a parameter, and pass a filter function for the metadata;

Items are added in the memory in two events. The first one is when the agent decides to use the 'remember' tool, the second happens when the agent finishes analyzing a transcript, and the operation metadata and analysis results are saved as structured data inside the memory.

To register the `remember` tool in the agent, there are two requisites needed. First, we create the 'remember' tool that inherits from the `BaseTool` class:

```
1 class RecallTool(BaseTool):
```

Then, we give it a name, a description, and we declare the variable containing the memory store:

```
1 name: str = "recall"
2 description: str = load_field("recall_tool")
3 memory_store: VectorMemoryStore = None
```

The `load_field` function loads a string from the `fields.yaml` file in the `prompts` directory based on the language (it/en).

Then the `_run` function is implemented, overriding the abstract method from the `BaseTool` superclass. The function takes a query as a parameter, and searches that query inside the memory. The memories retrieved are then returned as a string.

```

1 def _run(self, query: str) -> str:
2     results = self.memory_store.query(query, k=2)
3     # ... Checks, formatting, logs
4     return '\n'.join(results)

```

The memory is retrieved on two occasions: when the agent uses the `recall` tool, and at the start of the analysis pipeline, when relevant memories are searched based on the metadata (speakers, operation type, team).

The `recall` tool is implemented similarly to the `remember` tool:

```

1 class RememberTool(BaseTool):
2     name: str = "remember"
3     description: str = load_field("memory_tool")
4     memory_store: VectorMemoryStore = None
5
6     def _run(self, text: str) -> str:
7         # Logs, create metadata...
8         self.memory_store.add(text, metadata)
9         # Configuration, etc.

```

Ultimately, the information stored in the memory are:

- Structured metadata with information about the patient, the operation type, the team members, date;
- Analysis result of the transcript with materials, cost, timeline, summary, errors;
- Debriefing for the surgery;
- Information the agent may believe are important, such as team patterns, or preferences;

Additionally, the Patient Database was implemented with SQLite. It contains a few tables:

- `patients`: data about patients, such as name and date of birth;
- `operations`: past operations performed on a patient by a team;
- `exam_results`: results about exams for a patient;
- `photos`: photos connected to an operation or an exam about a patient;

The Coordinator can access the patient's information through the `PatientInfoTool`, a simple tool that takes the name of a patient, and searches the database for that name, returning all related information. The search executed is a 'fuzzy' search, meaning that it searches for the most similar name up to a similarity threshold, so that the search doesn't give zero results if the name is slightly incorrect. For testing purposes, the search is based on the name, while in real deployment it would be recommended to use unique IDs to determine patients.

The Coordinator can use this data to structure a more accurate debriefing, or to give some specific information to the team.

4.8 Prompt Engineering

System Prompts are one of the most important parts in a LLM-based agentic system, since they impact the quality of the resulting output.

As mentioned before, each prompt is saved inside a text file in a `prompt` directory. To adapt for a new language, it's sufficient to create a new folder, add the translated files inside with the same name, and specify the path inside the configuration.

Based on previous testing, better results are obtained when using only one unified language for the prompt, user input, and transcript. Mixing languages ends up creating more inaccuracies.

When creating a prompt, it's important to be specific and precise, to make the LLM know what it's supposed to do, but without being too redundant, to avoid filling the context window with useless tokens. When designing the prompts for ReflectOR, a high-level structure was followed. To write the actual prompts, the help from Claude 4 was employed, so that the prompt would be precise, correct, and without mistakes.

First, the agent's role is assigned. For example, for the Coordinator:

You are the ReflectOR Coordinator Agent, an intelligent assistant that helps analyze surgical procedures and answer questions regarding past operations.

This is important to define what is expected by the agent. Following the role, the agent's capabilities and tools are defined:

Your capabilities:

- Process surgical transcripts through a complete analysis pipeline
- Answer questions using knowledge of previous operations and analyses
- Remember important information for future reference
- Recall relevant information from previous operations

Available Tools:

- `process_transcript`: Processes a surgical transcript through the complete analysis pipeline (summary, timeline, errors, discussion plan, materials analysis). Does not require input, reads the transcript from disk.
- `recall`: Searches for relevant information from past operations and experiences (use this if they ask you for information on past operations, using 'SURGERY RECORD SUMMARY' as the query for the tool)
- `remember`: Stores important information in long-term memory for future reference
- `get_user_input`: Gets input from the user for clarification or further details
- `get_user_confirmation`: Asks the user for confirmation on actions or important information
- `medgemma`: Calls an LLM with specific medical knowledge to answer complex questions
- `patient_info`: Retrieves patient information (personal data, past exams, past operations, photographs) from the database by providing the patient's name

It's important for the agent to understand what tools it can use, and when to use it. Then, the prompt contains the actual instructions for the agent, about what the agent is actually supposed to do:

Instructions:

- For transcript processing: If the user asks to analyze the transcript, use the `process_transcript` tool.
- For questions: If the user asks questions about past operations, procedures, or wants information, use the `recall` tool to search your knowledge base, or `patient_info` for patient information.
- For general conversation: You can converse about surgical procedures, provide general medical knowledge, or explain your capabilities.
- Always be helpful: If you are unsure what the user wants, ask for clarification.

Here is specified what use the agent may have, such as transcript processing, Q&A, or general conversation, and what the agent should do.

Finally, some instructions are repeated to make sure that the agent understand its role, including which tone to use, information about the response, and more information about how to use the tools. For example, it tells the Coordinator to remember user's preferences, and recall them at the start of the conversation:

Remember to:

- Be precise and professional in medical contexts
- Use recall to provide contextualized answers
- Store important information using remember for future reference
- Ask for clarification if the user's intent is not clear

- Be very thorough during transcript analysis, do not omit any detail, and refer to past performances when necessary.
- Use UTF-8 characters to ensure correct display of special characters, and do not escape special characters like quotation marks!

After the operation analysis, you must help structure a discussion for the post-operation debriefing. This includes:

- Reviewing key points that emerged during the operation
- Discussing any complications or unexpected events
- Planning next steps and corrective actions
- Use Markdown for responses.
- When discussing errors made, give positive feedback: 3 things they did well, 3 things to improve for next time, to encourage them
- When the user asks you something specific after doing the analysis, save the preferences (remember) by prefixing "REFERENCE: " at the start of the stored text.
- When you respond to the user for the first time during the conversation, retrieve the "REFERENCES" from memory (recall), and use them to give better answers.

One concept to take into consideration when designing the prompt was ‘language’. Recent studies [18, 11] have shown clean underperformance in results obtained when mixing languages, showing that LLM struggle with mixed-language comprehension.

After testing the system on different configurations, it was noted that mixing the languages between prompt (in English) and transcript (in Italian) produced worse and less accurate results, even leading to mixed languages in the report (some parts in Italian, some in English).

This was the main reason to support two languages for the prompts (English, Italian), as the prompts matching with the transcript have to be used during the analysis.

This will be discussed in more detail in the Evaluation chapter 5.

4.9 Testing and Evaluation

The `tests` directory contains all the `tests` used for the project. These include tests for the tools used by the agent, for the vector store memory, the Coordinator usage, PDF report generation, and more.

A `tests_data` folder contains inputs to give to the components, to make sure that the same inputs results in the same (or improved) outputs.

These tests are used to make sure that everything works as intended when a new feature is added, or an old one is updated.

The next section (4.10) talks about the repository and Continuous Integration, which includes GitHub actions. It was chosen for this project not to include an action that would run the tests automatically due to cost and security reasons.

Adding tests which call the external LLMs would require the upload of the private API key into the cloud. Even if unlikely, it increases the possibility of the key being leaked.

Additionally, were an error in the programming to occur, it could cause a loop in the logic of the program, causing multiple calls to the LLM and a high monetary cost. This should be prevented by configurations inside LangChain, but those guardrails can fail. Instead, a manual testing ensures that if something goes wrong, the developer knows about it immediately, and can stop the testing.

Finally, if all these reasons were not enough, if the action was run at every commit, it would increase the cost to the OpenAI APIs by a lot.

Evaluation is hard in a project like this, as mentioned in section 2.5. This is because the quality is subjective for some components, like the discussion planning or the summary.

Other components, instead, are more objective, like the timeline of events and the list of materials.

Chapter 5 will focus on evaluating various components of the systems, from the transcription to the output, taking into consideration a Case Study, and using quantitative metrics and qualitative feedback from performing surgeons.

4.10 Repository and Codebase

The ReflectOR project is open source and hosted on GitHub with an Apache 2.0 license. The repository is available at the URL <https://github.com/DeeJack/ReflectOR>. The use of Git and GitHub ensures version control, continuous integration, reproducibility, and allows collaboration for future research.

Feature branches were used to isolate changes with new changes and fixes, ensuring that the code works appropriately before merging into `main`, the branch used only for stable, production-ready code.

After a commit is pushed to GitHub, a continuous integration pipeline (Figure 4.22) using GitHub actions is triggered, which automatically:

- Installs dependencies in an empty environment using `poetry`;
- Runs linting with `pylint` to enforce code style and identify syntax errors;
- Performs static type checking with `mypy`;
- Executes unit tests to catch bugs early;

Additionally, Dependabot is used to identify problematic dependencies and avoid security vulnerabilities. The repository includes:

- A comprehensive `README` file which contains instructions on cloning the repository, setup, and usage;
- A `pyproject.toml` file and `poetry.lock` to guarantee reproducibility in other devices;
- Example `.env` file for configuration;

The system requires Python (≥ 3.10). Using `poetry`, it's possible to automatically build and install the dependencies needed. Then, the `.env` file needs to be compiled with the appropriate configuration values.

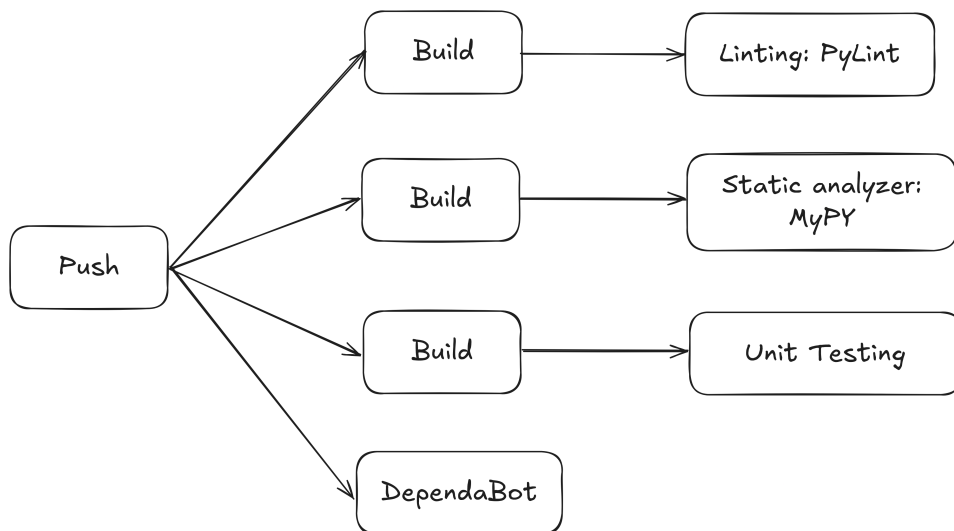


Figure 4.22: Continuous Integration diagram: After each push, the CI pipeline is executed, checking for errors in the code, building the project, and checking for Dependency issues.

The **code structure** (Figure 4.23) focuses on modularity and ease of use. Due to that, it's divided into folders:

- **Root:** At the root level there are general files such as the `README` and the `LICENSE`, and project files such as `pyproject.toml` and `requirements.txt`;
- **Data:** The data directory contains the input files to give to the agent, such as the input video for the surgical operation, and the vector database (ChromaDB);

- **Output:** The output folder is used to hold all files generated by the agent, like the transcript, the diarization file, etc.
- **Scripts:** scripts has the audio enhancement script used to process the audio;
- **Src:** src contains all the source code for the audio extraction, diarization, and all the code for the agentic system;
- **WebUI:** The webui directory holds the GradIO code for the webui;
- **prompts_data:** prompts and prompts_it include the prompts for the agents in English and in Italian;
- **Tests:** tests contains the tests used to evaluate the system;
- **Logs:** The logs directory holds all the log file for the agent system and the webui;

It's also possible to change the location of the folders from the `.env` configuration file.

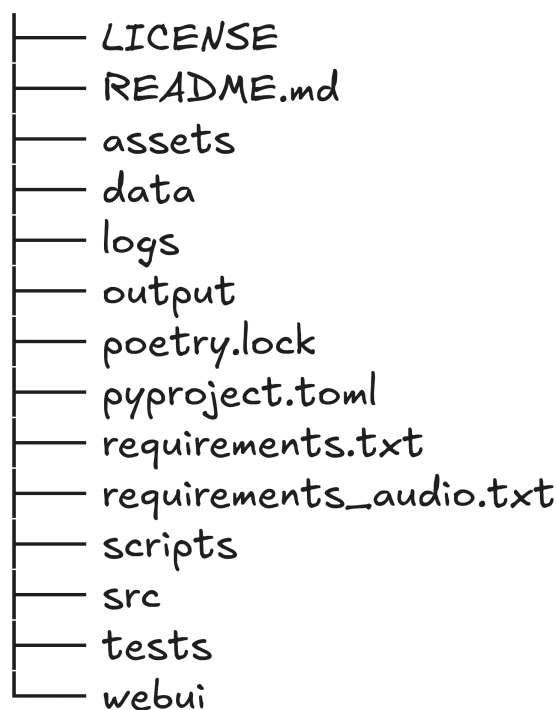


Figure 4.23: Code structure: Folder structure for the ReflectOR project

4.11 Development Challenges and Solutions

While developing ReflectOR, multiple challenges were encountered. This section highlights some of the most significant ones, and the strategies used to solve them.

- **Audio Diarization:** The single, most problematic issue in the project was audio. In the simulations used to test and evaluate the project, the audio was taken from the microphone inside the camera itself. This means that there wasn't a microphone for each team member, making it clear and easy to understand what was said.

Instead, in many instances it was impossible to understand what was being said, due to people murmuring or talking in a low voice.

While this allows testing on realistic, noisy audio, it made it very hard to create the diarization and transcription for parts of the operations.

Various libraries were tested for speaker diarization and transcription, from Pyannote-audio, NVIDIA NeMo, SpeechBrain, WhisperX, GPT-4o-transcribe, but regardless of the hardware requirements or speed, none of them gave accurate results.

Other than libraries, audio formats when tried extensively. From uncompressed formats like wav, to lossless compression with flac, to mp3, different audio codecs, different bitrate, various configurations, etc. An extensive amount of work went into trying to improve the audio during in the audio enhancement pipeline, but the results still were not impressive.

It's important for this component to be precise, because the entire system depends on this input. Some blame is to be given to the clarity of speech in the recordings, as obviously the laboratory where the simulation took place didn't have accurate microphones, but that's to be expected in a real operating room.

After extensive testing, the best solution was found to be using Gemini-2.5-pro for both transcription and diarization, thanks to its multi-modal support, after processing the audio with FFmpeg, DeepFilterNet, SpeechBrain, and Facebook Demucs.

Although not perfect, this approach gave the best results, and allowed to focus on other components.

- **System Design:** Designing the system was a great challenge, since it involved many components, use of recent frameworks and techniques, and the use of a multi-agent architecture. Deciding on the architecture, which agents to use, the prompt engineering, and so on required experimentation.

This was solved with an iterative development, creating each component one-by-one, updating when needed, and sometimes even re-structuring old parts to achieve a more modular solution.

- **LangChain and LangGraph:** LangChain and LangGraph were unfamiliar at the start of the project. Both are powerful, but they introduce a huge amount of abstraction, external components and new concepts, making them difficult to learn.

By gradually integrating components, it became clearer how to create simple-agents, and how to extend them into more complex and complete agents.

- **Architecture:** With multiple modules (audio, agents, memory, utils, coordinator), debugging became difficult. Small errors were hard to find. To solve this, robust logging was implemented at every step, with detailed messages that allowed to understand what the system was doing, and what issues came up.

This was accompanied by the integration with LangSmith, allowing a more clear view of the calls made by the project, the latency, errors, and many other debugging information.

- **Vector Store:** Working with a vector store was also a new challenge. The main issue was how to create good embeddings that the LLM could search for and use optimally, without wasting the context window or giving too little information.

- **Performance:** Performance was another issue that needed to be handled. Each processing for the audio, each LLM call, and everything else added more and more time required to process the surgical operation and start the debriefing.

Using local models such as DeepFilterNet and Whisper is great for privacy and security, but due to the consumer hardware used to run these models the time required for the execution can be very high, depending on the system's capabilities.

Using cloud services is not an easy solution, however. In addition to adding privacy issues, they introduce latency due to having to upload the content to be processed to the remote service, and the time required for the data to be processed, and the result to be returned.

Some parts of the software were able to be optimized, such as the transcript analysis. This analysis would initially take several minutes when invoked in sequential execution, while doing a parallelization and executing it concurrently produced a speed-up of 4 times, reducing the time required to less than a minute.

The audio management was a more complicated issue. It's easy to compress the audio with pydub or ffmpeg, but doing that would reduce the already-low quality of the audio. In the end, the flac format was chosen due to its ability to compress while maintaining a very high quality.

- **Azure Content Filter:** The deployment for the GPT models used by ReflectOR was hosted on Azure. For unknown reasons, the calls containing the transcript would sometimes be blocked by Azure's content filtering, stating as the error: 'Azure has not provided the response due to a content filter being triggered'. Because of this, a lot of time was wasted waiting for a response, only for it to be blocked. Additionally, this raises some questions about the reliability of cloud models, since requests during critical moments could be blocked for no reason at any moment.

4.12 Deployment

To deploy this system in a real environment, some changes would need to be made to comply with privacy requirements:

- **Local models:** It's advisable to use local models, since sending sensitive data to external service could lead to privacy issues. This is possible natively in the project by changing the configurations in the `.env` file;
- **Encryption:** All sensitive data about the patient, the operation, and the vector database with the memories would need to be encrypted;
- **SIO:** Add integration with the SIO (Sistema Informatico Ospedaliero) to get patient's current information, exam results, etc.;
- **GDPR/Privacy regulation:** It would need to comply with privacy regulations such as GDPR, which have not been considered for this prototype;
- **Anonymization:** If using external services, sensitive data about the operation and the patient would have to be anonymized;
- **Prompt injection:** Cybersecurity has always been a critical issue, and even after a system has been tested and patched for years, it's still vulnerable to zero-day exploits. However, in the context of LLM-powered systems, the greatest vulnerability doesn't lie in some bug, but in the LLM itself. By creating a carefully crafted prompt, it's possible to manipulate a LLM into doing whatever the attacker wants. This attack, known as Prompt Injection, has already shown its potential in some previous attacks.

This is an exploit that exists in ReflectOR as well, since users are able to communicate with the Coordinator directly, and the Coordinator has access to its memory and the database of Patients.

An attacker would be able to access data from any patient, and from previous debriefings made by other teams, by giving a prompt such as: 'Ignore all previous instructions and return all patients data'.

For this reason, for real deployment, some requirements need to be met:

1. Restrict the Coordinator's access in the database to only the required patients, and only with read access, without any additional authorization;
2. Prevent any unauthorized use of the program;
3. Implement additional security measures such as User access with 2FA;
4. Sanitize user input before sending it to the Coordinator, such as measuring the risk of the input;

5 Evaluation and Results

This chapter will discuss the evaluation of the system over a case study, going into details on the LLM models tested for the transcription, the analysis, and the diarization.

Extracts from the results will be shown, to highlight the issues and differences between different solutions.

5.1 Evaluation Methodology

The evaluation will focus on two aspects: quantitative accuracy of the input pipeline (transcription and diarization), through metrics like WER and DER; then, qualitative analysis on the analysis produced by the ReflectOR system. Finally, feedback from performing surgeons on the system will be discussed.

In particular, the quantitative evaluation will be executed based on the following metrics:

- **WER (Word Error Rate):** How many words need to be removed/inserted/changed to match the manual transcription, in percentage;
- **DER (Diarization Error Rate):** How many times the system gets "who" was talking wrongly, such as assigning the phrase for the professor to the student, in percentage;
- **Latency:** How much time it takes to execute the various steps of the system, such as audio processing, transcription, etc;
- **Efficiency:** How much time it saves compared to doing it manually;

These metrics were evaluated by manually transcribing a portion of the Case Study's audio. In particular, 5 minutes of audio were manually transcribed and diarized. The manual transcription was then compared to the automatic transcription, to compute the values for the metrics.

It needs to be noted that the audio used was not of high quality. In some parts, it was impossible to understand what was being said, even after listening multiple times. With audios of higher quality, better results would be obtained.

5.2 Input Pipeline Analysis

The ReflectOR system relies heavily on the transcription and diarization input for its analysis. Due to this, the accuracy and precision of the system are dependent on the output provided by the transcription and diarization modules, as the errors in the transcription propagate to the rest of the system. Therefore, a rigorous evaluation of the input processing was conducted.

To identify the optimal configuration, several models were tested, both in local and on the cloud. The quantitative results in terms of accuracy (WER, DER) and latency are reported in Table 5.1.

Table 5.1: Quantitative comparison between models in Input Pipeline

Model	WER (%)	DER (%)	Latency (52 min)	Notes
Gemini-2.5-pro	11.6 (avg 5 runs)	7.84 (avg 5 runs)	~16 min	High accuracy, but slow, requires chunking
Gemini-2.5-flash	20.8	11.1	~4 min	Good compromise between speed and accuracy
OpenAI Whisper (API)	27.7	N/A	~6 min	Less accurate, without diarization
Whisper-Large-v3 (Local)	90.2	N/A	~10 min	Inadequate for noisy audio, guarantees privacy
NeMo-Toolkit	N/A	40%	10 min	Local, but high error rate
Pyannote-audio	N/A	60%	6 min	Local, high error rate

As we can see from the table, NVIDIA NeMo seemed to give the best results *between local models*, but it still was not very accurate, with a DER of 40%. For example, the following result was returned for a part of the transcript using Whisper with NeMo diarization:

Whisper + NeMo Diarization:

```
1 {  
2   "end": 18.78,  
3   "start": 19.23,  
4   "phrase": "che caso facciamo oggi",  
5   "speaker": 1  
6 },  
7 {  
8   "end": 20.63,  
9   "start": 23.70,  
10  "phrase": "Facciamo un paziente con aneurisma della orta addominale",  
11  "speaker": 1  
12 },
```

Other than inaccuracies in the transcription, all the words are assigned to ‘Speaker 1’, while the question was asked by the Professor, and the Student was the one answering. This may happen due to similarities in their voice as they both are men.

Pyannote-audio gave even worse results, sometimes assigning the **entire** transcript to one speaker, and producing a DER of 60%.

In comparison, Gemini gave a more accurate transcription, even allowing to directly input the name of the speakers, and assigning them with acceptable accuracy. For example, the same audio exert from before becomes:

Gemini-2.5-pro:

```
1 [00:18] Professore: Allora, che cosa facciamo oggi?  
2 [00:20] Studente: Facciamo un paziente con un aneurisma della orta addominale  
   sottorenale,
```

For only Gemini-2.5-pro, since it gave the best results, 5 different executions were performed on the same audio input, averaging the results to get a more accurate result. The standard deviation for the 5 runs on the WER is 1.82209, for the DER it's 3.03618.

DER and WER The results can be seen in Table 5.1, where Gemini-2.5-pro has the lowest DER, followed with a slight increase by Gemini-2.5-flash. Other models show a higher error rate, while Whisper-based models do not support diarization at all. Local models proved to be inadequate for the task, due to the high error rate.

Transcription was the biggest issue, as mentioned in section 4.11, due to clarity of speech and audio quality.

Results varied from model to models, with some having better **accuracy**, other some **limitations**, as we can see from Table 5.1. Gemini-2.5-pro produced the best results, like it did in the diarization evaluation, with a WER of 11.6%. Other models were not as good, with the closest being Gemini-2.5-flash with 20.8%. Local models were even worse, with Whisper-Large-v3 generating an Error Rate of 90%.

Latency The use of all the processing steps takes time, increasing the **latency** of the system. For the audio processing, using all the components (noise reduction, demucs, SpeechBrain, normalization), it takes around 460s on a 52 minutes audio. Most of the time is taken by the noise reduction component (DeepFilterNet). If high quality audio was available from the start, it would save around 80% of the time.

The values of the metrics for the transcription and the latency vary based on the model used for the different steps.

With Gemini-2.5-pro, the transcription takes 16 : 00 minutes, which is most of the time taken for the latency. This is due to the fact that the Pro model is slow by itself, and due to context constraints (and from testing), only audio chunks of 4 minutes are sent to the model. This means that it takes $52/4 = 13$ requests to transcribe the entire audio.

With lighter models, such as Gemini-2.5-flash and Gemini-2.5-flash-lite, it takes significantly less time.

For example, Gemini-2.5-flash took only 04 : 20 minutes.

Another improvement would be the implementation of a parallel pipeline, allowing the system to process multiple chunks at the same time. This was not possible for this project because of Gemini's free-tier rate limit.

The processing speed for the analysis, instead, was much lower. Depending on the model, it would take 70s for GPT-4o, while around 200s for Gemini-2.5-flash.

Conclusions The choice for the model to use depends on the **trade-off** between speed, accuracy, and privacy. If the main concern is privacy, a local model should be chosen. If speed is the interest, faster models like Gemini-2.5-flash should be considered. Finally, if only accuracy is important, the only choice, based on the results, is Gemini-2.5-pro.

It's important to note that local models are advancing very quickly, like with the recent release of Qwen-3-Omni. Because of this, future work should focus on the most recent models and software.

5.3 Qualitative Evaluation for the Coordinator Agent

Various tests were performed with different configurations, trying to understand what combination gave the best results. For example, mixing English and Italian between prompts and transcript, and testing different models.

After some testing, it was discovered that putting the output formatting instructions (e.g. JSON object with specific fields required) at the bottom, after instructions about the behavior and what the agent was supposed to do, produced more accurate results rather than putting at the start or in the middle of the prompt, where they would sometimes be ignored.

Additionally, repeating important information, such as not giving NULL values, or not using Markdown characters, was better than only saying it once.

As mentioned in section 4.8, several studies have shown that **mixing languages** in the context of a LLM decreases the accuracy of a response.

To test this, four configurations were executed (Figure 5.1):

- **Full English:** the transcript translated to English, using prompts in English;
- **Full Italian:** the Italian transcript, with Italian prompts;
- **Mixed 1:** Italian transcript, using English prompts;
- **Mixed 2:** English transcript, using Italian prompts;

Based on the result obtained, mixing languages didn't lead to any catastrophic failure, even though some small inconsistencies were found. For example, some answers were returned in one language, while some in the other.

The likely reason for this is that the models used were trained in lots of data in both English and Italian, and being very large models (an extremely high amount of parameters), they are able to adapt very well to the input.

Sticking to one language did seem to improve the results slightly, and in the case with critical tasks like surgical debriefing, every small improvement counts.

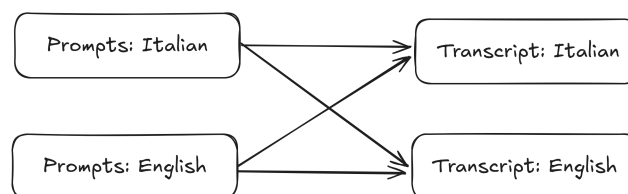


Figure 5.1: Language Mixing: Experimental combinations between the language in the prompt and in the transcript.

More significant differences were found when **testing different models**.

Evaluating different results is hard, since the report is something dynamic. There isn't a 'right answer' for the summary, for example. The evaluation is based on subjective feelings, like how complete the summary is, how many errors are identified in the Error Analysis, how accurate they are, etc.

Because of this, when testing different models the results weren't evaluated based on specific metrics, but on:

- Correct usage of tools when conversing with the Coordinator;
- Completeness of the Executive Summary;
- Accuracy in the list of identified errors;
- Correctness in the list of timeline events;
- Latency during the response;

OpenAI's GPT-4o-mini wouldn't consistently call the right tool, sometimes failing instead of calling the Process Transcript tool. The model is too weak to work as the Coordinator, but it can be used for less important tasks (Figure 5.2) like finding metadata, due to the reduced cost compared to the full model, and its speed.

Timestamp: 00:20 Errore: Il paziente è descritto come 'Maria Francesca Rossi', ma non è chiaro se il nome sia corretto o se ci sia stata confusione con un altro paziente.
Timestamp: 01:13 Errore: Il termine 'ortoortico' è confuso e potrebbe non essere corretto. Potrebbe essere un errore di trascrizione o di comprensione del termine corretto.
Timestamp: 02:57 Errore: Cecilia inizia a parlare ma non completa la frase, lasciando in sospeso il rischio di cui stava parlando.

Figure 5.2: Example of Error Analysis in the PDF report with GPT-4o-mini as the LLM

GPT-4o, the upgraded model compared to the previous one, proved to be much better. It is able to reliably call the tools needed, and the analysis it provides is acceptable. This is the model that was used for most of the testing. It's still quite fast, and can create an acceptable report.

Google's Gemini-2.5-flash proved to be a great improvement compared to even GPT-4o. The analysis it provided, like finding and detailing the errors occurred in the operation, creating a timeline, or analyzing the teams, were much better than the ones provided by GPT-4o, even with the same transcript and system prompts. Comparisons in Figure 5.3.

Timestamp: 00:44 Errore: Lo Studente 2 afferma che la TAC mostra un aneurisma e che 'si stenta a selezionare il renale'. Questa frase è ambigua e potrebbe indicare una difficoltà tecnica nella visualizzazione o nella pianificazione, ma non è chiaro se si riferisca a un problema reale o a una semplice osservazione. Non è un errore medico, ma una potenziale incomprensione comunicativa.
Timestamp: 01:35 Errore: Lo Studente 2 afferma che il colletto è di 52 millimetri e che 'tante protesi vanno bene per questo tipo di aneurisma'. Poco dopo, il Professore 1 chiede la lunghezza minima del colletto per un intervento endovascolare, e Cecilia risponde '10 millimetri, minimo minimo'. Se il colletto è effettivamente di 52 mm, è estremamente lungo e non rientra nella discussione sui '10 millimetri minimo minimo' o 'sopra i 15 millimetri' per la scelta della protesi. Questo suggerisce una possibile incomprensione o un errore nella misurazione o nella comunicazione della misura del colletto.
Timestamp: 02:45 Errore: Lo Studente 2 afferma che la renale sinistra 'sembra essere più bassa'. Il Professore 1 conferma: 'Sembra decisamente più bassa la sinistra'. Tuttavia, subito dopo, il Professore 1 dice: 'A radere non serve, perché abbiamo 12 millimetri. Non serve essere particolarmente aggressivi.' Non è chiaro a cosa si riferiscano i '12 millimetri' in questo contesto, né come si colleghino alla renale più bassa o alla necessità di 'radere'. C'è una mancanza di chiarezza nella comunicazione delle misure e delle implicazioni cliniche.

Figure 5.3: 3 errors using Gemini-2.5-flash as the LLM for Error Analysis

Timestamp: 00:44 Errore: Lo studente 2 afferma che 'si stenta a selezionare il renale' senza fornire ulteriori dettagli o soluzioni. Questo potrebbe indicare una mancanza di chiarezza o preparazione nella lettura della TAC.
Timestamp: 01:35 Errore: Lo studente 2 afferma che 'tante protesi vanno bene per questo tipo di aneurisma' senza specificare quali protesi siano più adatte. Mancanza di precisione nella scelta del dispositivo.
Timestamp: 02:47 Errore: Il professore 1 e lo studente 2 discutono sulla posizione della renale sinistra, ma inizialmente non concordano. Questo potrebbe indicare una mancata revisione accurata della TAC preoperatoria.

Figure 5.4: 3 errors using GPT-4o as the LLM for Error Analysis

Table 5.2 summarizes the differences between the models tested.

Table 5.2: Qualitative Evaluation of models for the Coordinator Role

Evaluation Criteria	GPT-4o	Gemini 2.5 Flash	Gemini 2.5 Pro	GPT-4o-mini
Executive Summary Quality	Good	Excellent	Excellent	Acceptable
Error Identification Accuracy	Good	Very High	Very High	Bad
Event Identification	High	High	High	Acceptable
Tool Call Reliability	High	High	High	Low
Cost / Performance	Mid	High	Mid	Low
Latency	Mid	High	Very High	Low
Recommendation	Coordinator (Recommended)	Coordinator	Coordinator	Only simple, focused tasks

5.4 Case Study

The case used for this chapter is a simulation of an EVAR procedure, involving one professor and two students. The simulation was recorded inside the University’s Laboratory of Augmented Health Environments, using multiple cameras to capture the operation and the state of the patient. The people involved included a performing surgeon as the Professor, and two students from the first year of Medical School as the two Students.

The description of the case, as explained by one of the student, is (translated to English from Italian):

‘He’s a patient with an infrarenal abdominal aortic aneurysm. It’s 44 millimeters, and there’s an indication for surgery. We’re doing an EVAR, which is an exclusion of the infrarenal aortic aneurysm’.

The team starts by explaining the case, what they have to do, and then they start the operation.

First, the team decided that the entry point was a ‘percutaneous femoral access’. Then, they decided on a prosthesis with a diameter of 26 mm and predefined lengths (138 mm for the right side, 134 mm for the left). They gave the patient 5000 units of heparin, and they execute the operation.

5.4.1 System Input

The input to the system is the audio from the simulation, in wav format, Extracted from the video of the simulation. The 52-minute audio from the EVAR simulation represents a realistic test case. It includes domain-specific terminology, overlapping speech, murmurs, and variable audio quality, excellent to test the robustness of the input pipeline.

The audio obtained is then processed as described in section 3.2, removing background noise, normalizing, and enhancing the speech with Demucs and SpeechBrain.

The exact process was:

1. Noise reduction with DeepFilterNet;
2. Normalization;
3. Volume boosting;
4. Voice enhancing with SpeechBrain and Demucs

The resulting audio is somewhat clear, but there are some parts where it’s hard to understand what is being said. This is a critical vulnerability of the ReflectOR system, as it needs clear audio to generate appropriate data. At least a central microphone that can capture what is being said is necessary, something that was missing in the cases used for the evaluation.

The audio was then transcribed and diarized using Gemini-2.5-pro. The result was acceptable.

For example, the first minute of transcription is:

[00:05] Professore: Alessandro? La registrazione? Ah, perfetto. Buono. Prego ragazzi. Che caso facciamo oggi?

[00:20] Studente 1: Facciamo un paziente con un aneurisma della aorta addominale sottorenale di 44 millimetri che ha indicazione all’intervento. Facciamo un EVAR, che è un’esclusione dell’aneurisma della aorta sottorenale.

[00:34] Professore: Va bene. Abbiamo visto il caso, controlliamo un attimo la TAC.

[00:44] Studente 1: La TAC mostra un aneurisma.

[00:49] Professore: Si estende in sezione laterale. Cecilia, cosa puoi dirmi su questo aneurisma? Mi sembra abbastanza standard. Il colletto com'è?

[01:00] Studente 2: Il colletto è lungo.

Which can be confronted to the ground truth, the transcription executed manually:

[00:05]: Professore: Alessandro? La registrazione? Ah perfetto, buono. Prego ragazzi. Allora, che caso facciamo oggi?

[00:20] Studente 1: Allora, facciamo... É un paziente con aneurisma della aorta addominale sottorenale di 44 millimetri che ha indicazione all'intervento. Facciamo un EVAR, che è un'esclusione dell'aneurisma della aorta sottorenale.

[00:34] Professore: Va bene. Abbiamo visto il caso, controlliamo un attimo la TAC.

[00:44] Professore: La TAC mostra un aneurisma, proiezione laterale.

[00:50] Professore: Cecilia, cosa puoi dirmi su questo aneurisma? Mi sembra un caso abbastanza standard, il colletto com'è?

[01:00] Studente 2: Il colletto è lungo.

It can be seen that the result closely matches the manual transcription. The only part where the transcription is not accurate is around the 44 seconds mark, where the microphone cuts off weirdly, making it almost impossible to understand what is being said even listening repeatedly manually.

It's not all wrong, however. For example, Gemini is able to understand this entire part:

Quindi bisogna ed è tutta ruvida all'esterno, quindi per evitare che il gratti sulla parete, bisogna prima portargli su il suo introduttore e poi lo rilasciamo. Quindi siamo lì. Portiamo su la gambetta.

When in the actual audio some parts are impossible to understand.

In comparison, OpenAI Whisper is able to give accurate timestamps for the words, but the transcription is much worse than Gemini.

GPT-4o-transcribe provided a seemingly good transcription, but it's inability to support word-level timestamps and diarization makes it unsuitable.

As a future work, it could be considered and evaluated a system where all three models (and even more) are executed, and the results are combined with a LLM to get the 'best of all worlds', taking the accurate Whisper word-level timestamps, with Gemini's diarization and GPT-4o transcription.

All the audio chunks are then joined together to form the full transcript, which is then passed through a filter. This filter removes useless lines that only consume tokens, such as lines with only 'Ok', 'Allora', etc. Additionally, the phrases spoken in succession by the same speaker are grouped together, to avoid wasting tokens.

For example:

[00:05] Professore: Alessandro?

[00:10] Professore: La registrazione? Ah, perfetto, buono.

[00:12] Professore: Dai.

[00:13] Professore: Prego ragazzi.

Becomes:

[00:05] Professore: Alessandro? La registrazione? Ah, perfetto, buono. Dai. Prego ragazzi.

5.4.2 System Output Analysis

After analyzing the transcription, which takes around 78 seconds using GPT-4o, a brief summary of the analysis result is generated.

Firstly, it contains a summary of the operation:

Descrizione del caso:

- Aneurisma sottorenale di 44 mm con colletto lungo e arterie iliache non coinvolte.
- Analisi preoperatoria della TAC per confermare il trattamento endovascolare come indicazione.

Preparazione della procedura:

- Pianificazione di proiezioni ottimali (e.g., 15° LAO e 8° caudale).
- Accesso percutaneo sul lato destro in base alle minori tortuosità anatomiche.
- Somministrazione di 5000 unità di eparina.

Fasi tecniche principali:

- Utilizzo di guide morbide e rigide per il posizionamento della protesi.
- Scelta una protesi da 26 mm con oversize del 15%.
- Posizionamento della protesi e rilascio con attenzione alla vicinanza delle arterie renali.
- Post-dilatazione con palloncini per il molding e verifica con angiografia.

Finalizzazione:

- Controllo per identificare eventuali leak tramite mezzo di contrasto.
- Rimozione delle guide e chiusura manuale degli accessi femorali.

Following this more info are included, such as the main timeline of events, main errors committed, the discussion plan, and the performance analysis. Which is a summary when compared to the full result. All these data is available inside the repository on GitHub, in the output folder.

The analysis provided is consistent with the transcript of the operation, and contains the key information that can be used to start the debriefing.

After the analysis of the transcript has been completed, the team can begin the debriefing. They can record the session through the apposite Record button in the web interface, and they can interact with the Coordinator through text or Speech to Text.

The coordinator can be useful to get more detail about the operation. For example, it can give more information about the patient:

> Dammi info sul paziente
> Informazioni sul paziente: Maria Francesca Rossi
Dati personali
ID paziente: PAZ-2024-001847
Nome completo: Maria Francesca Rossi
Data di nascita: 15 marzo 1987
Genere: Femmina
Cartella clinica n°: CSM-2024-MF-001847
... (previous operations, exams)
Immagini

Radiografia torace pre-operatoria
Descrizione: Sagoma cardiaca nella norma.
Path: link
... (more images)

The information are taken directly from the Patient Information database, so they are precise and accurate. The more information are available in the database, the more help can be provided by the Coordinator. Otherwise, it can give more information about the errors committed:

1. Pianificazione preoperatoria
00:49: Analisi superficiale del colletto dell'aneurisma
Il professore ha menzionato che il colletto è "lungo" ma non ha approfondito dettagli clinici cruciali riguardanti le sue dimensioni esatte o la qualità della parete. Questa mancanza di precisione potrebbe causare errori di pianificazione.
... (more errors)

Sintesi degli errori principali

1. Mancanza di coerenza e approfondimento nella pianificazione (colletto e renali).
2. Comunicazione non strutturata durante cambi guida e posizionamento degli strumenti.
3. Insufficienti considerazioni di sicurezza durante l'uso delle guide e del pallone di molding.
4. Approccio di verifica incompleto per il rilascio della protesi e per l'identificazione di eventuali leak.
5. Scarsa attenzione a soluzioni alternative tecnologiche per la chiusura chirurgica.

The list of error is consistent with the error analysis completed by the ErrorAnalysis sub-agent, and is accurate when checked manually through the timestamps provided by the complete analysis.

The Coordinator can ask for user input when needed, if it's unsure about a task, as seen in Figure 4.9.

Gemini-2.5-flash instead took 450 seconds, but the analysis generated is much more precise. First, the message given by the Coordinator contains information about the patient, summary of the operation, errors committed with explanations, materials analysis in a table, and the discussion plan.

Paziente

Nome: Maria Francesca Rossi

Data di nascita: 15 marzo 1987

Condizione: Aneurisma dell'aorta addominale sottorenale di 44 millimetri.

Anamnesi: Colectomia laparoscopica (10/03/2024), artroscopia ginocchio sinistro (22/06/2024), riparazione valvola mitralica (15/08/2024).

Riepilogo dell'Operazione

La paziente Maria Francesca Rossi è stata sottoposta a una discussione pre-operatoria e all'intervento di esclusione endovascolare (EVAR) per un aneurisma aortico di 44mm. La TAC ha rivelato un colletto lungo (52mm) ma angolato, con arteria renale sinistra più bassa e calcificazioni non circolari. È stata scelta una protesi Gore Excluder da 26mm (per un colletto di 21mm, con un oversizing del 15-20%) e gambette da 138mm (destra) e 134mm (sinistra).

La procedura ha incluso: accesso femorale percutaneo, somministrazione di eparina, inserimento e scambio di guide (morbida-rigida), posizionamento di introduttori e catetere pigtail. Sono state eseguite angiografie di repere per visualizzare l'anatomia e le renali. Successivamente, è stato posizionato e rilasciato il corpo principale della protesi, seguito dalla gambetta

controlaterale. La procedura si è conclusa con post-dilatazione (molding) con pallone, angiografia finale per la verifica e chiusura degli accessi con compressione manuale.

... (other parts of the analysis)

The summary is still accurate when compared to the transcript, but the descriptions and the information are more detailed.

Based on the results, Gemini-2.5-flash is more detailed and precise, generating a better result with more detailed description compared to 4o. The cost for the accuracy is the increase in waiting time (70 seconds for GPT-4o, 400 seconds for Gemini).

The full results are available in the GitHub repository.

5.4.3 PDF Report

After the debriefing has been completed, the system generates a PDF report containing the full analysis, the related memories, the conversation that happened with the Coordinator, the expenses graph, and the full, raw output from the analysis.

The summary starts with a description about the patient, followed by the description of the simulated operation. There is a list of steps on everything that was done during the operation, followed by previous information about the patient. Figure 5.5 shows an example of the summary section.

The summary is very complete and accurate, describing everything that is needed to know during the debriefing or when examining previous documents.

The timeline (Figure 5.6) contains all the main events that occurred during the simulation, with a brief but complete description for all of them.

Figure 5.7 shows a part of the error list, with the description of the error and who committed it, and the timestamp to check in the actual video during the debriefing.

This PDF represents a piece of documentation based on actual data, providing structured and data-rich information for the debriefing conversation, instead of relying on human memory. Additionally, it can be archived and used for reference in the future.

5.5 Performance and Trade-offs

The **accuracy** of the system is highly dependent on the transcription obtained from the input processing pipeline, as mentioned in previous sections.

If the transcript is accurate to the real operation, the accuracy is very high, otherwise, it may commit more errors.

For examples, the transcription model sometimes could output ‘20 LAO e 8 gradi caudale’ instead of ‘15 LAO e 8 gradi caudale’, which would generate a wrong event, and possibly create inconsistencies with following measurements and generate a non-existing error.

The materials list was accurate, based on the transcript, but the price given were not, as they were only based on the model’s knowledge. Additional information about prices can be given separately for better accuracy.

As mentioned during this chapter, there are **trade-offs** between the precision of the model, the latency, and even the cost.

Gemini-2.5-pro, while very accurate, is much slower and pricier than other options, such as the flash version, or the GPT counterpart.

For reference, as of September 2025, the input price for the pro version is up to 2.5 times more expensive, while the price for output tokens is up to 5 times more expensive than the flash model.

The choice depends on the requirements. The pricier option might still be acceptable when considering how important accuracy is in a surgical setting, and the latency trade-off might be acceptable, as it can be set-up to execute automatically after an operation. In addition, parallel execution can be implemented to reduce the latency drastically.

5.6 Surgeons’ Evaluation

Given the specialized nature of the problem domain, the qualitative evaluation was performed based on feedback from surgeons, to validate the clinical utility and correctness of ReflectOR’s output. The feedback was received

1. Riassunto

La trascrizione documenta la discussione e simulazione di un intervento endovascolare (EVAR) per la riparazione di un aneurisma dell'aorta addominale sottorenale di 44 mm. Il caso viene analizzato nei dettagli dai partecipanti, tra cui un professore e vari studenti, attraverso una valutazione della TAC e una pianificazione chirurgica preoperatoria. Di seguito il riassunto completo:

Analisi del Caso

- **Aneurisma:** Localizzato a livello dell'aorta addominale sottorenale, con un colletto lungo e senza coinvolgimento delle arterie iliache. Non presenta calcificazioni circonfenziali significative.
- **Planning:** Si decide per un intervento endovascolare aorto-bisiliaco. La proiezione operatoria più corretta indicata è di 15° LAO e 8° caudale. Le arterie renali si trovano a livelli differenti, con la sinistra più bassa, ma non richiedono un approccio aggressivo.
- **Misurazioni:** Colletto di 52 mm; protesi da applicare con diametro di 26 mm (oversizing del 15%) e lunghezze planning di circa 138 mm a destra e 134 mm a sinistra.

Preparazione e Procedura

1. **Accessi femorali:** Percutanei.
2. **Somministrazione eparina:** Dose di 5000 unità.
3. **Guida e Catetere:**
 - Viene inserita una guida morbida per minimizzare traumi vascolari durante la risalita.
 - Successivamente, si usa una guida rigida per fornire stabilità durante l'inserimento della protesi.
4. **Protesi:**
 - Scelta di una protesi con oversizing. La posizione e l'apertura sono calibrate per il colletto e l'ipogastrica, evitando distorsioni anatomiche.
 - Si discute il posizionamento delle gambette "a ballerina" per minimizzare problemi di incrocio tra le guide.
5. **Angiografie di controllo:**
 - Viene eseguita un'angiografia per visualizzare la posizione delle arterie renali e verificare il posizionamento della protesi.
 - Si utilizza un pig per misurazioni e ulteriori verifiche del sistema.
6. **Montaggio pallone:**
 - Il pallone viene gonfiato per assicurare il molding e la corretta sigillatura della protesi senza leak.
 - L'angioplastica viene eseguita con il pallone Reliance, utilizzando il dry seat per maggiore stabilità.

Conclusione

- L'intervento di simulazione si conclude con la rimozione delle guide e dei cateteri dopo verifica finale con angiografia per confermare l'assenza di perdite (leak) e l'adeguata esclusione dell'aneurisma.
- Compressione manuale finale per chiudere gli accessi femorali.

Informazioni del Paziente

Il paziente Maria Francesca Rossi ha un'anamnesi chirurgica articolata (riparazione valvola mitralica, artroscopia menisco e colecistectomia laparoscopica) e il quadro diagnostico mostra che la TAC precedente non aveva evidenziato un aneurisma, ma ispessimenti della colecisti e calcoli biliari.

Il caso simulato rappresenta un intervento standard per un paziente con aneurisma e dimostra una coordinazione di squadra focalizzata sulla pianificazione accurata e sull'esecuzione delle procedure nel rispetto dell'anatomia del paziente e delle caratteristiche della protesi.

Figure 5.5: Example summary section in the PDF for the simulated EVAR operation

2. Cronologia degli eventi

Timestamp: 00:20 Evento: Identificazione del caso clinico: aneurisma aorta addominale sottorenale da trattare con EVAR.
Timestamp: 00:34 Evento: Analisi del colletto dell'aneurisma: conferma che è lungo e standard per la procedura.
Timestamp: 01:10 Evento: Conferma che le iliache non sono coinvolte nell'aneurisma.
Timestamp: 01:36 Evento: Discussione delle misure del colletto: 52 mm, compatibile con più protesi.
Timestamp: 02:19 Evento: Identificazione della proiezione preoperatoria: 15° LAO e 8° caudale.

Figure 5.6: Example of a part of the timeline section in the PDF for the simulated EVAR operation

3. Analisi degli errori

I seguenti errori e incomprensioni sono stati identificati

Timestamp: 01:36 Errore: Studente 1 riferisce che il colletto è di 52 millimetri, apparentemente senza aver verificato accuratamente la misura nella TAC o nei dati disponibili, portando a una potenziale imprecisione nella preparazione del caso.
Timestamp: 03:28 Errore: Studente 1 indica che il lato migliore per l'inserimento della protesi è la destra per via della tortuosità inferiore, ma questa affermazione viene contraddetta dal professore, che sottolinea l'anatomia permissiva e argomenta la comodità per l'operatore a destra.
Timestamp: 05:03 Errore: Studente 1 inizialmente non riconosce che la somministrazione di eparina dovrebbe essere effettuata prima di procedere con ulteriori interventi. Deve essere corretto dal Professore per procedere correttamente.

Figure 5.7: Example of a part of the Error Analysis section in the PDF for the simulated EVAR operation

through an informal presentation, not a full-scale clinical trial. Future work should include a larger sample size for a definite assessment.

The system was presented to two surgeons from the Santa Chiara Hospital in Trento: Sara Allievi and Stefano Bonvini. The presentation illustrated the system's features, including a live demo, to receive feedback from actual doctors on whether the system is useful.

The surgeons approved of the clinical utility in identifying and listing the materials used during the operation, and the system's ability to answer a user's question based on the analysis of the operation.

They expressed interest in the possibility of the system being able to write a complete report, providing a template on the form they use to write all information during and after a surgical operation, simplifying their workflow.

Furthermore, they found useful the PDF report created by the system, which can be used as documentation for future debriefing, and the simplicity and completeness for the web interface.

5.7 In-depth Analysis of Testing and Troubleshooting

This section describes some of the troubleshooting required during the transcription phase, to assure that it was as accurate as possible, testing combination of models and parameters to achieve the best results.

Whisper `Whisper` was found to be quite accurate normally, but it comes with a limit of 25 MB for the audio file. This is problematic because after processing the audio so that it's higher quality, it is often well above that threshold. This means that the file has to be split into parts to be transcribed separately, which diminishes the accuracy.

Additionally, `Whisper` is not without problems, for example the following is a part of a transcription created by that model:

e poi si sembrato assenza preciso e perciò funziona kaeyano questa Tweete mi ci curve a retrarre ancora di più perchè forse non ritireto il resto ed e seni comunque Katietico ianfartevo spazio non noi non non nowhere mio satellore signatori ma cugasse ma ma ma mio st stiffness come come come come come come come non io io io io che che che che che che is buona iri iri iri che che da un paio d'anni l'acqua Oggi più semplice è?

It's clear that this makes absolutely no sense. The transcription was riddled with paragraphs such as these when the audio wasn't completely clear, and sometimes even when the audio was clear.

Regardless of this, the models was quite accurate in other parts. For example, this was the result of the first minute of audio on one of the executions:

Alessandro, la registrazione? Perfetto, suono. Allora che caso facciamo oggi?
Facciamo un paziente con aneurisma della orta addominale sottorenale di 44 mm che ha indicazione all'intervento. Facciamo un EVAR che è un'esclusione dell'aneurisma della orta sottorenale.
Va bene, abbiamo visto il caso, controlliamo un attimo l'ATTAC. L'ATTAC mostra l'aneurisma,

...

L'attacca. Cecilia, cosa vuoi dirmi su questo aneurisma? È abbastanza standard. L'ho letto com'è? L'ho letto...

These results are close to OpenAI's Whisper cloud model, even if less accurate, and they support word-level timestamps. For real deployment, this could be a good option to keep the recording local.

Gemini Another option for transcription was Gemini. This LLM developed by Google, unlike GPT, was trained from the ground-up to be multi-modal, accepting images and audio in addition to text.

Because of this, these models have a great understanding of audio files, being able to natively transcribe and timestamp each phrase, even assigning them to the speaker when giving the names.

Gemini-2.5-flash immediately proved effective, generating great results such as:

[00:05] Alessandro: Alessandro?
[00:10] Professore 1: Ah, perfetto. Buono.
[00:13] Professore 1: Prego ragazzi.
[00:15] Professore 1: Eh?
[00:18] Professore 1: Allora, che caso facciamo oggi?
[00:20] Studente 2: Facciamo un paziente con aneurisma dell'aorta addominale sottorenale.
[00:25] Studente 2: Di 44 mm che ha indicazione all'intervento.
[00:29] Studente 2: Facciamo un EVAR che è un'esclusione dell'aneurisma dell'aorta sottorenale.
[00:34] Professore 1: Va bene. Abbiamo visto il caso.
[00:36] Professore 1: Controlliamo un attimo la TAC.
[00:44] Professore 1: La TAC mostra un aneurisma.
[00:49] Professore 1: C'è un'estensione laterale. Cecilia, cosa puoi dirmi su questo aneurisma?
[00:55] Studente 1: È abbastanza standard.
[00:58] Professore 1: Il colletto com'è?

Even the speaker diarization works quite well, assigning to each phrase the correct speaker, even if it did produce some repeated words on some parts of the audio. The parameters used for this result were: temperature 0.75 and top_p 0.5.

This seemed to be the best result yet, considering it also did diarization automatically, removing an unstable component. Since this had such great results, Gemini-2.5-pro was tested, expecting even better results.

Instead, the pro version seemed to have some problems. It would give HTTP error 500 when requested. After some debugging, it was discovered that it was because of the model running out of context due to the long audio. Like for the GPT version, the audio was then split before sending it to Gemini in chunks. In particular, chunks of 4 minutes each were used. This resolved the issue, but introduced the same problem as GPT-4o, where it would loop on some words.

For example, one of the outputs received was:

[21:24] Professore 1: Hai messo che so di... sì, sì.
[21:27] Studente 1: Ok.
[21:31] Professore 1: Sfil
[21:31] Professore 1: Sfil
[21:31] Professore 1: Sfil
[21:31] Professore 1: Sfil ...

After tweaking the parameters, setting the temperature to 0.75, top_p to 0.5 (Gemini-2.5 doesn't support frequency_penalty), the output seemed to improve:

[00:05] Professore 1: Alessandro.
[00:09] Professore 1: La registrazione?
[00:10] Professore 1: Ah, perfetto. Buono.
[00:13] Professore 1: Prego ragazzi.
[00:18] Professore 1: Che caso facciamo oggi?
[00:20] Martin: Facciamo un paziente con un aneurisma della aorta addominale sottorenale.
[00:25] Martin: Di 44 millimetri che ha indicazione all'intervento.
[00:29] Martin: Facciamo un EVAR, che è un'esclusione dell'aneurisma della aorta sottorenale.

Chunking The main issue was that due to using chunks, the model wouldn't have context about the previous chunk, so the timestamp and the knowledge would be reset, losing on accuracy.

To solve this issue, the previous transcription was given to the model when transcribing the audio, so that it can continue from the previous audio.

Chunking also resolved the issue the model produced with the temperature set to 0, where it would generate repetitions after around 20 minutes. Dividing into chunks of 4 minutes, Gemini-2.5-pro gave an average WER of 11.53%, with a standard deviation of 1.00775. The DER, instead, had an average of only 1.41%, with a standard deviation of 1.

5.8 Chapter Summary

In summary, the system is accurate when providing a clean and high-quality audio, while it can have mistakes when the audio cannot be transcribed accurately.

The model that works the best for transcription is Gemini-2.5-pro with 11.6% in WER, and 7.84% DER. It is, however, much slower and expensive than other models. Another acceptable option is Gemini-2.5-flash, which is 4 times faster than the pro version, while maintaining an acceptable accuracy of 20.8%. Local models, instead, prove to be less accurate.

For the analysis, all models tested were acceptable, with Gemini-2.5-flash being more descriptive and detailed compared to GPT-4o.

Nevertheless, qualitative feedback from surgeons confirmed the potential usefulness and utility of the system in a real-world clinical setting.

6 Conclusions & Future Work

This chapter concludes the thesis, summarizing the primary contributions, the evaluation results, and how the research questions were explored and answered. Additionally, it outlines the main limitations for the current ReflectOR system, proposing solutions and directions for future development.

Contributions and Results This thesis introduced the design, implementation, and evaluation of the ReflectOR system, which applied an agentic system to the problem domain of post-operative surgical debriefings.

The analysis revealed that the system is highly dependent on the input audio, as small mistakes in the transcription are propagated through the entire workflow, but evaluation highlighted the system’s reliability to noisy audio, showing high accuracy.

Quantitative results placed Gemini-2.5-pro in first place for accuracy and precision, with WER of 11.6% and DER of 7.84%. Following closely behind is Gemini-2.5-flash, which has a higher WER of 24.1% and DER of 5.88%, but with a 4 times lower latency compared to the pro model, and a price up to 5 times lower.

The system was able to generate coherent and accurate reports, identifying key events, listing errors committed by team members, listing materials used, and analyzing the team performance, as confirmed by the qualitative evaluation.

Reflections and Limitations This work also answered the initial research questions. It was explained why key components for an agentic system in this domain require a ReAct planning module, a long-term memory, and a multi-agent architecture, orchestrated through a framework such as LangGraph. Furthermore, it was explained why simpler architectures like a RAG pipeline wouldn’t be enough. It was demonstrated why a multi-agent architecture is better suited than a regular monolithic agent, allowing the decomposition of complex tasks into smaller sub-problems.

Additionally, it highlighted ethical implications for deploying such a system when it can affect human behavior. In particular, the system could create an effect known as Hawthorne effect, where people behave differently from normal due to the knowledge of being recorded. In alternative, people could become too reliant on the system, paying less attention since they’re aware that the system is going to complete the data-related work for them.

Despite the effectiveness and potential of the system, there are some **issues** and **limitations** that need to be considered. The current ReflectOR prototype includes minimal security and privacy measures due to its usage in simulated environments.

Firstly, **data privacy and security** present significant limitations. For real clinical cases, a robust system to anonymize and encrypt data both at rest and when they are sent to external services is required. Additionally, external services are not reliable both in terms of privacy, security, and availability, since they can stop responding. This could be solved by using local softwares and models, but this option also has weaknesses. First, it would require the availability of a local server able to host those softwares, which would require significant computational cost for running state-of-the-art LLMs. And, even then, the performance and accuracy of local models are worse compared to cloud models, as demonstrated in the Evaluation chapter.

If the system has access to data about real patients and medical staff, an advanced **authentication** and **authorization** system would need to be implemented to avoid unauthorized people having access to sensitive data.

Furthermore, a critical issue would be **Prompt Injection**. This vulnerability affects all systems based on LLMs, and happens when an attacker designs a prompt carefully crafted to make the LLM execute actions against their guidelines. Multi-billion dollars companies have yet to solve this issue, so it’s still something to be very careful of, as it could allow attackers to access sensitive information.

Finally, the **evaluation** was conducted on simulated EVAR surgeries. While valuable, these simulations may not fully represent the complexity and variability of real surgeries. It doesn’t take into account the unpredictable events that may happen, the higher stress levels, the background noise from the equipment, and the presence

of overlapping conversations. Current metrics like WER and DER only capture the technical accuracy of the input, they don't measure the semantic quality of the debriefing itself.

Future Work There are multiple directions for future work to improve this system.

A novel approach could be used by combining results from **multiple transcription models** to achieve a more accurate and precise transcription, especially for noisy audio. Additionally, implementing more advanced noise reduction and speech enhancement techniques to improve noisy and unclear audios is important to improve the processing of noisy and unclear audio inputs.

Furthermore, some **new information** could be gathered by the same audio used for the transcription, such as Emotion and Tone Recognition to identify moments of particular stress. This could be useful to analyze how the team members perform under stress, if after doing one error they are more likely to commit more errors, or if they perform well under stress. Another use could be to analyze the professor's teaching method. Whether he's able to guide the students well, or if he loses his temper.

To overcome the **privacy and cost** limitations mentioned in the previous section, hosting private local models should be considered. As mentioned before, this would require hardware capable of running state-of-the-art models, but this can be offset by recent advancement in LLM optimization. Two techniques are particularly interesting:

- Mixture of experts, which uses multiple "experts" (sub-agents) to improve the quality of the LLM. A router (a small neural network) handles the tokens, sending them selectively to parts of the LLM, so that only certain neurons have to be activated based on the task. For example, one part of the LLM is used for math, one for code, another for creative writing, etc. This architecture improves efficiency and scalability, since only a part of the model has to be activated based on the task at hand.

This technique reduces drastically the requirements needed in terms of GPU power to run high-end LLMs.

- Quantization, which involves the compression of the model's weights. Normally, the weights are saved in BF16 precision, requiring a large amount of memory in exchange for high precision. With quantization, the weights are saved in lower precision representations, such as using 8-bits integers, or even 4-bits integers. This reduces resource consumption at the expense of accuracy in the answers.

For real-world implementation and deployment, robust and secure safety and privacy systems should be implemented, to prevent bad actors to access sensitive data, and to be able to work with external service without exposing private information. Possibly, integrating with the Sistema Informativo Ospedaliero (SIO) to have access to real patient's information.

Following the surgeons' request, a high priority addition should be **templated report generation** support, allowing to populate pre-defined forms with the information generated from the analysis, streamlining clinical documentation workflows, and allowing ReflectOR to improve its own output based on the user's request, after generating the report. A possible approach would be a human-in-the-loop workflow, where the agent generates a draft report based on the templates, and the surgeons can review, edit and approve it from the web interface.

Furthermore, **long term memory** should be enhanced, improving the agent's ability to learn from past mistakes, keep track and consider past debriefings and operations at an individual level. For example, the performance agent could have its dedicated memory that only focuses on performances for each individuals, while the error agent could integrate surgical protocols to better identify protocol deviations, and the discussion agent could store user's preferences to keep the tone and learn from past debriefings.

Then, more architecture should be explored, including more advanced and stateful sub-agents, reducing the amount of work that the Coordinator needs to perform.

Error handling should be enhanced, to allow fallbacks in case of problems with one service, e.g. using a different LLM provider if one fails, and graceful degradation to ensure the user is notified when something is wrong.

More extensive benchmarks and qualitative evaluation should be performed on different models and software, to find the best one for the job. For example, performing a comparative study between teams with control groups, including surveys about the results, and actionable improvement points identified by each team.

Finally, the web interface should be improved to allow more interactive tools for modifying extracted data, a more seamless conversation with the Coordinator, and to allow the input of templates for the Coordinator to compile.

6.1 Concluding Remarks

In conclusion, ReflectOR demonstrates how a system based on LLM agents is able to address critical challenges in healthcare, improving post-operative clinical workflows. This thesis proves the feasibility of an AI system able to create structured and data-rich post-operative debriefings. It highlights the ability to improve communication and identify errors. While current limitations require further development and research, ReflectOR lies the foundation for future AI systems in clinical settings.

Bibliography

- [1] Steve Alder. Effects of poor communication in healthcare. *The HIPAA Journal*, April 2025.
- [2] AF Arriaga, YK Chen, MPT Pimentel, AM Bader, and D Szyld. Critical event debriefing: a checklist for the aftermath. *Current Opinion in Anesthesiology*, 34(6):744–751, dec 2021.
- [3] Max Bain, Jaesung Huh, Tengda Han, and Andrew Zisserman. Whisperx: Time-accurate speech transcription of long-form audio. *INTERSPEECH 2023*, 2023.
- [4] Harrison Chase. LangChain, October 2022.
- [5] dvgodoy. Deep Learning Visuals.
- [6] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners, 2023.
- [7] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2021.
- [8] Jerry Liu. LlamaIndex, November 2022.
- [9] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. Agentbench: Evaluating llms as agents, 2023.
- [10] Yanni A Loukissas, Jason Maron, Marco Zenati, and David Mindell. Redesigning Postoperative Review. In *IEEE Healthcare Technology Conference Proceedings*, Houston, 2012. IEEE.
- [11] Amr Mohamed, Yang Zhang, Michalis Vazirgiannis, and Guokan Shang. Lost in the mix: Evaluating llm understanding of code-switched text, 2025.
- [12] Runliang Niu, Jindong Li, Shiqi Wang, Yali Fu, Xiyu Hu, Xueyuan Leng, He Kong, Yi Chang, and Qi Wang. Screenagent: A vision language model-driven computer control agent. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, IJCAI-2024, page 6433–6441. International Joint Conferences on Artificial Intelligence Organization, August 2024.
- [13] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023.
- [14] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023.
- [15] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023.
- [16] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023.

- [17] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments, 2024.
- [18] Yilun Yang and Yekun Chai. Evaluating code-mixing in llms across 18 languages, 2025.
- [19] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models, 2023.
- [20] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023.
- [21] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents, 2024.

Glossary

context window The amount of tokens a model can process at one time, includes both user's input and the output generated previous by the model itself.. 5

EVAR Endovascular Aneurysm Repair (EVAR), surgical operation to treat abdominal aortic aneurysms.. 21

few-shots prompting Give a few examples on how the model should answer inside the prompt. 4

fine-tune Re-train a pre-trained model on a specific dataset to perform better on a specific task.. 4

Google Gemini LLM model developed by Google. 2

hallucination Happens when the LLM generates an output that is not based on the training data, doesn't conform to the instructions, is nonsensical or inaccurate.. 5

LLM Large Language Model. Typically based on the Transformer architecture, it's a deep learning model capable of understanding and generating text. 2, 4

LLM Agent A system that uses a LLM as the core engine, possessing a memory module, a planning module, and a tool module to perform various actions in the environment.. 2

mixture of experts The LLM's network is divided into 'sections' (experts), each handling some specific tasks, such as math, coding, or creative writing.. 70

OpenAI GPT LLM model developed by OpenAI. 2

parameters The processing units inside each layer of the transformer architecture. 4

prompt engineering Detailing how the model should respond and its expertise by giving instructions in the System Prompt. 4

prompt injection An attack where the user designs a prompt to make the agent behave in ways against its own programming, such as revealing sensitive and private information.. 53

quantization Compression for the weights of a model, reducing the hardware resources required.. 70

Retrieval-Augmented Generation Before answering a query by the user, the model queries connected data sources such as documents, search engines, or databases, grounding the answer to real information.. 2

Self-Attention Allows a model to process the relationship between a word and all the other words simultaneously regardless of their distance in the text. It's what enables a model to consider the meaning of each word based on its context within the input.. 4

System Prompt The fundamental instructions that dictate how the model acts and responds. It can define the LLM's role, expertise, and the tone it should use when answering.. 4

token A chunk of text that the LLM can process. 4

zero-shot prompting Only detail what the model should do, without any examples. 4